

Chapter 6. I/O system

The Windows I/O system consists of several executive components that, together, manage hardware devices and provide interfaces to hardware devices for applications and the system. This chapter lists the design goals of the I/O system, which have influenced its implementation. It then covers the components that make up the I/O system, including the I/O manager, Plug and Play (PnP) manager, and power manager. Then it examines the structure and components of the I/O system and the various types of device drivers. It discusses the key data structures that describe devices, device drivers, and I/O requests, after which it describes the steps necessary to complete I/O requests as they move through the system. Finally, it presents the way device detection, driver installation, and power management work.

I/O system components

The design goals for the Windows I/O system are to provide an abstraction of devices, both hardware (physical) and software (virtual or logical), to applications with the following features:

- Uniform security and naming across devices to protect shareable resources. (See [Chapter 7](#), “[Security](#),” for a description of the Windows security model.)
- High-performance asynchronous packet-based I/O to allow for the implementation of scalable applications.
- Services that allow drivers to be written in a high-level language and easily ported between different machine architectures.
- Layering and extensibility to allow for the addition of drivers that transparently modify the behavior of other drivers or devices, without requiring any changes to the driver whose behavior or device is modified.
- Dynamic loading and unloading of device drivers so that drivers can be loaded on demand and not consume system resources when unneeded.
- Support for Plug and Play, where the system locates and installs drivers for newly detected hardware, assigns them hardware resources they require, and allows applications to discover and activate device interfaces.
- Support for power management so that the system or individual devices can enter low-power states.
- Support for multiple installable file systems, including FAT (and its variants, FAT32 and exFAT), the CD-ROM file system (CDFS), the Universal Disk Format (UDF) file system, the Resilient File System (ReFS), and the Windows file system (NTFS). (See Chapter 13, “File systems,” in Part 2 of this book for more specific information on file system types and architecture.)
- Windows Management Instrumentation (WMI) support and diagnosability so that drivers can be managed and monitored through WMI applications and scripts. (WMI is described in Chapter 9, “Management mechanisms,” in Part 2.)

To implement these features, the Windows I/O system consists of several executive components as well as device drivers, which are shown in [Figure 6-1](#).

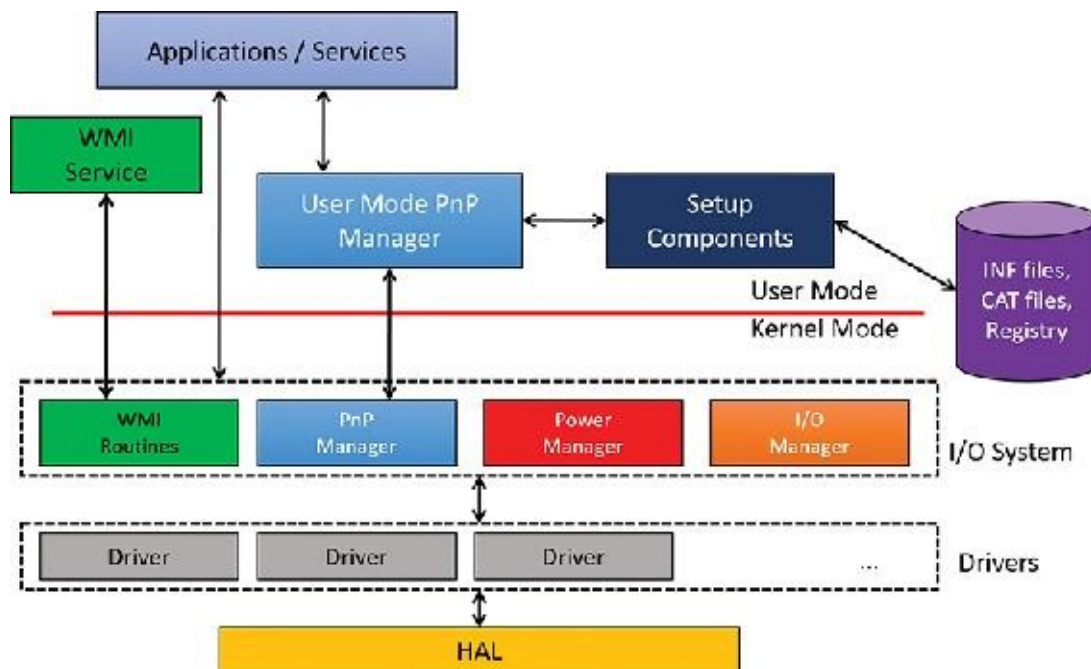


FIGURE 6-1 I/O system components.

- The I/O manager is the heart of the I/O system. It connects applications and system components to virtual, logical, and physical devices, and it defines the infrastructure that supports device drivers.
- A device driver typically provides an I/O interface for a particular type of device. A *driver* is a software module that interprets high-level commands, such as read or write commands, and issues low-level, device-specific commands, such as writing to control registers. Device drivers receive commands routed to them by the I/O manager that are directed at the devices they manage, and they inform the I/O manager when those commands are complete. Device drivers often use the I/O manager to forward I/O commands to other device drivers that share in the implementation of a device's interface or control.
- The PnP manager works closely with the I/O manager and a type of device driver called a *bus driver* to guide the allocation of hardware resources as well as to detect and respond to the arrival and removal of hardware devices. The PnP manager and bus drivers are responsible for loading a device's driver when the device is detected. When a device is added to a system that doesn't have an appropriate device driver, the executive Plug and Play component calls on the device-installation services of the user-mode PnP manager.
- The power manager also works closely with the I/O manager and the PnP manager to guide the system, as well as individual device drivers, through power-state transitions.
- WMI support routines, called the Windows Driver Model (WDM) WMI provider, allow device drivers to indirectly act as providers, using the WDM WMI provider as an intermediary to communicate with the WMI service in user mode.
- The registry serves as a database that stores a description of basic hardware devices attached to the system as well as driver initialization and configuration settings. (See the section "The registry" in Chapter 9 in Part 2 for more information.)
- INF files, which are designated by the .inf extension, are driver-installation files. INF files are the link between a particular hardware device and the driver that assumes primary control of that device. They are made up of script-like instructions describing the device they correspond to, the source and target locations of driver files, required driver-installation registry modifications, and driver-dependency information. Digital signatures that Windows uses to verify that a driver file has

passed testing by the Microsoft Windows Hardware Quality Labs (WHQL) are stored in .cat files. Digital signatures are also used to prevent tampering of the driver or its INF file.

- The hardware abstraction layer (HAL) insulates drivers from the specifics of the processor and interrupt controller by providing APIs that hide differences between platforms. In essence, the HAL is the bus driver for all the devices soldered onto the computer's motherboard that aren't controlled by other drivers.

The I/O manager

The I/O manager is the core of the I/O system. It defines the orderly framework, or model, within which I/O requests are delivered to device drivers. The I/O system is packet driven. Most I/O requests are represented by an I/O request packet (IRP), which is a data structure that contains information completely describing an I/O request. The IRP travels from one I/O system component to another. (As you'll discover in the section "[Fast I/O](#)," fast I/O is the exception; it doesn't use IRPs.) The design allows an individual application thread to manage multiple I/O requests concurrently. (For more information on IRPs, see the section "[I/O request packets](#)" later in this chapter.)

The I/O manager creates an IRP in memory to represent an I/O operation, passing a pointer to the IRP to the correct driver and disposing of the packet when the I/O operation is complete. In contrast, a driver receives an IRP, performs the operation the IRP specifies, and passes the IRP back to the I/O manager, either because the requested I/O operation has been completed or because it must be passed on to another driver for further processing.

In addition to creating and disposing of IRPs, the I/O manager supplies code that is common to different drivers and that the drivers can call to carry out their I/O processing. By consolidating common tasks in the I/O manager, individual drivers become simpler and more compact. For example, the I/O manager provides a function that allows one driver to call other drivers. It also manages buffers for I/O requests, provides timeout support for drivers, and records which installable file systems are loaded into the operating system. There are about 100 different routines in the I/O manager that can be called by device drivers.

The I/O manager also provides flexible I/O services that allow environment subsystems, such as Windows and POSIX (the latter is no longer supported), to implement their respective I/O functions. These services include support for asynchronous I/O that allow developers to build scalable, high-performance server applications.

The uniform, modular interface that drivers present allows the I/O manager to call any driver without requiring any special knowledge of its structure or internal details. The operating system treats all I/O requests as if they were directed at a file; the driver converts the requests from requests made to a virtual file to hardware-specific requests. Drivers can also call each other (using the I/O manager) to achieve layered, independent processing of an I/O request.

Besides providing the normal open, close, read, and write functions, the Windows I/O system provides several advanced features, such as asynchronous, direct, buffered, and scatter/gather I/O, which are described in the "[Types of I/O](#)" section later in this chapter.

Typical I/O processing

Most I/O operations don't involve all the components of the I/O system. A typical I/O request starts with an application executing an I/O-related function (for example, reading data from a device) that is processed by the I/O manager, one or more device drivers, and the HAL.

As mentioned, in Windows, threads perform I/O on virtual files. A virtual file refers to any source or

destination for I/O that is treated as if it were a file (such as devices, files, directories, pipes, and mailslots). A typical user mode client calls the `CreateFile` or `CreateFile2` functions to get a handle to a virtual file. The function name is a little misleading—it’s not just about files, it’s anything that is known as a *symbolic link* within the object manager’s directory called `GLOBAL??`. The suffix “File” in the `CreateFile*` functions really means a virtual file object (`FILE_OBJECT`) that is the entity created by the executive as a result of these functions. [Figure 6-2](#) shows a screenshot of the WinObj Sysinternals tool for the `GLOBAL??` directory.

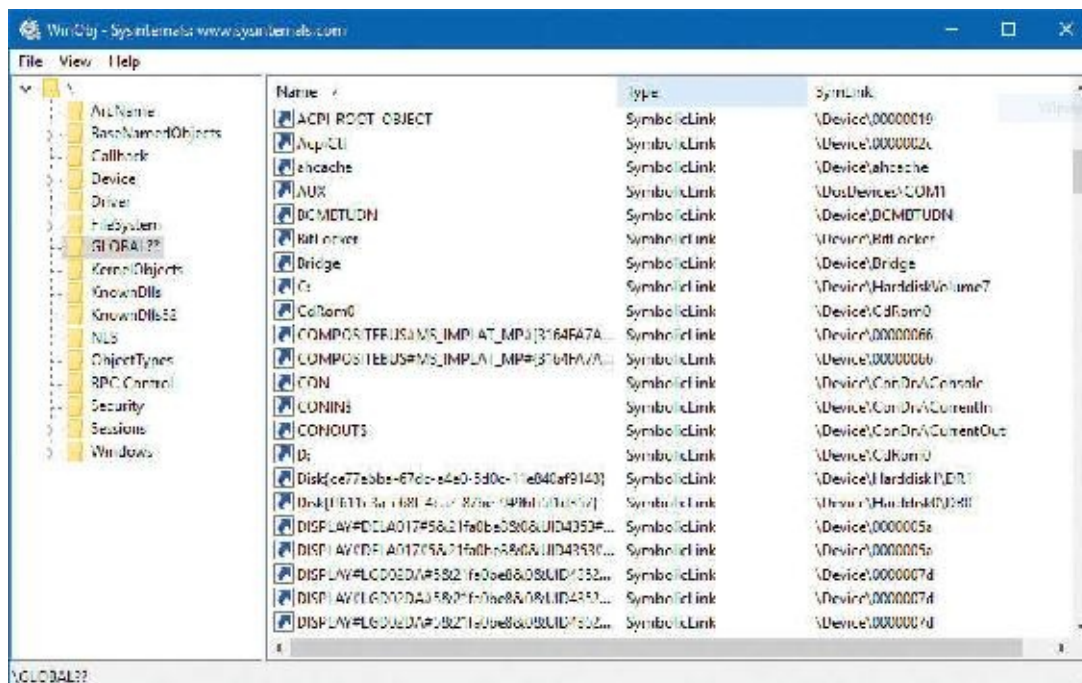


FIGURE 6-2 The object manager’s `GLOBAL??` directory.

As shown in [Figure 6-2](#), a name such as `C:` is just a symbolic link to an internal name under the Device object manager directory (in this case, `\Device\HarddiskVolume7`). (See Chapter 8, “System mechanisms,” in Part 2 for more on the object manager and the object manager namespace.) All the names in the `GLOBAL??` directory are candidates for arguments to `CreateFile(2)`. Kernel mode clients such as device drivers can use the similar `ZwCreateFile` to obtain a handle to a virtual file.



Note

Higher-level abstractions such as the .NET Framework and the Windows Runtime have their own APIs for working with files and devices (for example, the `System.IO.File` class in .NET or the `Windows.Storage.StorageFile` class in WinRT), but these eventually call `CreateFile(2)` to get the actual handle they hide under the covers.



Note

The `GLOBAL??` object manager directory is sometimes called `DosDevices`, which is an older name. `DosDevices` still works because it’s defined as a symbolic link to `GLOBAL??` in the root of the object manager’s namespace. In driver code, the `??` string is typically used to reference the `GLOBAL??` directory.

The operating system abstracts all I/O requests as operations on a virtual file because the I/O manager has no knowledge of anything but files, therefore making it the responsibility of the driver to translate file-oriented comments (open, close, read, write) into device-specific commands. This abstraction thereby generalizes an application's interface to devices. User-mode applications call documented functions, which in turn call internal I/O system functions to read from a file, write to a file, and perform other operations. The I/O manager dynamically directs these virtual file requests to the appropriate device driver. [Figure 6-3](#) illustrates the basic structure of a typical I/O read request flow. (Other types of I/O requests, such as write, are similar; they just use different APIs.)

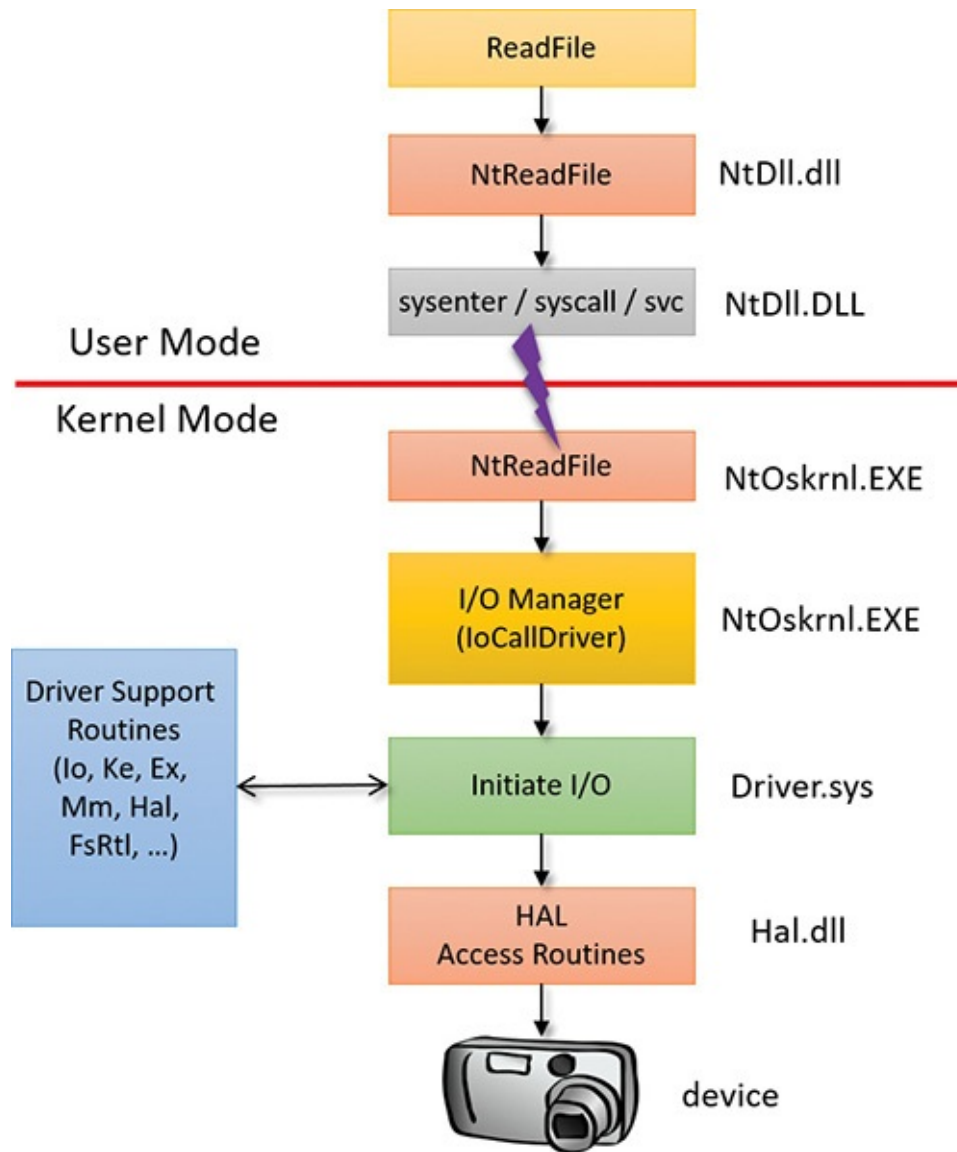


FIGURE 6-3 The flow of a typical I/O request.

The following sections look at these components more closely, covering the various types of device drivers, how they are structured, how they load and initialize, and how they process I/O requests. Then we'll cover the operation and roles of the PnP manager and the power manager.

Interrupt Request Levels and Deferred Procedure Calls

Before we proceed, we must introduce two very important concepts of the Windows kernel that play an important role within the I/O system: Interrupt Request Levels (IRQL) and Deferred Procedure Calls (DPC). A thorough discussion of these concepts is reserved for Chapter 8 in Part 2, but we'll provide enough information in this section to enable you to understand the mechanics of I/O processing that follow.

Interrupt Request Levels

The IRQL has two somewhat distinct meanings, but they converge in certain situations:

- **An IRQL is a priority assigned to an interrupt source from a hardware device** This number is set by the HAL (in conjunction with the interrupt controller to which devices that require interrupt servicing are connected).
- **Each CPU has its own IRQL value** It should be considered a register of the CPU (even though current CPUs do not implement it as such).

The fundamental rule of IRQLs is that lower IRQL code cannot interfere with higher IRQL code and vice versa—code with a higher IRQL can preempt code running at a lower IRQL. You’ll see examples of how this works in practice in a moment. A list of IRQLs for the Windows-supported architectures is shown in [Figure 6-4](#). Note that IRQLs are not the same as thread priorities. In fact, thread priorities have meaning only when the IRQL is less than 2.

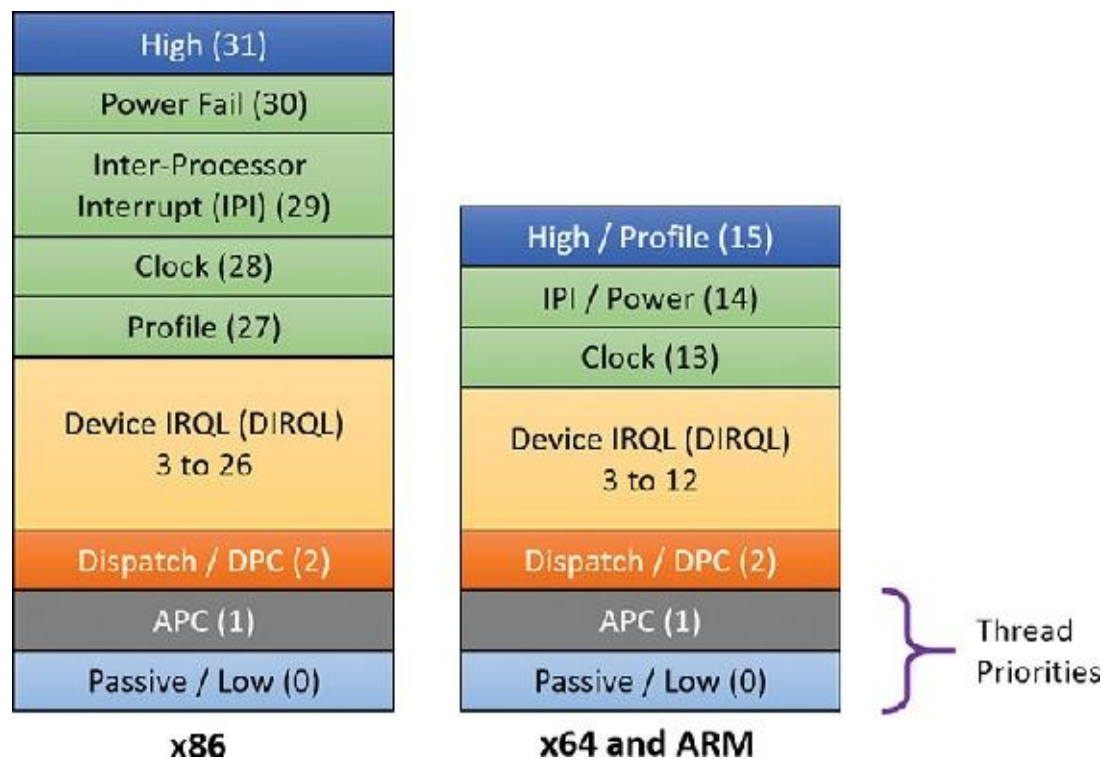


FIGURE 6-4 IRQLs.

Note

IRQL is not the same as IRQ (interrupt request). IRQs are hardware lines connecting devices to an interrupt controller. See Chapter 8 in Part 2 for more on interrupts, IRQs, and IRQLs.

Normally, the IRQL of a processor is 0. This means “nothing special” is happening in that regard, and that the kernel’s scheduler that schedules threads based on priorities and so on works as described in [Chapter 4, “Threads.”](#) In user mode, the IRQL can only be 0. There is no way to raise IRQL from user mode. (That’s why user-mode documentation never mentions the IRQL concept at all; there would be no point.)

Kernel-mode code can raise and lower the current CPU IRQL with the `KeRaiseIrql` and `KeLowerIrql` functions. However, most of the time-specific functions are called with the IRQL raised

to some expected level, as you'll see shortly when we discuss a typical I/O processing by a driver.

The most important IRQLs for this I/O-related discussions are the following:

- **Passive(0)** This is defined by the `PASSIVE_LEVEL` macro in the WDK header `wdm.h`. It is the normal IRQL where the kernel scheduler is working normally, as described at length in [Chapter 4](#).
- **Dispatch/DPC (2) (`DISPATCH_LEVEL`)** This is the IRQL the kernel's scheduler works at. This means if a thread raises the current IRQL to 2 (or higher), the thread has essentially an infinite quantum and will not be preempted by another thread. Effectively, the scheduler cannot wake up on the current CPU until the IRQL drops below 2. This implies a few things:
 - With the IRQL at level 2 or above, any waiting on kernel dispatcher objects (such as mutexes, semaphores, and events) would crash the system. This is because waiting implies that the thread might enter a wait state and another should be scheduled on the same CPU. However, because the scheduler is not around at this level, this cannot happen; instead, the system will bug-check (the only exception is if the wait timeout is zero, meaning no waiting is requested, just getting back the signaled state of the object).
 - No page faults can be handled. This is because a page fault would require a context switch to one of the modified page writers. However, context switches are not allowed, so the system would crash. This means code running at IRQL 2 or above can access only non-paged memory—typically memory allocated from non-paged pool, which by definition is always resident in physical memory.
- **Device IRQL (3–26 on x86; 3–12 on x64 and ARM) (`DIRQL`)** These are the levels assigned to hardware interrupts. When an interrupt arrives, the kernel's trap dispatcher calls the appropriate interrupt service routine (ISR) and raises its IRQL to that of the associated interrupt. Because this value is always higher than `DISPATCH_LEVEL` (2), all rules associated with IRQL 2 apply for `DIRQL` as well.

Running at a particular IRQL masks interrupts with that and lower IRQLs. For example, an ISR running with IRQL of 8 would not let any code interfere (on that CPU) with IRQL of 7 or lower. Specifically, no user mode code is able to run because it always runs at IRQL 0. This implies that running in high IRQL is not desirable in the general case; there are a few specific scenarios (which we'll look at in this chapter) where this makes sense and is in fact required for normal system operation.

Deferred Procedure Calls

A Deferred Procedure Call (DPC) is an object that encapsulates calling a function at IRQL `DPC_LEVEL` (2). DPCs exist primarily for post-interrupt processing because running at `DIRQL` masks (and thus delays) other interrupts waiting to be serviced. A typical ISR would do the minimum work possible, mostly reading the state of the device and telling it to stop its interrupt signal and then deferring further processing to a lower IRQL (2) by requesting a DPC. The term *Deferred* means the DPC will not execute immediately—it can't because the current IRQL is higher than 2. However, when the ISR returns, if there are no pending interrupts waiting to be serviced, the CPU IRQL will drop to 2 and it will execute the DPCs that have accumulated (maybe just one). [Figure 6-5](#) shows a simplified example of the sequence of events that may occur when interrupts from hardware devices (which are asynchronous in nature, meaning they can arrive at any time) occur while code executes normally at IRQL 0 on some CPU.

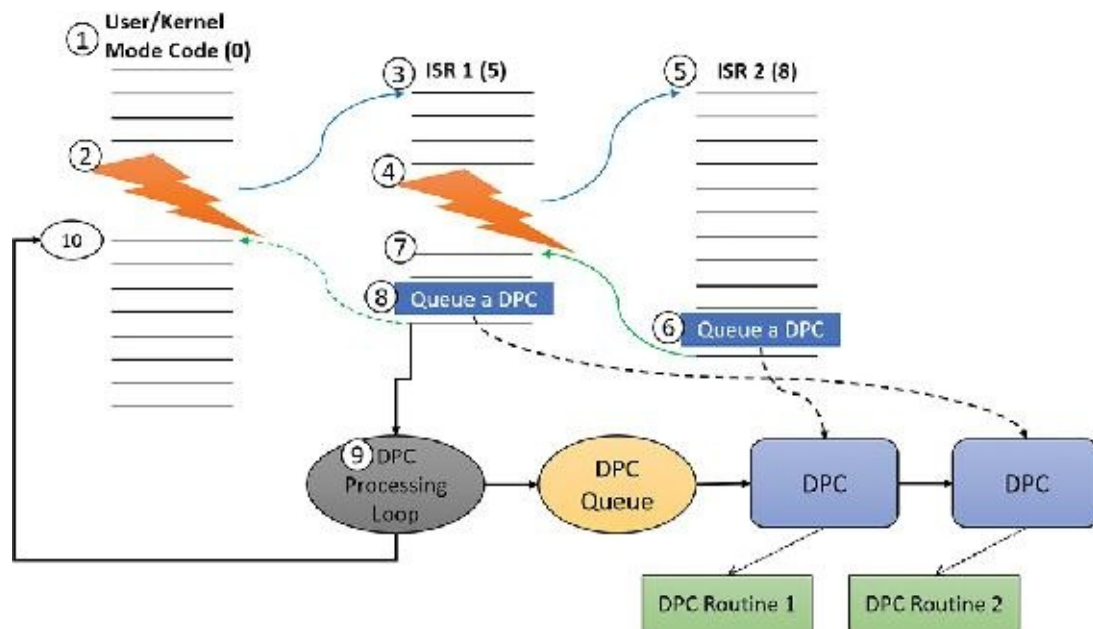


FIGURE 6-5 Example of interrupt and DPC processing.

Here is a rundown of the sequence of events shown in [Figure 6-5](#):

1. Some user-mode or kernel-mode code is executing while the CPU is at IRQL 0, which is the case most of the time.
2. A hardware interrupt arrives with an IRQL of 5 (remember that Device IRQLs have a minimum value of 3). Because 5 is greater than zero (the current IRQL), the CPU state is saved, the IRQL is raised to 5, and the ISR associated with that interrupt is called. Note that there is no context switch; it's the same thread that now happens to execute the ISR code. (If the thread was in user mode, it switches to kernel mode whenever an interrupt arrives.)
3. ISR 1 starts executing while the CPU IRQL is 5. At this point, any interrupt with IRQL 5 or lower cannot interrupt.
4. Suppose another interrupt arrives with an IRQL of 8. Assume the system decides that the same CPU should handle it. Because 8 is greater than 5, the code is interrupted again, the CPU state is saved, the IRQL is raised to 8, and the CPU jumps to ISR 2. Note again that it's the same thread. No context switch can happen because the thread scheduler cannot wake up if the IRQL is 2 or higher.
5. ISR 2 is executing. Before it's done, ISR 2 would like to do some more processing at a lower IRQL so that interrupts with IRQLs less than 8 could be services as well.
6. As its final act, ISR 2 inserts a DPC initialized properly to point to a driver routine to do any post processing after the interrupt is dismissed by calling the `KeInsertQueueDpc` function. (We'll discuss what this post-processing typically includes in the next section.) Then the ISR returns, restoring the CPU state saved before entering ISR 2.
7. At this point, the IRQL drops to its previous level (5) and the CPU continues execution of ISR 1 that was interrupted before.
8. Just before ISR 1 finishes, it queues a DPC of its own to do its required post-processing. These DPCs are collected in a DPC queue that has not been examined yet. Then ISR 1 returns, restoring the CPU state saved before ISR 1 started execution.
9. At this point, the IRQL would want to drop to the old value of zero before all the interrupt handling began. However, the kernel notices that there are DPCs pending and so drops the IRQL to level 2 (`DPC_LEVEL`) and enters a DPC processing loop that iterates over the accumulated DPCs and

calls each DPC routine in sequence. When the DPC queue is empty, DPC processing ends.

10. Finally, the IRQL can drop back to zero, restore the state of the CPU again, and resume execution of the original user or kernel code that got interrupted in the first place. Again, notice that all the processing described was done by the same thread (whichever one that may be). This fact implies that ISRs and DPC routines should not rely on any particular thread (and hence part of a particular process) to execute their code. It could be any thread, the significance of which will be discussed in the next section.

The preceding description is a bit simplified. It doesn't mention DPC importance, other CPUs that may handle DPCs for quicker DPC processing, and more. These details are not important for the discussion in this chapter. However, they are described fully in Chapter 8 in Part 2.

Device drivers

To integrate with the I/O manager and other I/O system components, a device driver must conform to implementation guidelines specific to the type of device it manages and the role it plays in managing the device. This section discusses the types of device drivers Windows supports as well as the internal structure of a device driver.



Note

Most kernel-mode device drivers are written in C. Starting with the Windows Driver Kit 8.0, drivers can also be safely written in C++ due to specific support for kernel-mode C++ in the new compilers. Use of assembly language is highly discouraged because of the complexity it introduces and its effect of making a driver difficult to port between the hardware architectures supported by Windows (x86, x64, and ARM).

Types of device drivers

Windows supports a wide range of device-driver types and programming environments. Even within a particular type of device driver, programming environments can differ depending on the specific type of device for which a driver is intended.

The broadest classification of a driver is whether it is a user-mode or kernel-mode driver. Windows supports a couple of types of user-mode drivers:

- **Windows subsystem printer drivers** These translate device-independent graphics requests to printer-specific commands. These commands are then typically forwarded to a kernel-mode port driver such as the universal serial bus (USB) printer port driver (Usbprint.sys).
- **User-Mode Driver Framework (UMDF) drivers** These are hardware device drivers that run in user mode. They communicate to the kernel-mode UMDF support library through advanced local procedure calls (ALPC). See the “[User-Mode Driver Framework](#)” section later in this chapter for more information.

In this chapter, the focus is on kernel-mode device drivers. There are many types of kernel-mode drivers, which can be divided into the following basic categories:

- **File-system drivers** These accept I/O requests to files and satisfy the requests by issuing their own more explicit requests to mass storage or network device drivers.
- **Plug and Play drivers** These work with hardware and integrate with the Windows power manager

and PnP manager. They include drivers for mass storage devices, video adapters, input devices, and network adapters.

- **Non-Plug and Play drivers** These include kernel extensions, which are drivers or modules that extend the functionality of the system. They do not typically integrate with the PnP manager or power manager because they usually do not manage an actual piece of hardware. Examples include network API and protocol drivers. The Sysinternals tool Process Monitor has a driver, and is an example of a non-PnP driver.

Within the category of kernel-mode drivers are further classifications based on the driver model to which the driver adheres and its role in servicing device requests.

WDM drivers

WDM drivers are device drivers that adhere to the Windows Driver Model (WDM). WDM includes support for Windows power management, Plug and Play, and WMI, and most Plug and Play drivers adhere to WDM. There are three types of WDM drivers:

- **Bus drivers** These manage a logical or physical bus. Examples of buses include PCMCIA, PCI, USB, and IEEE 1394. A bus driver is responsible for detecting and informing the PnP manager of devices attached to the bus it controls and for managing the power setting of the bus. These are typically provided by Microsoft out of the box.
- **Function drivers** These manage a particular type of device. Bus drivers present devices to function drivers via the PnP manager. The function driver is the driver that exports the operational interface of the device to the operating system. In general, it's the driver with the most knowledge about the operation of the device.
- **Filter drivers** These logically layer either above function drivers (these are called *upper filters* or *function filters*) or above the bus driver (these are called *lower filters* or *bus filters*), augmenting or changing the behavior of a device or another driver. For example, a keyboard-capture utility could be implemented with a keyboard filter driver that layers above the keyboard function driver.

[Figure 6-6](#) shows a device node (also called a *devnode*) with a bus driver that creates a physical device object (PDO), lower filters, a function driver that creates a functional device object (FDO), and upper filters. The only required layers are the PDO and FDO. The various filters may or may not exist.

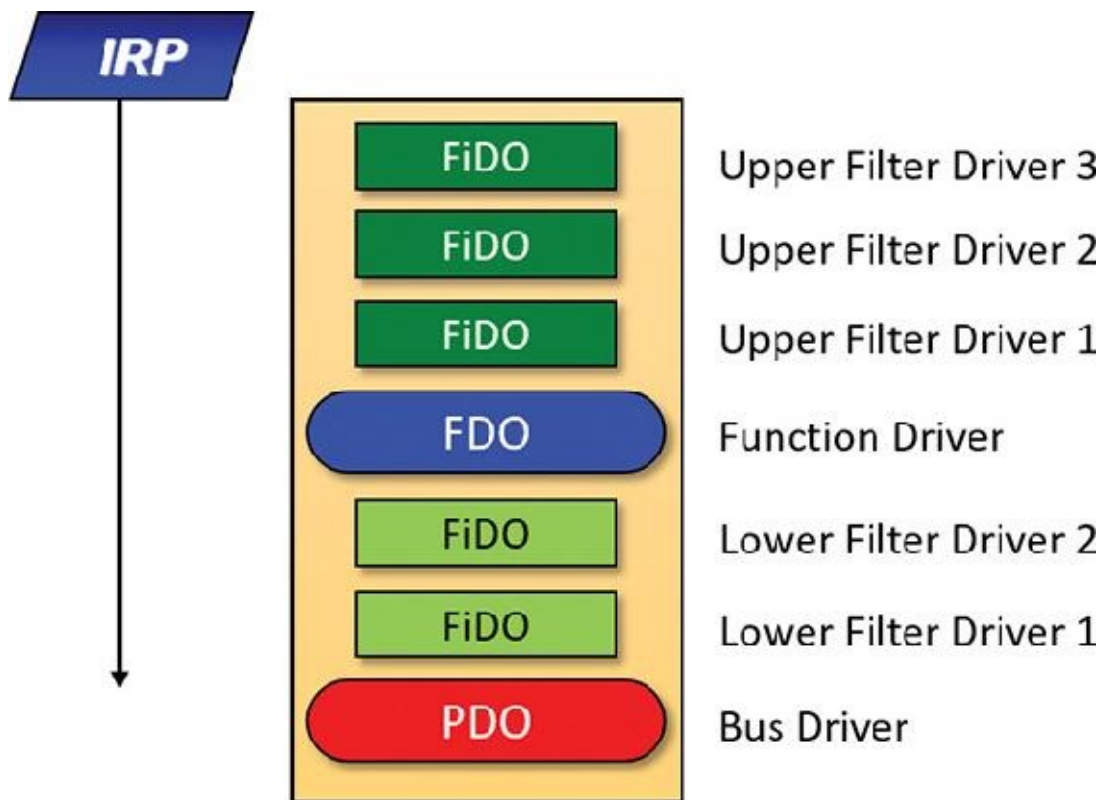


FIGURE 6-6 WDM device node (devnode).

In WDM, no one driver is responsible for controlling all aspects of a particular device. The bus driver is responsible for detecting bus membership changes (device addition or removal), assisting the PnP manager in enumerating the devices on the bus, accessing bus-specific configuration registers, and, in some cases, controlling power to devices on the bus. The function driver is generally the only driver that accesses the device's hardware. The exact manner in which these devices came to be is described in [“The Plug and Play manager”](#) section later in this chapter.

Layered drivers

Support for an individual piece of hardware is often divided among several drivers, each providing a part of the functionality required to make the device work properly. In addition to WDM bus drivers, function drivers, and filter drivers, hardware support might be split between the following components:

- **Class drivers** These implement the I/O processing for a particular class of devices, such as disk, keyboard, or CD-ROM, where the hardware interfaces have been standardized so one driver can serve devices from a wide variety of manufacturers.
- **Miniclass drivers** These implement I/O processing that is vendor-defined for a particular class of devices. For example, although Microsoft has written a standardized battery class driver, both uninterruptible power supplies (UPS) and laptop batteries have highly specific interfaces that differ wildly between manufacturers, such that a miniclass is required from the vendor. *Miniclass drivers* are essentially kernel-mode DLLs and do not perform IRP processing directly. Instead, the class driver calls into them and they import functions from the class driver.
- **Port drivers** These implement the processing of an I/O request specific to a type of I/O port, such as SATA, and are implemented as kernel-mode libraries of functions rather than actual device drivers. Port drivers are almost always written by Microsoft because the interfaces are typically standardized in such a way that different vendors can still share the same port driver. However, in certain cases, third parties may need to write their own for specialized hardware. In some cases, the concept of I/O port extends to cover logical ports as well. For example, Network Driver Interface

Specification (NDIS) is the network “port” driver.

- **Miniport drivers** These map a generic I/O request to a type of port into an adapter type, such as a specific network adapter. Miniport drivers are actual device drivers that import the functions supplied by a port driver. Miniport drivers are written by third parties, and they provide the interface for the port driver. Like miniclass drivers, they are kernel-mode DLLs and do not perform IRP processing directly.

[Figure 6-7](#) shows a simplified example for illustrative purposes that will help demonstrate how device drivers and layering work at a high level. As you can see, a file-system driver accepts a request to write data to a certain location within a particular file. It translates the request into a request to write a certain number of bytes to the disk at a particular (that is, the logical) location. It then passes this request (via the I/O manager) to a simple disk driver. The disk driver, in turn, translates the request into a physical location on the disk and communicates with the disk to write the data.

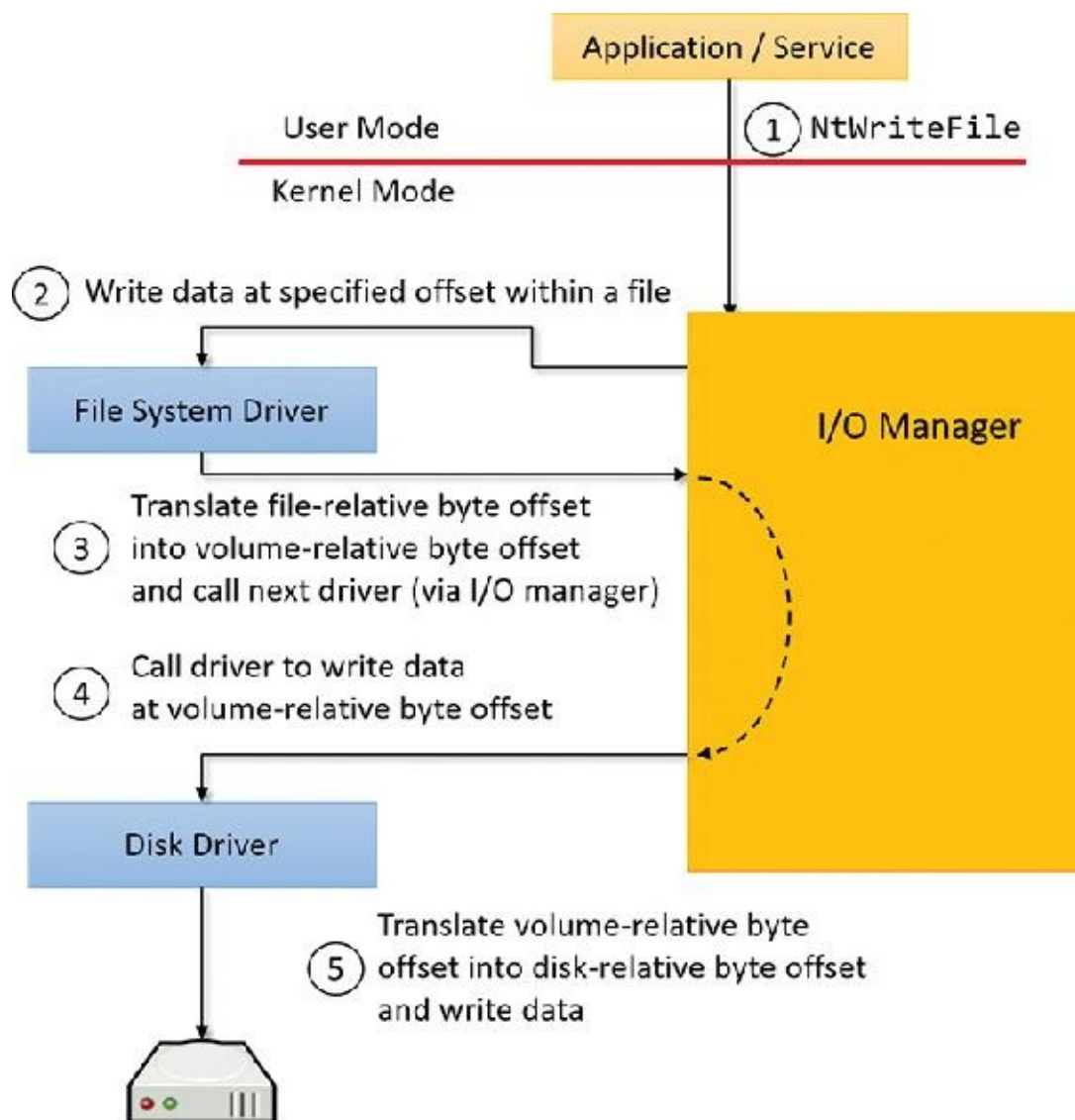


FIGURE 6-7 Layering of a file-system driver and a disk driver.

This figure illustrates the division of labor between two layered drivers. The I/O manager receives a write request that is relative to the beginning of a particular file. The I/O manager passes the request to the file-system driver, which translates the write operation from a file-relative operation to a starting location (a sector boundary on the disk) and a number of bytes to write. The file-system driver calls the I/O manager to pass the request to the disk driver, which translates the request to a physical disk location and transfers the data.

Because all drivers—both device drivers and file-system drivers—present the same framework to the operating system, another driver can easily be inserted into the hierarchy without altering the existing drivers or the I/O system. For example, several disks can be made to seem like a very large single disk by adding a driver. This logical volume manager driver is located between the file system and the disk drivers, as shown in the conceptual simplified architectural diagram presented in [Figure 6-8](#). (For the actual storage driver stack diagram as well as volume manager drivers, see Chapter 12, “Storage management” in Part 2.)

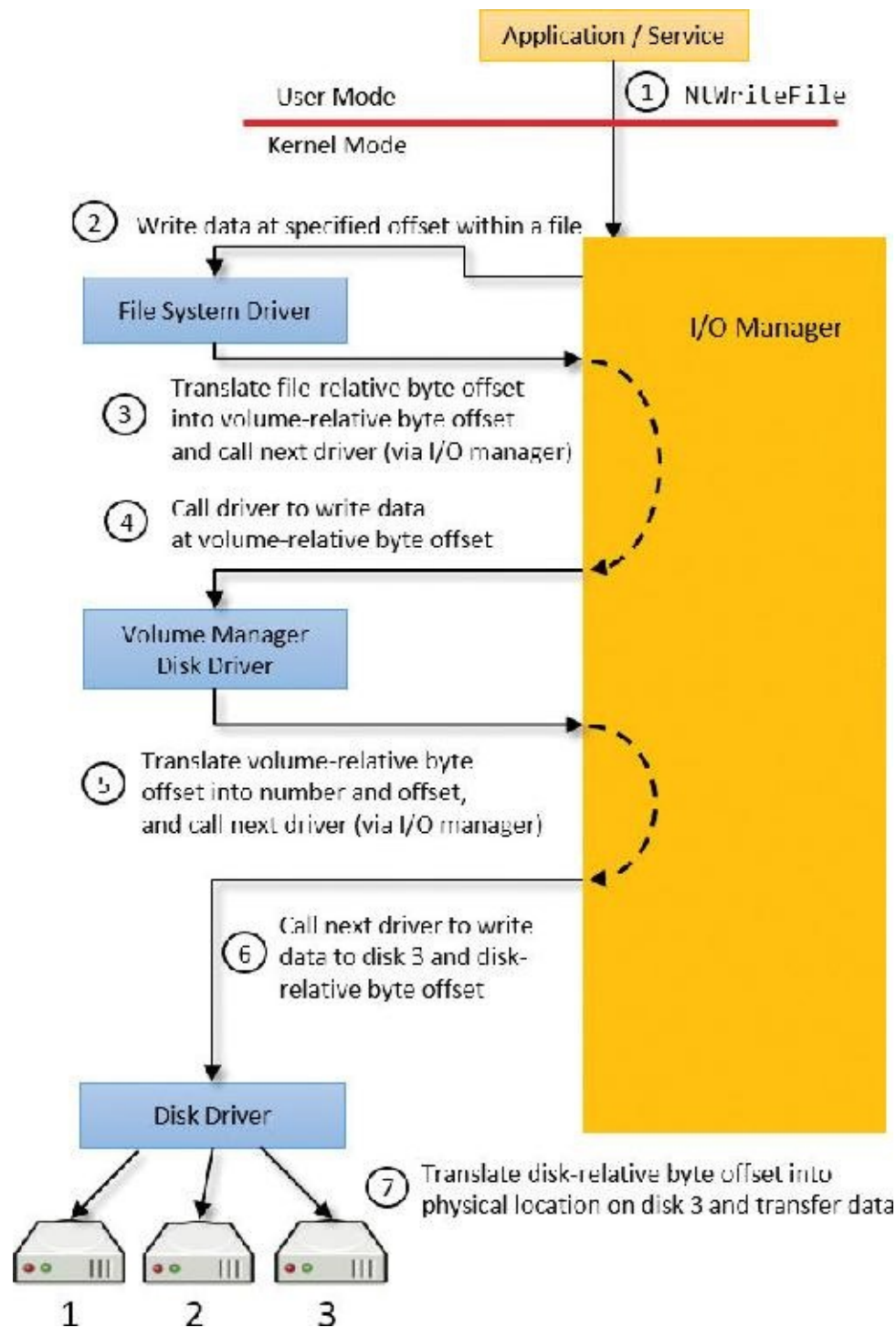
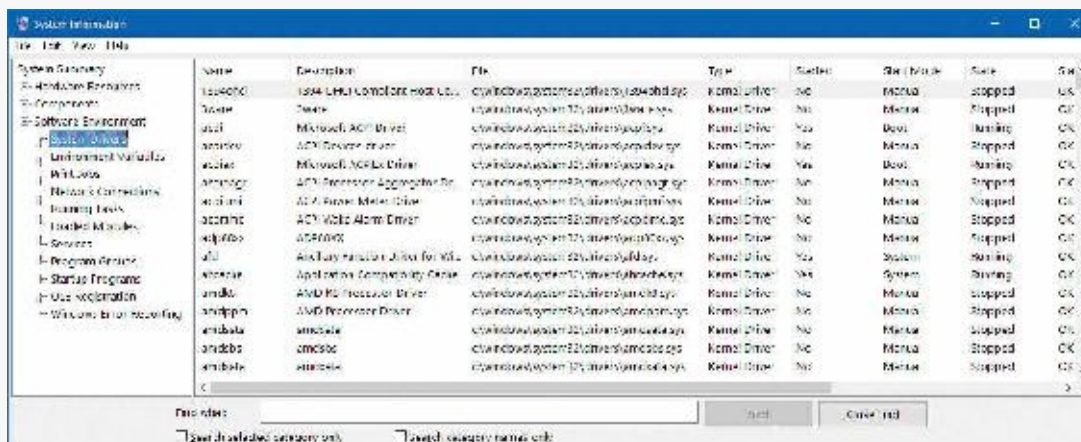


FIGURE 6-8 Adding a layered driver.

EXPERIMENT: Viewing the loaded driver list

You can see a list of registered drivers by executing the `Msiinfo32.exe` utility from the Run dialog box, accessible from the Start menu. Select the **System Drivers** entry under **Software Environment** to see the list of drivers configured on the system. Those that are loaded contain the text *Yes* in the

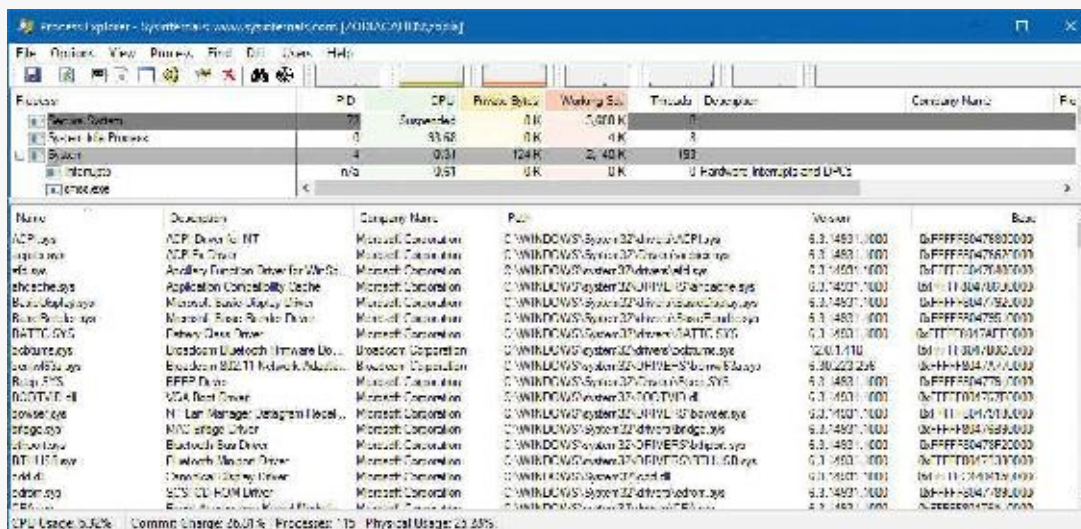
Started column, as shown here:



Name	Description	File	Type	Started	Start Mode	Status
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped
lsasvc.dll	Local Security Authority	C:\Windows\system32\lsasvc.dll	Kernel Driver	Yes	Manual	Stopped

The list of drivers comes from the registry subkeys under HKLM\System\CurrentControlSet\Services. This key is shared between drivers and services. Both can be started by the Service Control Manager (SCM). The way to distinguish between a driver and a service for each subkey is by looking at the Type value. A small value (1, 2, 4, 8) indicates a driver, while 16 (0x10) and 32 (0x20) indicate a Windows service. For more information on the Services subkey, consult Chapter 9 in Part 2.

You can also view the list of loaded kernel-mode drivers with Process Explorer. Run Process Explorer, select the **System** process, and select **DLLs** from the **Lower Pane View** menu entry in the **View** menu:



Name	Description	Company Name	Path	Version	Base
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332
ACPI.sys	ACPI Driver for NT	Microsoft Corporation	C:\Windows\System32\drivers\ACPI.sys	6.0.6002.18000	0x77F14332

Process Explorer lists the loaded drivers, their names, version information (including company and description), and load address (assuming you have configured Process Explorer to display the corresponding columns).

Finally, if you're looking at a crash dump (or live system) with the kernel debugger, you can get a similar display with the kernel debugger `!m kv` command:

[Click here to view code image](#)

```
kd> !m kv
start      end      module name
80626000 80631000  kdcom    (deferred)
Image path: kdcom.dll
Image name: kdcom.dll
Browse all global symbols functions data
Timestamp: Sat Jul 16 04:27:27 2016 (57898D7F)
Checksum:  0000821A
```

```

    ImageSize:          0000B000
    Translations:       0000.04b0 0000.04e4 0409.04b0 0409.04e4
81009000 81632000      nt          (pdb
symbols)              e:\symbols\ntkrpamp.
pdb\A54DF85668E54895982F873F58C984591\ntkrpamp.pdb
    Loaded symbol image file: ntkrpamp.exe
    Image path: ntkrpamp.exe
    Image name: ntkrpamp.exe
    Browse all global symbols functions data
    Timestamp:         Wed Sep 07 07:35:39 2016 (57CF991B)
    CheckSum:          005C6B08
    ImageSize:         00629000
    Translations:       0000.04b0 0000.04e4 0409.04b0 0409.04e4
81632000 81693000      hal          (deferred)
    Image path: halmacpi.dll
    Image name: halmacpi.dll
    Browse all global symbols functions data
    Timestamp:         Sat Jul 16 04:27:33 2016 (57898D85)
    CheckSum:          00061469
    ImageSize:         00061000
    Translations:       0000.04b0 0000.04e4 0409.04b0 0409.04e4
8a800000 8a84b000      FLTMGR       (deferred)
    Image path: \SystemRoot\System32\drivers\FLTMGR.SYS
    Image name: FLTMGR.SYS
    Browse all global symbols functions data
    Timestamp:         Sat Jul 16 04:27:37 2016 (57898D89)
    CheckSum:          00053B90
    ImageSize:         0004B000
    Translations:       0000.04b0 0000.04e4 0409.04b0 0409.04e4
...

```

Structure of a driver

The I/O system drives the execution of device drivers. Device drivers consist of a set of routines that are called to process the various stages of an I/O request. [Figure 6-9](#) illustrates the key driver-function routines, which are described next.

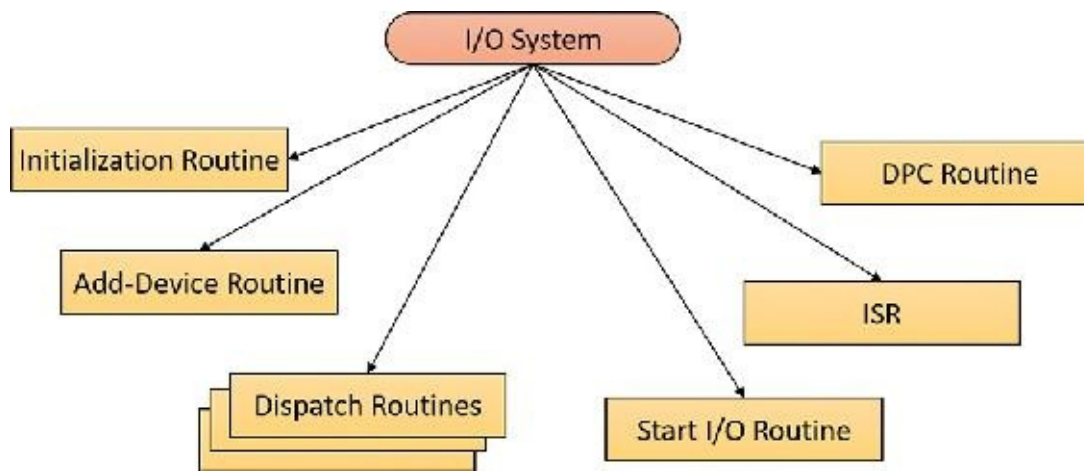


FIGURE 6-9 Primary device driver routines.

- **An initialization routine** The I/O manager executes a driver's initialization routine, which is set by the WDK to `GSDriverEntry` when it loads the driver into the operating system. `GSDriverEntry` initializes the compiler's protection against stack-overflow errors (called a *cookie*) and then calls `DriverEntry`, which is what the driver writer must implement. The routine fills in system data structures to register the rest of the driver's routines with the I/O manager and performs any necessary global driver initialization.
- **An add-device routine** A driver that supports Plug and Play implements an add-device routine. The PnP manager sends a notification to the driver via this routine whenever a device for which the driver is responsible is detected. In this routine, a driver typically creates a device object (described later in this chapter) to represent the device.
- **A set of dispatch routines** Dispatch routines are the main entry points that a device driver provides. Some examples are open, close, read, write, and Plug and Play. When called on to perform an I/O operation, the I/O manager generates an IRP and calls a driver through one of the driver's dispatch routines.
- **A start I/O routine** A driver can use a start I/O routine to initiate a data transfer to or from a device. This routine is defined only in drivers that rely on the I/O manager to queue their incoming I/O requests. The I/O manager serializes IRPs for a driver by ensuring that the driver processes only one IRP at a time. Drivers can process multiple IRPs concurrently, but serialization is usually required for most devices because they cannot concurrently handle multiple I/O requests.
- **An interrupt service routine (ISR)** When a device interrupts, the kernel's interrupt dispatcher transfers control to this routine. In the Windows I/O model, ISRs run at device interrupt request level (DIRQL), so they perform as little work as possible to avoid blocking lower IRQL interrupts (as discussed in the previous section). An ISR usually queues a DPC, which runs at a lower IRQL (DPC/dispatch level) to execute the remainder of interrupt processing. Only drivers for interrupt-driven devices have ISRs; a file-system driver, for example, doesn't have one.
- **An interrupt-servicing DPC routine** A DPC routine performs most of the work involved in handling a device interrupt after the ISR executes. The DPC routine executes at IRQL 2, which is a "compromise" between the high DIRQL and the low passive level (0). A typical DPC routine initiates I/O completion and starts the next queued I/O operation on a device.

Although the following routines aren't shown in [Figure 6-9](#), they're found in many types of device drivers:

- **One or more I/O completion routines** A layered driver might have I/O completion routines that

notify it when a lower-level driver finishes processing an IRP. For example, the I/O manager calls a file-system driver's I/O completion routine after a device driver finishes transferring data to or from a file. The completion routine notifies the file-system driver about the operation's success, failure, or cancellation, and allows the file-system driver to perform cleanup operations.

- **A cancel I/O routine** If an I/O operation can be canceled, a driver can define one or more cancel I/O routines. When the driver receives an IRP for an I/O request that can be canceled, it assigns a cancel routine to the IRP. As the IRP goes through various stages of processing, this routine can change or outright disappear if the current operation is not cancellable. If a thread that issues an I/O request exits before the request is completed or the operation is cancelled (for example, with the `CancelIo` or `CancelIoEx` Windows functions), the I/O manager executes the IRP's cancel routine if one is assigned to it. A cancel routine is responsible for performing whatever steps are necessary to release any resources acquired during the processing that has already taken place for the IRP as well as for completing the IRP with a canceled status.
- **Fast-dispatch routines** Drivers that make use of the cache manager, such as file-system drivers, typically provide these routines to allow the kernel to bypass typical I/O processing when accessing the driver. (See Chapter 14, "Cache manager," in Part 2, for more information on the cache manager.) For example, operations such as reading or writing can be quickly performed by accessing the cached data directly instead of taking the I/O manager's usual path that generates discrete I/O operations. Fast dispatch routines are also used as a mechanism for callbacks from the memory manager and cache manager to file-system drivers. For instance, when creating a section, the memory manager calls back into the file-system driver to acquire the file exclusively.
- **An unload routine** An unload routine releases any system resources a driver is using so that the I/O manager can remove the driver from memory. Any resources acquired in the initialization routine (`DriverEntry`) are usually released in the unload routine. A driver can be loaded and unloaded while the system is running if the driver supports it, but the unload routine will be called only after all file handles to the device are closed.
- **A system shutdown notification routine** This routine allows driver cleanup on system shutdown.
- **Error-logging routines** When unexpected errors occur (for example, when a disk block goes bad), a driver's error-logging routines note the occurrence and notify the I/O manager. The I/O manager then writes this information to an error log file.

Driver objects and device objects

When a thread opens a handle to a file object (described in the "[I/O processing](#)" section later in this chapter), the I/O manager must determine from the file object's name which driver it should call to process the request. Furthermore, the I/O manager must be able to locate this information the next time a thread uses the same file handle. The following system objects fill this need:

- **A driver object** This represents an individual driver in the system (`DRIVER_OBJECT` structure). The I/O manager obtains the address of each of the driver's dispatch routines (entry points) from the driver object.
- **A device object** This represents a physical or logical device on the system and describes its characteristics (`DEVICE_OBJECT` structure), such as the alignment it requires for buffers and the location of its device queue to hold incoming IRPs. It is the target for all I/O operations because this object is what the handle communicates with.

The I/O manager creates a driver object when a driver is loaded into the system. It then calls the

driver's initialization routine (`DriverEntry`), which fills in the object attributes with the driver's entry points.

At any time after loading, a driver creates device objects to represent logical or physical devices—or even a logical interface or endpoint to the driver—by calling `IoCreateDevice` or `IoCreateDevice-Secure`. However, most Plug and Play drivers create devices in their add-device routine when the PnP manager informs them of the presence of a device for them to manage. Non-Plug and Play drivers, on the other hand, usually create device objects when the I/O manager invokes their initialization routine. The I/O manager unloads a driver when the driver's last device object has been deleted and no references to the driver remain.

The relationship between a driver object and its device objects is shown in [Figure 6-10](#).

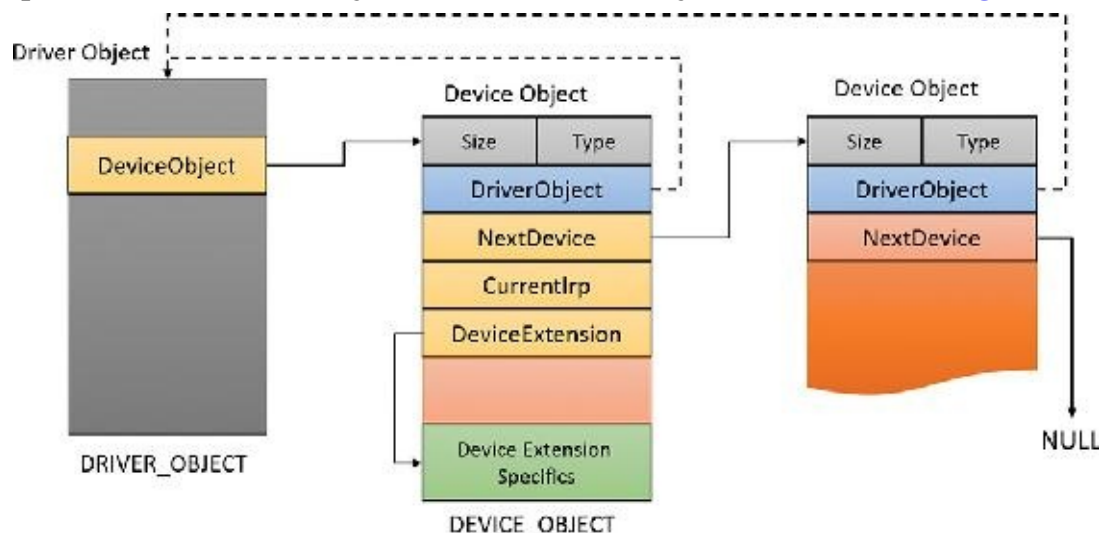


FIGURE 6-10 A driver object and its device objects.

A driver object holds a pointer to its first device object in the `DeviceObject` member. The second device object is pointed to by the `NextDevice` member of `DEVICE_OBJECT` until the last one points to `NULL`. Each device object points back to its driver object with the `DriverObject` member. All the arrows shown in [Figure 6-10](#) are built by the device-creation functions (`IoCreateDevice` or `IoCreateDevice-Secure`). The `DeviceExtension` pointer shown is a way a driver can allocate an extra piece of memory that is attached to each device object it manages.



Note

It's important to distinguish driver objects from device objects. A driver object represents the behavior of a driver, while individual device objects represent communication endpoints. For example, on a system with four serial ports, there would be one driver object (and one driver binary) but four instances of device objects, each representing a single serial port, that can be opened individually with no effect on the other serial ports. For hardware devices, each device also represents a distinct set of hardware resources, such as I/O ports, memory-mapped I/O, and interrupt line. Windows is device-centric, rather than driver-centric.

When a driver creates a device object, the driver can optionally assign the device a name. A name places the device object in the object manager namespace. A driver can either explicitly define a name or let the I/O manager auto-generate one. By convention, device objects are placed in the `\Device` directory in the namespace, which is inaccessible by applications using the Windows API.



Note

Some drivers place device objects in directories other than `\Device`. For example, the IDE driver creates the device objects that represent IDE ports and channels in the `\Device\Ide` directory. See Chapter 12 in Part 2 for a description of storage architecture, including the way storage drivers use device objects.

If a driver needs to make it possible for applications to open the device object, it must create a symbolic link in the `\GLOBAL??` directory to the device object's name in the `\Device` directory. (The `IoCreateSymbolicLink` function accomplishes this.) Non-Plug and Play and file-system drivers typically create a symbolic link with a well-known name (for example, `\Device\HarddiskVolume2`). Because well-known names don't work well in an environment in which hardware appears and disappears dynamically, PnP drivers expose one or more interfaces by calling the `IoRegisterDeviceInterface` function, specifying a globally unique identifier (GUID) that represents the type of functionality exposed. GUIDs are 128-bit values that can be generated by using tools such as `uuidgen` and `guidgen`, which are included with the WDK and the Windows SDK. Given the range of values that 128 bits represents (and the formula used to generate them), it's statistically almost certain that each GUID generated will be forever and globally unique.

`IoRegisterDeviceInterface` generates the symbolic link associated with a device instance. However, a driver must call `IoSetDeviceInterfaceState` to enable the interface to the device before the I/O manager actually creates the link. Drivers usually do this when the PnP manager starts the device by sending the driver a start-device IRP—in this case, `IRP_MJ_PNP` (major function code) with `IRP_MN_START_DEVICE` (minor function code). IRPs are discussed in the “[I/O request packets](#)” section later in this chapter.

An application that wants to open a device object whose interfaces are represented with a GUID can call Plug and Play setup functions in user space, such as `SetupDiEnumDeviceInterfaces`, to enumerate the interfaces present for a particular GUID and to obtain the names of the symbolic links it can use to open the device objects. For each device reported by `SetupDiEnumDeviceInterfaces`, the application executes `SetupDiGetDeviceInterfaceDetail` to obtain additional information about the device, such as its auto-generated name. After obtaining a device's name from `SetupDiGetDeviceInterface- Detail`, the application can execute the Windows function `CreateFile` or `CreateFile2` to open the device and obtain a handle.

EXPERIMENT: Looking at device objects

You can use the WinObj tool from Sysinternals or the `!object` kernel debugger command to view the device names under `\Device` in the object manager namespace. The following screenshot shows an I/O manager–assigned symbolic link that points to a device object in `\Device` with an auto-generated name:

	89c323c8	Device	00000028
	8554fb50	Device	KMDF0
04	958bb040	Device	ProcessManagement
	97ad9fe0	SymbolicLink	MailslotRedirector
	854f0090	Device	00000036
	854c6538	Device	FakeVid5
	8bf14e98	Device	Video1
	8bf2fe20	Device	KeyboardClass1
	89c332a0	Device	00000029
	89c05030	Device	VolMgrControl
	89c3a1a8	Device	VMBus
...			

When you enter the `!object` command and specify an object manager directory object, the kernel debugger dumps the contents of the directory according to the way the object manager organizes it internally. For fast lookups, a directory stores objects in a hash table based on a hash of the object names, so the output shows the objects stored in each bucket of the directory's hash table.

As [Figure 6-10](#) illustrates, a device object points back to its driver object, which is how the I/O manager knows which driver routine to call when it receives an I/O request. It uses the device object to find the driver object representing the driver that services the device. It then indexes into the driver object by using the function code supplied in the original request. Each function code corresponds to a driver entry point (called a *dispatch routine*).

A driver object often has multiple device objects associated with it. When a driver is unloaded from the system, the I/O manager uses the queue of device objects to determine which devices will be affected by the removal of the driver.

EXPERIMENT: Displaying driver and device objects

You can display driver and device objects with the `!drvobj` and `!devobj` kernel debugger commands, respectively. In the following example, the driver object for the keyboard class driver is examined, and one of its device objects viewed:

[Click here to view code image](#)

```
1: kd> !drvobj kbdclass
Driver object (8a557520) is for:
  \Driver\kbdclass
Driver Extension List: (id , addr)

Device Object list:
9f509648 8bf2fe20 8a541030
1: kd> !devobj 9f509648
Device object (9f509648) is for:
  KeyboardClass2 \Driver\kbdclass DriverObject 8a557520
Current Irp 00000000 RefCount 0 Type 0000000b Flags 00002044
Dacl 82090960 DevExt 9f509700 DevObjExt 9f5097f0
ExtensionFlags (0x000000c0)  DOE_SESSION_DEVICE,
DOE_DEFAULT_SD_PRESENT
```

Characteristics (0x00000100) FILE_DEVICE_SECURE_OPEN

AttachedTo (Lower) 9f509848 \Driver\terminpt

Device queue is not busy.

Notice that the !devobj command also shows you the addresses and names of any device objects that the object you're viewing is layered over (the AttachedTo line). It can also show the device objects layered on top of the object specified (the AttachedDevice line), although not in this case.

The !drvobj command can accept an optional argument that indicates more information to show. Here is an example with the most information to show:

[Click here to view code image](#)

```
1: kd> !drvobj kbdclass 7
```

```
Driver object (8a557520) is for:
```

```
\Driver\kbdclass
```

```
Driver Extension List: (id , addr)
```

```
Device Object list:
```

```
9f509648 8bf2fe20 8a541030
```

```
DriverEntry: 8c30a010
```

```
kbdclass!GsDriverEntry
```

```
DriverStartIo: 00000000
```

```
DriverUnload: 00000000
```

```
AddDevice: 8c307250
```

```
kbdclass!KeyboardAddDevice
```

```
Dispatch routines:
```

```
[00]
```

```
IRP_MJ_CREATE
```

```
8c301d80
```

```
kbdclass!KeyboardC
```

```
[01]
```

```
IRP_MJ_CREATE_NAMED_PIPE
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[02]
```

```
IRP_MJ_CLOSE
```

```
8c301c90
```

```
kbdclass!KeyboardC
```

```
[03]
```

```
IRP_MJ_READ
```

```
8c302150
```

```
kbdclass!KeyboardC
```

```
[04]
```

```
IRP_MJ_WRITE
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[05]
```

```
IRP_MJ_QUERY_INFORMATION
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[06]
```

```
IRP_MJ_SET_INFORMATION
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[07]
```

```
IRP_MJ_QUERY_EA
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[08]
```

```
IRP_MJ_SET_EA
```

```
81142342
```

```
nt!IopInvalidDevic
```

```
[09]
```

```
IRP_MJ_FLUSH_BUFFERS
```

```
8c303678
```

```
kbdclass!KeyboardC
```

```
[0a]
```

```
IRP_MJ_QUERY_VOLUME_INFORMATION
```

```
81142342
```

```
nt!IopInvalidDevic
```

[0b]	IRP_MJ_SET_VOLUME_INFORMATION	81142342	nt!IopInvalidDeviceRequest
[0c]	IRP_MJ_DIRECTORY_CONTROL	81142342	nt!IopInvalidDeviceRequest
[0d]	IRP_MJ_FILE_SYSTEM_CONTROL	81142342	nt!IopInvalidDeviceRequest
[0e]	IRP_MJ_DEVICE_CONTROL	8c3076d0	kbdclass!KeyboardControl
[0f]	IRP_MJ_INTERNAL_DEVICE_CONTROL	8c307ff0	kbdclass!KeyboardControl
[10]	IRP_MJ_SHUTDOWN	81142342	nt!IopInvalidDeviceRequest
[11]	IRP_MJ_LOCK_CONTROL	81142342	nt!IopInvalidDeviceRequest
[12]	IRP_MJ_CLEANUP	8c302260	kbdclass!KeyboardControl
[13]	IRP_MJ_CREATE_MAILSLLOT	81142342	nt!IopInvalidDeviceRequest
[14]	IRP_MJ_QUERY_SECURITY	81142342	nt!IopInvalidDeviceRequest
[15]	IRP_MJ_SET_SECURITY	81142342	nt!IopInvalidDeviceRequest
[16]	IRP_MJ_POWER	8c301440	kbdclass!KeyboardControl
[17]	IRP_MJ_SYSTEM_CONTROL	8c307f40	kbdclass!KeyboardControl
[18]	IRP_MJ_DEVICE_CHANGE	81142342	nt!IopInvalidDeviceRequest
[19]	IRP_MJ_QUERY_QUOTA	81142342	nt!IopInvalidDeviceRequest
[1a]	IRP_MJ_SET_QUOTA	81142342	nt!IopInvalidDeviceRequest
[1b]	IRP_MJ_PNP	8c301870	kbdclass!KeyboardControl

The dispatch routines array is clearly shown, and will be discussed in the next section. Note that operations that are not supported by the driver point to an I/O manager's routine `IopInvalidDeviceRequest`.

The address to the `!drvobj` command is for a `DRIVER_OBJECT` structure, and the address for the `!devobj` command is for a `DEVICE_OBJECT`. You can view these structures directly using the debugger:

[Click here to view code image](#)

```
1: kd> dt nt!_driver_object 8a557520
```

```

+0x000 Type           : 0n4
+0x002 Size           : 0n168
+0x004 DeviceObject   : 0x9f509648 _DEVICE_OBJECT
+0x008 Flags          : 0x412
+0x00c DriverStart    : 0x8c300000 Void
+0x010 DriverSize     : 0xe000
+0x014 DriverSection  : 0x8a556ba8 Void
+0x018 DriverExtension : 0x8a5575c8 _DRIVER_EXTENSION
+0x01c DriverName     : _UNICODE_STRING "\Driver\kbdclass"
+0x024 HardwareDatabase : 0x815c2c28 _UNICODE_STRING
"\REGISTRY\MACHINE\HARDWARE\
DESCRIPTION\SYSTEM"
+0x028 FastIoDispatch : (null)
+0x02c DriverInit     : 0x8c30a010      long  +fffffffff8c30a010
+0x030 DriverStartIo  : (null)
+0x034 DriverUnload   : (null)
+0x038 MajorFunction  : [28]
0x8c301d80      long  +fffffffff8c301d80
1: kd> dt nt!_device_object 9f509648
+0x000 Type           : 0n3
+0x002 Size           : 0x1a8
+0x004 ReferenceCount : 0n0
+0x008 DriverObject   : 0x8a557520 _DRIVER_OBJECT
+0x00c NextDevice     : 0x8bf2fe20 _DEVICE_OBJECT
+0x010 AttachedDevice : (null)
+0x014 CurrentIrp     : (null)
+0x018 Timer          : (null)
+0x01c Flags          : 0x2044
+0x020 Characteristics : 0x100
+0x024 Vpb            : (null)
+0x028 DeviceExtension : 0x9f509700 Void
+0x02c DeviceType     : 0xb
+0x030 StackSize      : 7 ''
+0x034 Queue          : <unnamed-tag>
+0x05c AlignmentRequirement : 0
+0x060 DeviceQueue    : _KDEVICE_QUEUE
+0x074 Dpc             : _KDPC
+0x094 ActiveThreadCount : 0
+0x098 SecurityDescriptor : 0x82090930 Void
...
```

There are some interesting fields in these structures, which we'll discuss in the next section.

Using objects to record information about drivers means that the I/O manager doesn't need to know details about individual drivers. The I/O manager merely follows a pointer to locate a driver, thereby providing a layer of portability and allowing new drivers to be loaded easily.

Opening devices

A *file object* is a kernel-mode data structure that represents a handle to a device. File objects clearly fit the criteria for objects in Windows: They are system resources that two or more user-mode processes can share; they can have names; they are protected by object-based security; and they support synchronization. Shared resources in the I/O system, like those in other components of the Windows executive, are manipulated as objects. (See Chapter 8 in Part 2 for more on object management.)

File objects provide a memory-based representation of resources that conform to an I/O-centric interface, in which they can be read from or written to. [Table 6-1](#) lists some of the file object’s attributes. For specific field declarations and sizes, see the structure definition for `FILE_OBJECT` in `wdm.h`.

Attribute	Purpose
File name	This identifies the virtual file that the file object refers to, which was passed in to the <code>CreateFile</code> or <code>CreateFile2</code> APIs.
Current byte offset	This identifies the current location in the file (valid only for synchronous I/O).
Share modes	These indicate whether other callers can open the file for read, write, or delete operations while the current caller is using it.
Open mode flags	These indicate whether I/O will be synchronous or asynchronous, cached or non-cached, sequential or random, and so on.
Pointer to device object	This indicates the type of device the file resides on.
Pointer to the volume parameter block (VPB)	This indicates the volume, or partition, that the file resides on (in the case of file system files).
Pointer to section object pointers	This indicates a root structure that describes a mapped/cached file. This structure also contains the shared cache map, which identifies which parts of the file are cached (or rather, mapped) by the cache manager and where they reside in the cache.
Pointer to private cache map	This is used to store per-handle caching information such as the read patterns for this handle or the page priority for the process. See Chapter 5, “Memory management,” for more information on page priority.
List of I/O request packets (IRPs)	If thread-agnostic I/O (described in the section “Thread-agnostic I/O” later in this chapter) is used and the file object is associated with a completion port (described in the section “I/O completion ports”), this is a list of all the I/O operations that are associated with this file object.
I/O completion context	This is context information for the current I/O completion port, if one is active.
File object extension	This stores the I/O priority (explained later in this chapter) for the file and whether share-access checks should be performed on the file object, and contains optional file object extensions that store context-specific information.

TABLE 6-1 File object attributes

To maintain some level of opacity toward driver code that uses the file object, and to enable extending the file object functionality without enlarging the structure, the file object also contains an extension field, which allows for up to six different kinds of additional attributes, described in [Table 6-2](#).

Extension	Purpose
Transaction parameters	This contains the transaction parameter block, which contains information about a transacted file operation. It's returned by <code>IoGetTransactionParameterBlock</code> .
Device object hint	This identifies the device object of the filter driver with which this file should be associated. It's set with <code>IoCreateFileEx</code> or <code>IoCreateFileSpecifyDeviceObjectHint</code> .
I/O status block range	This allows applications to lock a user-mode buffer into kernel-mode memory to optimize asynchronous I/Os. It's set with <code>SetFileIoOverlappedRange</code> .
Generic	This contains filter driver–specific information, as well as extended create parameters (ECPs) that were added by the caller. It's set with <code>IoCreateFileEx</code> .
Scheduled file I/O	This stores a file's bandwidth reservation information, which is used by the storage system to optimize and guarantee throughput for multimedia applications. (See the section "Bandwidth reservation (scheduled file I/O)" later in this chapter.) It's set with <code>SetFileBandwidthReservation</code> .
Symbolic link	This is added to the file object upon creation, when a mount point or directory junction is traversed (or a filter explicitly reparses the path). It stores the caller-supplied path, including information about any intermediate junctions, so that if a relative symbolic link is hit, it can walk back through the junctions. See Chapter 13 in Part 2 for more information on NTFS symbolic links, mount points, and directory junctions.

TABLE 6-2 File object extensions

When a caller opens a file or a simple device, the I/O manager returns a handle to a file object. Before that happens, the driver responsible for the device in question is asked via its Create dispatch routine (`IRP_MJ_CREATE`) whether it's OK to open the device and allow the driver to perform any initialization necessary if the open request is to succeed.



Note

File objects represent open instances of files, not files themselves. Unlike UNIX systems, which use vnodes, Windows does not define the representation of a file; Windows file-system drivers define their own representations.

Similar to executive objects, files are protected by a security descriptor that contains an access control list (ACL). The I/O manager consults the security subsystem to determine whether a file's ACL allows the process to access the file in the way its thread is requesting. If it does, the object manager grants the access and associates the granted access rights with the file handle that it returns. If this thread or another thread in the process needs to perform additional operations not specified in the original request, the thread must open the same file again with a different request (or duplicate the handle with the requested access) to get another handle, which prompts another security check. (See [Chapter 7](#) for more information about object protection.)

EXPERIMENT: Viewing device handles

Any process that has an open handle to a device will have a file object in its handle table corresponding to the open instance. You can view these handles with Process Explorer by selecting a process and checking **Handles** in the **Lower Pane View** submenu of the **View** menu. Sort by the **Type** column and scroll to where you see the handles that represent file objects, which are labeled as *File*.

CurrentByteOffset: 0

Because a file object is a memory-based representation of a shareable resource and not the resource itself, it's different from other executive objects. A file object contains only data that is unique to an object handle, whereas the file itself contains the data or text to be shared. Each time a thread opens a file, a new file object is created with a new set of handle-specific attributes. For example, for files opened synchronously, the current byte offset attribute refers to the location in the file at which the next read or write operation using that handle will occur. Each handle to a file has a private byte offset even though the underlying file is shared. A file object is also unique to a process—except when a process duplicates a file handle to another process (by using the Windows `DuplicateHandle` function) or when a child process inherits a file handle from a parent process. In these situations, the two processes have separate handles that refer to the same file object.

Although a file handle is unique to a process, the underlying physical resource is not. Therefore, as with any shared resource, threads must synchronize their access to shareable resources such as files, file directories, and devices. If a thread is writing to a file, for example, it should specify exclusive write access when opening the file to prevent other threads from writing to the file at the same time. Alternatively, by using the Windows `LockFile` function, the thread could lock a portion of the file while writing to it when exclusive access is required.

When a file is opened, the file name includes the name of the device object on which the file resides. For example, the name `\Device\HarddiskVolume1\Myfile.dat` may refer to the file `Myfile.dat` on the C: volume. The substring `\Device\HarddiskVolume1` is the name of the internal Windows device object representing that volume. When opening `Myfile.dat`, the I/O manager creates a file object and stores a pointer to the `HarddiskVolume1` device object in the file object and then returns a file handle to the caller. Thereafter, when the caller uses the file handle, the I/O manager can find the `HarddiskVolume1` device object directly.

Keep in mind that internal Windows device names can't be used in Windows applications—instead, the device name must appear in a special directory in the object manager's namespace, which is `\GLOBAL??`. This directory contains symbolic links to the real, internal Windows device names. As was described earlier, device drivers are responsible for creating links in this directory so that their devices will be accessible to Windows applications. You can examine or even change these links programmatically with the Windows `QueryDosDevice` and `DefineDosDevice` functions.

I/O processing

Now that we've covered the structure and types of drivers and the data structures that support them, let's look at how I/O requests flow through the system. I/O requests pass through several predictable stages of processing. The stages vary depending on whether the request is destined for a device operated by a single-layered driver or for a device reached through a multilayered driver. Processing varies further depending on whether the caller specified synchronous or asynchronous I/O, so we'll begin our discussion of I/O types with these two and then move on to others.

Types of I/O

Applications have several options for the I/O requests they issue. Furthermore, the I/O manager gives drivers the choice of implementing a shortcut I/O interface that can often mitigate IRP allocation for I/O processing. In this section, we'll explain these options for I/O requests.

Synchronous and asynchronous I/O

Most I/O operations issued by applications are *synchronous* (which is the default). That is, the application thread waits while the device performs the data operation and returns a status code when the I/O is complete. The program can then continue and access the transferred data immediately. When used in their simplest form, the Windows `ReadFile` and `WriteFile` functions are executed synchronously. They complete the I/O operation before returning control to the caller.

Asynchronous I/O allows an application to issue multiple I/O requests and continue executing while the device performs the I/O operation. This type of I/O can improve an application's throughput because it allows the application thread to continue with other work while an I/O operation is in progress. To use asynchronous I/O, you must specify the `FILE_FLAG_OVERLAPPED` flag when you call the Windows `CreateFile` or `CreateFile2` functions. Of course, after issuing an asynchronous I/O operation, the thread must be careful not to access any data from the I/O operation until the device driver has finished the data operation. The thread must synchronize its execution with the completion of the I/O request by monitoring a handle of a synchronization object (whether that's an event object, an I/O completion port, or the file object itself) that will be signaled when the I/O is complete.

Regardless of the type of I/O request, I/O operations issued to a driver on behalf of the application are performed asynchronously. That is, once an I/O request has been initiated, the device driver must return to the I/O system as soon as possible. Whether or not the I/O system returns immediately to the caller depends on whether the handle was opened for synchronous or asynchronous I/O. [Figure 6-3](#) illustrates the flow of control when a read operation is initiated. Notice that if a wait is done, which depends on the overlapped flag in the file object, it is done in kernel mode by the `NtReadFile` function.

You can test the status of a pending asynchronous I/O operation with the Windows `HasOverlapped-IoCompleted` macro or get more details with the `GetOverlappedResult(Ex)` functions. If you're using I/O completion ports (described in the "[I/O completion ports](#)" section later in this chapter), you can use the `GetQueuedCompletionStatus(Ex)` function(s).

Fast I/O

Fast I/O is a special mechanism that allows the I/O system to bypass the generation of an IRP and instead go directly to the driver stack to complete an I/O request. This mechanism is used for optimizing certain I/O paths, which are somewhat slower when using IRPs. (Fast I/O is described in detail in Chapter 13 and Chapter 14 in Part 2.) A driver registers its fast I/O entry points by entering them in a structure pointed to by the `PFAST_IO_DISPATCH` pointer in its driver object.

EXPERIMENT: Looking at a driver's registered fast I/O routines

The `!drvobj` kernel debugger command can list the fast I/O routines that a driver registers in its driver object. Typically, however, only file-system drivers have any use for fast I/O routines—although there are exceptions, such as network protocol drivers and bus filter drivers. The following output shows the fast I/O table for the NTFS file-system driver object:

[Click here to view code image](#)

```

lkd> !drvobj \filesystem\ntfs 2
Driver object (ffffc404b2fbf810) is for:
  \FileSystem\NTFS
DriverEntry:      fffffff80e5663a030      NTFS!GsDrj
DriverStartIo:    000000000
DriverUnload:     000000000
AddDevice:        000000000

```

Dispatch routines:

...

Fast I/O routines:

FastIoCheckIfPossible	ffffff80e565d6750	
NTFS!NtfsFastIoCheckIfPossible		
FastIoRead	ffffff80e56526430	NTFS!Ntfs
FastIoWrite	ffffff80e56523310	NTFS!NtfsC
FastIoQueryBasicInfo	ffffff80e56523140	
NTFS!NtfsFastQueryBasicInfo		
FastIoQueryStandardInfo	ffffff80e56534d20	NTFS!NtfsF
FastIoLock	ffffff80e5651e610	NTFS!NtfsF
FastIoUnlockSingle	ffffff80e5651e3c0	NTFS!NtfsF
FastIoUnlockAll	ffffff80e565d59e0	NTFS!NtfsF
FastIoUnlockAllByKey	ffffff80e565d5c50	
NTFS!NtfsFastUnlockAllByKey		
ReleaseFileForNtCreateSection	ffffff80e5644fd90	NTFS!NtfsF
Section		
FastIoQueryNetworkOpenInfo	ffffff80e56537750	NTFS!NtfsF
OpenInfo		
AcquireForModWrite	ffffff80e5643e0c0	
NTFS!NtfsAcquireFileForModWrite		
MdlRead	ffffff80e5651e950	NTFS!NtfsM
MdlReadComplete	ffffff802dc6cd844	
nt!FsRtlMdlReadCompleteDev		
PrepareMdlWrite	ffffff80e56541a10	NTFS!NtfsF
MdlWriteComplete	ffffff802dcb76e48	
nt!FsRtlMdlWriteCompleteDev		
FastIoQueryOpen	ffffff80e5653a520	
NTFS!NtfsNetworkOpenCreate		
ReleaseForModWrite	ffffff80e5643e2c0	
NTFS!NtfsReleaseFileForModWrite		
AcquireForCcFlush	ffffff80e5644ca60	
NTFS!NtfsAcquireFileForCcFlush		
ReleaseForCcFlush	ffffff80e56450cf0	
NTFS!NtfsReleaseFileForCcFlush		

The output shows that NTFS has registered its `NtfsCopyReadA` routine as the fast I/O table's `FastIoRead` entry. As the name of this fast I/O entry implies, the I/O manager calls this function when issuing a read I/O request if the file is cached. If the call doesn't succeed, the standard IRP path is

selected.

Mapped-file I/O and file caching

Mapped-file I/O is an important feature of the I/O system—one that the I/O system and the memory manager produce jointly. (See [Chapter 5](#) for details on how mapped files are implemented.) *Mapped-file I/O* refers to the ability to view a file residing on disk as part of a process's virtual memory. A program can access the file as a large array without buffering data or performing disk I/O. The program accesses memory, and the memory manager uses its paging mechanism to load the correct page from the disk file. If the application writes to its virtual address space, the memory manager writes the changes back to the file as part of normal paging.

Mapped-file I/O is available in user mode through the Windows `CreateFileMapping`, `MapViewOfFile`, and related functions. Within the operating system, mapped-file I/O is used for important operations such as file caching and image activation (loading and running executable programs). The other major consumer of mapped-file I/O is the cache manager. File systems use the cache manager to map file data in virtual memory to provide better response time for I/O-bound programs. As the caller uses the file, the memory manager brings accessed pages into memory. Whereas most caching systems allocate a fixed number of bytes for caching files in memory, the Windows cache grows or shrinks depending on how much memory is available. This size variability is possible because the cache manager relies on the memory manager to automatically expand (or shrink) the size of the cache using the normal working set mechanisms explained in [Chapter 5](#)—in this case applied to the system working set. By taking advantage of the memory manager's paging system, the cache manager avoids duplicating the work that the memory manager already performs. (The workings of the cache manager are explained in detail in Chapter 14 in Part 2.)

Scatter/gather I/O

Windows supports a special kind of high-performance I/O called *scatter/gather*, available via the Windows `ReadFileScatter` and `WriteFileGather` functions. These functions allow an application to issue a single read or write from more than one buffer in virtual memory to a contiguous area of a file on disk instead of issuing a separate I/O request for each buffer. To use scatter/gather I/O, the file must be opened for non-cached I/O, the user buffers being used must be page-aligned, and the I/Os must be asynchronous (overlapped). Furthermore, if the I/O is directed at a mass storage device, the I/O must be aligned on a device sector boundary and have a length that is a multiple of the sector size.

I/O request packets

An *I/O request packet (IRP)* is where the I/O system stores information it needs to process an I/O request. When a thread calls an I/O API, the I/O manager constructs an IRP to represent the operation as it progresses through the I/O system. If possible, the I/O manager allocates IRPs from one of three per-processor IRP non-paged look-aside lists:

- **The small-IRP look-aside list** This stores IRPs with one stack location. (IRP stack locations are described shortly.)
- **The medium-IRP look-aside list** This contains IRPs with four stack locations (which can also be used for IRPs that require only two or three stack locations).
- **The large-IRP look-aside list** This contains IRPs with more than four stack locations. By default, the system stores IRPs with 14 stack locations on the large-IRP look-aside list, but once per minute, the system adjusts the number of stack locations allocated and can increase it up to a maximum of

20, based on how many stack locations have been recently required.

These lists are also backed by global look-aside lists as well, allowing efficient cross-CPU IRP flow. If an IRP requires more stack locations than are contained in the IRPs on the large-IRP look-aside list, the I/O manager allocates IRPs from non-paged pool. The I/O manager allocates IRPs with the `IoAllocate-Irp` function, which is also available for device-driver developers, because in some cases a driver may want to initiate an I/O request directly by creating and initializing its own IRPs. After allocating and initializing an IRP, the I/O manager stores a pointer to the caller's file object in the IRP.



Note

If defined, the `DWORD` registry value `LargeIrpStackLocations` in the `HKLM\System\CurrentControlSet\Session Manager\I/O System` key specifies how many stack locations are contained in IRPs stored on the large-IRP look-aside list. Similarly, the `MediumIrpStackLocations` value in the same key can be used to change the size of IRP stack locations on the medium-IRP look-aside list.

[Figure 6-11](#) shows some of the important members of the IRP structure. It is always accompanied by one or more `IO_STACK_LOCATION` objects (described in the next section).

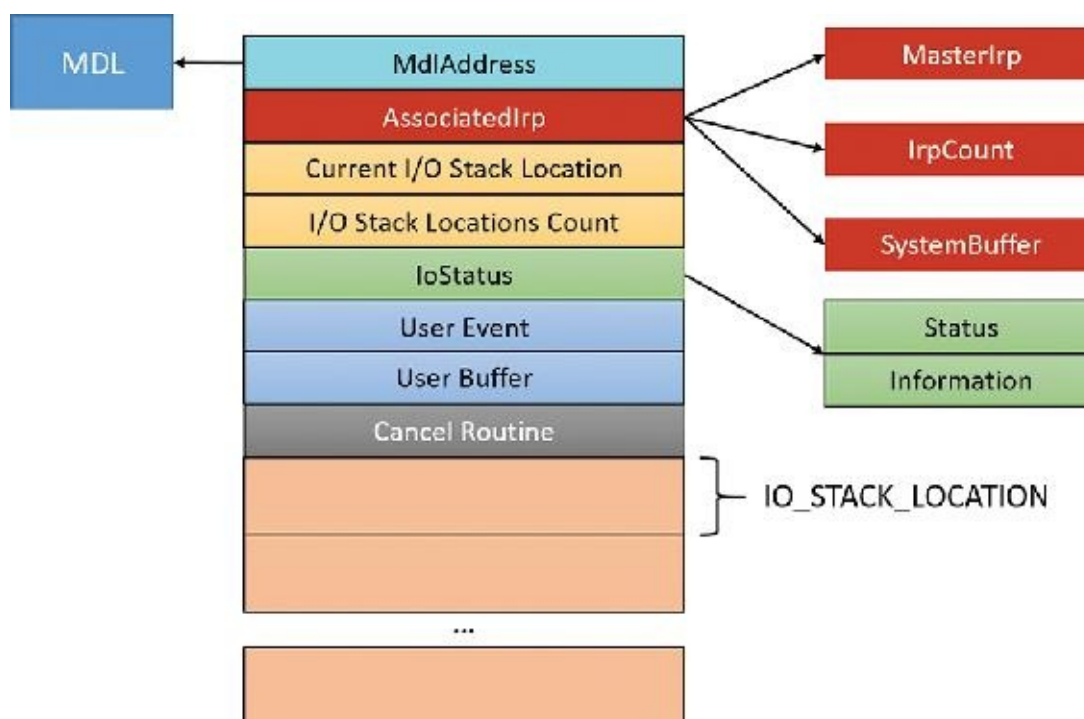


FIGURE 6-11 Important members of the IRP structure.

Here is a quick rundown of the members:

- **IoStatus** This is the status of the IRP, consisting of two members; `Status`, which is the actual code itself and `Information`, a polymorphic value that has meaning in some cases. For example, for a read or write operation, this value (set by the driver) indicates the number of bytes read or written. This same value is the one reported as an output value from the functions `ReadFile` and `WriteFile`.
- **MdlAddress** This is an optional pointer to a memory descriptor list (MDL). An MDL is a structure that represents information for a buffer in physical memory. We'll discuss its main usage in device

drivers in the next section. If an MDL was not requested, the value is NULL.

- **I/O stack locations count and current stack location** These store the total number of trailing I/O stack location objects and point to the current one that this driver layer should look at, respectively. The next section discusses I/O stack locations in detail.
- **User buffer** This is the pointer to the buffer provided by the client that initiated the I/O operation. For example, it is the buffer provided to the `ReadFile` or `WriteFile` functions.
- **User event** This is the kernel event object that was used with an overlapped (asynchronous) I/O operation (if any). An event is one way to be notified when the I/O operation completes.
- **Cancel routine** This is the function to be called by the I/O manager in case the IRP is cancelled.
- **AssociatedIrp** This is a union of one of three fields. The `SystemBuffer` member is used in case the I/O manager used the buffered I/O technique for passing the user's buffer to the driver. The next section discusses buffered I/O, as well as other options for passing user mode buffers to drivers. The `MasterIrp` member provides a way to create a "master IRP" that splits its work into sub-IRPs, where the master is considered complete only when all its sub-IRPs have completed.

I/O stack locations

An IRP is always followed by one or more I/O stack locations. The number of stack locations is equal to the number of layered devices in the device node the IRP is destined for. The I/O operation information is split between the IRP body (the main structure) and the current I/O stack location, where *current* means the one set up for the particular layer of devices. [Figure 6-12](#) shows the important fields of an I/O stack location. When an IRP is created, the number of requested I/O stack locations is passed to `IoAllocateIrp`. The I/O manager then initializes the IRP body and the first I/O stack location only, destined for the top-most device in the device node. Each layer in the device node is responsible for initializing the next I/O stack location if it decides to pass the IRP down to the next device.

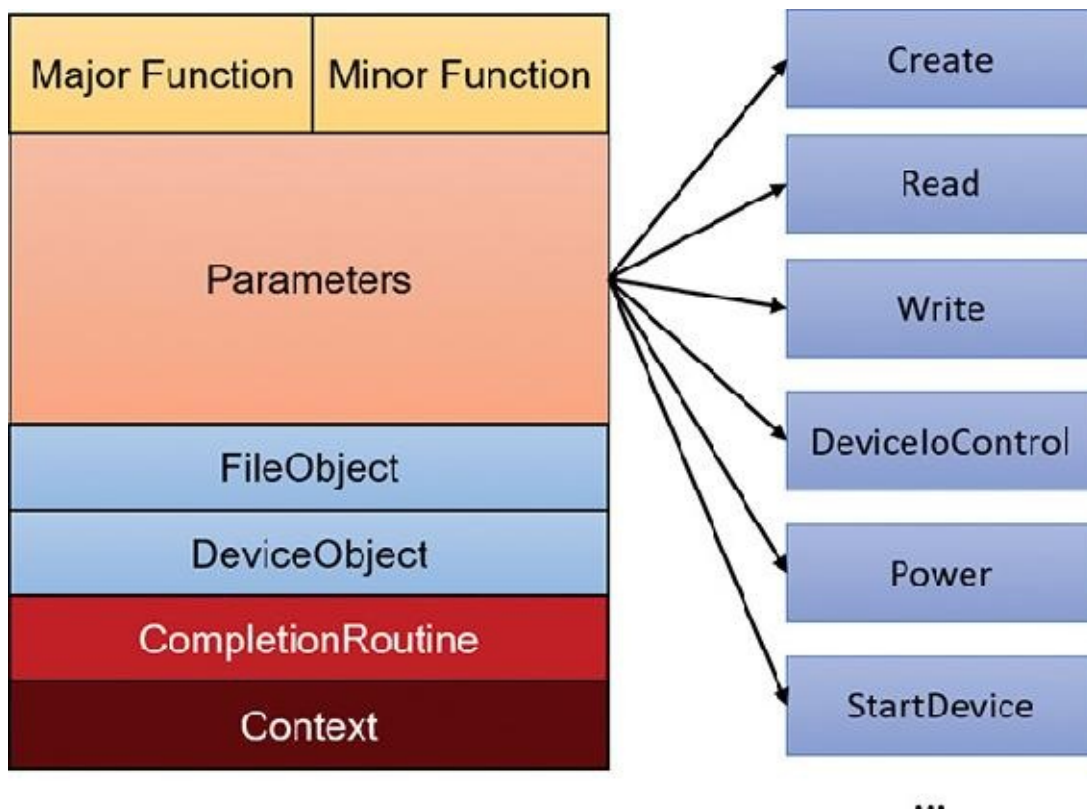


FIGURE 6-12 Important members of the `IO_STACK_LOCATION` structure.

Here is a rundown of the members shown in [Figure 6-12](#):

- **Major function** This is the primary code that indicates the type of request (read, write, create, Plug and Play, and so on), also known as dispatch routine code. It's one of 28 constants (0 to 27) starting with `IRP_MJ_` in `wdm.h`. This index is used by the I/O manager into the `MajorFunction` array of function pointers in the driver object to jump to the appropriate routine within a driver. Most drivers specify dispatch routines to handle only a subset of possible major function codes, including create (open), read, write, device I/O control, power, Plug and Play, system control (for WMI commands), cleanup, and close. File-system drivers are an example of a driver type that often fills in most or all of its dispatch entry points with functions. In contrast, a driver for a simple USB device would probably fill in only the routines needed for open, close, read, write, and sending I/O control codes. The I/O manager sets any dispatch entry points that a driver doesn't fill to point to its own `IoInvalidDeviceRequest`, which completes the IRP with an error status indicating that the major function specified in the IRP is invalid for that device.
- **Minor function** This is used to augment the major function code for some functions. For example, `IRP_MJ_READ` (read) and `IRP_MJ_WRITE` (write) have no minor functions. But Plug and Play and Power IRPs always have a minor IRP code that specializes the general major code. For example, the Plug and Play `IRP_MJ_PNP` major code is too generic; the exact instruction is given by the minor IRP, such as `IRP_MN_START_DEVICE`, `IRP_MN_REMOVE_DEVICE`, and so on.
- **Parameters** This is a monstrous union of structures, each of which valid for a particular major function code or a combination of major/minor codes. For example, for a read operation (`IRP_MJ_READ`), the `Parameters.Read` structure holds information on the read request, such as the buffer size.
- **File object and Device object** These point to the associated `FILE_OBJECT` and `DEVICE_OBJECT` for this I/O request.
- **Completion routine** This is an optional function that a driver can register with the `IoSetCompletionRoutine(Ex)` DDI, to be called when the IRP is completed by a lower layer driver. At that point, the driver can look at the completion status of the IRP and do any needed post-processing. It can even undo the completion (by returning the special value `STATUS_MORE_PROCESSING_REQUIRED` from the function) and resend the IRP (perhaps with modified parameters) to the device node—or even a different device node—again.
- **Context** This is an arbitrary value set with the `IoSetCompletionRoutine(Ex)` call that is passed, as is, to the completion routine.

The split of information between the IRP body and its I/O stack location allows for the changing of I/O stack location parameters for the next device in the device stack, while keeping the original request parameters. For example, a read IRP targeted at a USB device is often changed by the function driver to a device I/O control IRP where the input buffer argument of the device control points to a USB request packet (URB) that is understood by the lower-layer USB bus driver. Also, note that completion routines can be registered by any layer (except the bottom-most one), each having its own place in an I/O stack location (the completion routine is stored in the *next* lower I/O stack location).

EXPERIMENT: Looking at driver dispatch routines

You can obtain a list of the functions a driver has defined for its dispatch routines by using bit 1 (value of 2) with the `!drvobj` kernel debugger command. The following output shows the major function codes supported by the NTFS driver. (This is the same experiment as with fast I/O.)

[Click here to view code image](#)

```
lkd> !drvobj \filesystem\ntfs 2
Driver object (ffffc404b2fbf810) is for:
  \FileSystem\NTFS
DriverEntry:      fffffff80e5663a030          NTFS!GsDriv
DriverStartIo:    000000000
DriverUnload:     000000000
AddDevice:        000000000
```

Dispatch routines:

[00]		
IRP_MJ_CREATE	ffffff80e565278e0	NTFS!NtfsFsdCrea
[01]		
IRP_MJ_CREATE_NAMED_PIPE	ffffff802dc762c80	nt!IopInvalidDev
[02]		
IRP_MJ_CLOSE	ffffff80e565258c0	NTFS!NtfsFsdClos
[03]		
IRP_MJ_READ	ffffff80e56436060	NTFS!NtfsFsdReac
[04]		
IRP_MJ_WRITE	ffffff80e564461d0	NTFS!NtfsFsdWrit
[05]		
IRP_MJ_QUERY_INFORMATION	ffffff80e565275f0	NTFS!NtfsFsdDisp
[06]		
IRP_MJ_SET_INFORMATION	ffffff80e564edb80	NTFS!NtfsFsdSetI
[07]		
IRP_MJ_QUERY_EA	ffffff80e565275f0	NTFS!NtfsFsdDisp
[08]		
IRP_MJ_SET_EA	ffffff80e565275f0	NTFS!NtfsFsdDisp
[09]		
IRP_MJ_FLUSH_BUFFERS	ffffff80e5653c9a0	NTFS!NtfsFsdFlus
[0a]		
IRP_MJ_QUERY_VOLUME_INFORMATION	ffffff80e56538d10	NTFS!NtfsFsdDisp
[0b]		
IRP_MJ_SET_VOLUME_INFORMATION	ffffff80e56538d10	NTFS!NtfsFsdDisp
[0c] IRP_MJ_DIRECTORY_CONTROL	ffffff80e564d7080	
NTFS!NtfsFsdDirectoryControl		
[0d] IRP_MJ_FILE_SYSTEM_CONTROL	ffffff80e56524b20	
NTFS!NtfsFsdFileSystemControl		
[0e]		
IRP_MJ_DEVICE_CONTROL	ffffff80e564f9de0	NTFS!NtfsFsdDevi
[0f]		
IRP_MJ_INTERNAL_DEVICE_CONTROL	ffffff802dc762c80	nt!IopInvalidDev
[10]		
IRP_MJ_SHUTDOWN	ffffff80e565efb50	NTFS!NtfsFsdShut
[11]		
IRP_MJ_LOCK_CONTROL	ffffff80e5646c870	NTFS!NtfsFsdLock
[12]		
IRP_MJ_CLEANUP	ffffff80e56525580	NTFS!NtfsFsdClea

[13]	IRP_MJ_CREATE_MAILSLLOT	ffffff802dc762c80	nt!IopInvalidDev
[14]	IRP_MJ_QUERY_SECURITY	ffffff80e56538d10	NTFS!NtfsFsdDisp
[15]	IRP_MJ_SET_SECURITY	ffffff80e56538d10	NTFS!NtfsFsdDisp
[16]	IRP_MJ_POWER	ffffff802dc762c80	nt!IopInvalidDev
[17]	IRP_MJ_SYSTEM_CONTROL	ffffff802dc762c80	nt!IopInvalidDev
[18]	IRP_MJ_DEVICE_CHANGE	ffffff802dc762c80	nt!IopInvalidDev
[19]	IRP_MJ_QUERY_QUOTA	ffffff80e565275f0	NTFS!NtfsFsdDisp
[1a]	IRP_MJ_SET_QUOTA	ffffff80e565275f0	NTFS!NtfsFsdDisp
[1b]	IRP_MJ_PNP	ffffff80e56566230	NTFS!NtfsFsdPnp

Fast I/O routines:
...

While active, each IRP is usually queued in an IRP list associated with the thread that requested the I/O. (Otherwise, it is stored in the file object when performing thread-agnostic I/O, which is described in the “Thread agnostic I/O” section, later in this chapter.) This allows the I/O system to find and cancel any outstanding IRPs if a thread terminates with I/O requests that have not been completed. Additionally, paging I/O IRPs are also associated with the faulting thread (although they are not cancellable). This allows Windows to use the thread-agnostic I/O optimization—when an asynchronous procedure call (APC) is not used to complete I/O if the current thread is the initiating thread. This means page faults occur inline instead of requiring APC delivery.

EXPERIMENT: Looking at a thread’s outstanding IRPs

The `!thread` command prints any IRPs associated with the thread. The `!process` command does this as well, if requested. Run the kernel debugger with local or live debugging and list the threads of an explorer process:

[Click here to view code image](#)

```
lkd> !process 0 7 explorer.exe
PROCESS fffffc404b673c780
    SessionId: 1  Cid: 10b0    Peb: 00cbb000  ParentCid: 1038
    DirBase: 8895f000  ObjectTable: fffffe689011b71c0  HandleCount:
<Data Not
Accessible>
    Image: explorer.exe
    VadRoot fffffc404b672b980 Vads 569 Clone 0 Private 7260. Modified
366527. Locked 784.
```

```

DeviceMap fffffe688fd7a5d30
Token fffffe68900024920
ElapsedTime 18:48:28.375
UserTime 00:00:17.500
KernelTime 00:00:13.484
...
MemoryPriority BACKGROUND
BasePriority 8
CommitCharge 10789
Job fffffc404b6075060

```

```

        THREAD fffffc404b673a080  Cid 10b0.10b4  Teb:
000000000000c000 Win32Thread:
fffffc404b66e7090 WAIT: (WrUserRequest) UserMode Non-Alertable
        fffffc404b6760740  SynchronizationEvent
        Not impersonating
...

```

```

        THREAD fffffc404b613c7c0  Cid 153c.15a8  Teb:
0000000000006a3000 Win32Thread:
fffffc404b6a83910 WAIT: (UserRequest) UserMode Non-Alertable
        fffffc404b58d0d60  SynchronizationEvent
        fffffc404b566f310  SynchronizationEvent
        IRP List:
        fffffc404b69ad920: (0006,02c8) Flags: 00060800  Mdl:
000000000
...

```

You should see many threads, with most of them having IRPs reported in the `IRP List` section of the thread information (note that the debugger will show only the first 17 IRPs for a thread that has more than 17 outstanding I/O requests). Choose an IRP and examine it with the `!irp` command:

[Click here to view code image](#)

```

lkd> !irp fffffc404b69ad920
Irp is active with 2 stacks 1 is current (= 0xfffffc404b69ad9f0)
  No Mdl: No System Buffer: Thread fffffc404b613c7c0:  Irp stack
trace.
      cmd  flg  cl Device      File      Completion-Context
>[IRP_MJ_FILE_SYSTEM_CONTROL(d), N/A(0)]
      5 e1 fffffc404b253cc90 fffffc404b5685620 ffffff80e55752ed0-
fffffc404b63c0e00
Success Error Cancel pending
      \FileSystem\Npfs      FLTMGR!FltpPassThroughCompleti
      Args: 00000000 00000000 00110008
00000000
[IRP_MJ_FILE_SYSTEM_CONTROL(d), N/A(0)]
      5  0 fffffc404b3cdca00 fffffc404b5685620 00000000-00000000

```

00000000

The IRP has two stack locations and is targeted at a device owned by the Named Pipe File System (NPFS) driver. (NPFS is described in Chapter 10, “Networking,” in Part 2.)

IRP flow

IRPs are typically created by the I/O manager, and then sent to the first device on the target device node. [Figure 6-13](#) shows a typical IRP flow for hardware-based device drivers.

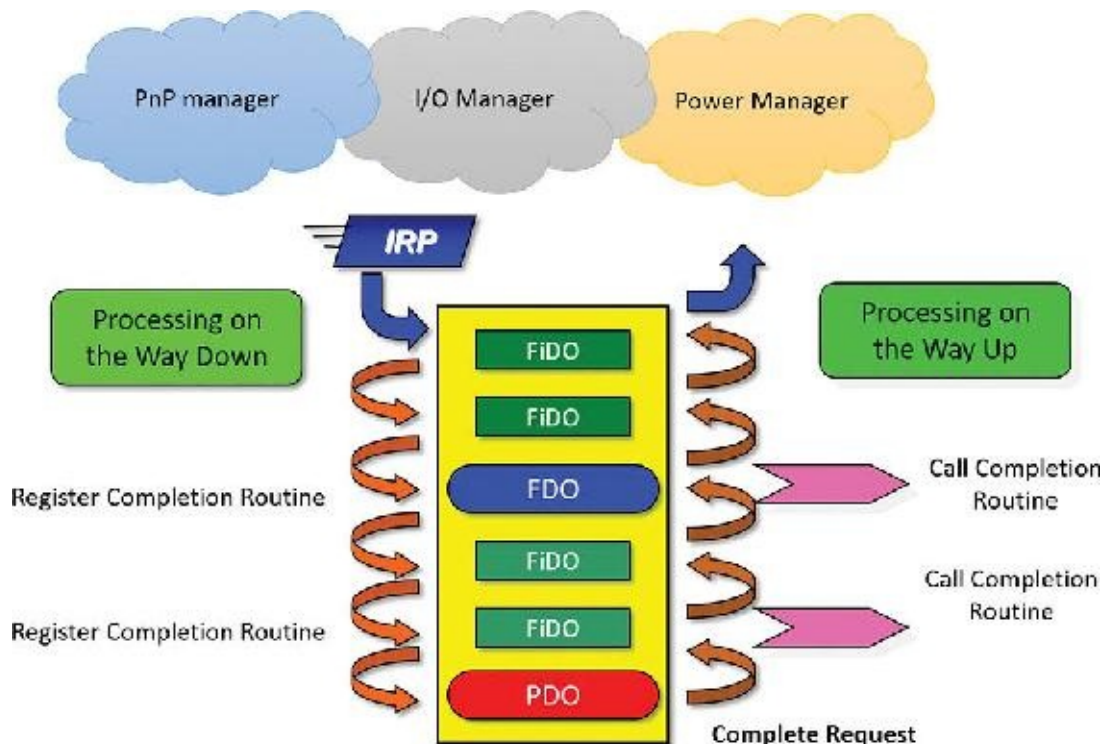


FIGURE 6-13 IRP flow.

The I/O manager is not the only entity that creates IRPs. The Plug and Play manager and the Power manager are also responsible for creating IRPs with major function code `IRP_MJ_PNP` and `IRP_MJ_POWER`, respectively.

[Figure 6-13](#) shows an example device node with six layered device objects: two upper filters, the FDO, two lower filters, and the PDO. This means an IRP targeted at this devnode is created with six I/O stack locations—one for each layer. An IRP is always delivered to the highest layered device, even if a handle was opened to a named device that is lower in the device stack.

A driver that receives an IRP can do one of the following:

- It can complete the IRP then and there by calling `IoCompleteRequest`. This could be because the IRP has some invalid parameters (for example, insufficient buffer size or bad I/O control code), or because the operation requested is quick and can be accomplished immediately, such as getting some status from the device or reading a value from the registry. The driver calls `IoGetCurrentIrpStackLocation` to get a pointer to the stack location that it should refer to.
- The driver can forward the IRP to the next layer after optionally doing some processing. For example, an upper filter can do some logging of the operation and send the IRP down to be executed

normally. Before sending the request down, the driver must prepare the next I/O stack location that would be looked at by the next driver in line. It can use the `IoSkipCurrentIrpStackLocation` macro if it does not wish to make changes, or it can make a copy with `IoCopyIrpStackLocationToNext` and make changes to the copied stack location by getting a pointer with `IoGetNextIrpStackLocation` and making appropriate changes. Once the next I/O stack location is prepared, the driver calls `IoCallDriver` to do the actual IRP forwarding.

- As an extension of the previous point, the driver can also register for a completion routine by calling `IoSetCompletionRoutine(Ex)` before passing down the IRP. Any layer except the bottom-most one can register a completion routine (there is no point in registering for the bottom-most layer since that driver must complete the IRP, so no callback is needed). After `IoCompleteRequest` is called by a lower-layer driver, the IRP travels up (refer to [Figure 6-13](#)), calling any completion routines on the way up in reverse order of registration. In fact, the IRP originator (I/O manager, PnP manager, or power manager) use this mechanism to do any post-IRP processing and finally free the IRP.



Note

Because the number of devices on a given stack is known in advance, the I/O manager allocates one stack location per device driver on the stack. However, there are situations in which an IRP might be directed into a new driver stack. This can happen in scenarios involving the filter manager, which allows one filter to redirect an IRP to another filter (for example, going from a local file system to a network file system). The I/O manager exposes an API, `IoAdjustStackSizeForRedirection`, that enables this functionality by adding the required stack locations because of devices present on the redirected stack.

EXPERIMENT: Viewing a device stack

The `!devstack` kernel debugger command shows you the device stack of layered device objects associated with a specified device object. This example shows the device stack associated with a device object, `\device\keyboardclass0`, which is owned by the keyboard class driver:

[Click here to view code image](#)

```
lkd> !devstack keyboardclass0
!DevObj          !DrvObj          !DevExt          ObjectName
>
ffff9c80c0424440  \Driver\kbdclass  fffff9c80c0424590 KeyboardClass0
ffff9c80c04247c0  \Driver\kbdhid    fffff9c80c0424910
ffff9c80c0414060  \Driver\mshidkmdf fffff9c80c04141b0 0000003f
!DevNode fffff9c80c0414d30 :
DeviceInst is "HID\MSHW0029&Col01\5&1599b1c7&0&0000"
ServiceName is "kbdhid"
```

The output highlights the entry associated with `KeyboardClass0` with the `>` character in the first column. The entries above that line are drivers layered above the keyboard class driver, and those below are layered beneath it.

EXPERIMENT: Examining IRPs

In this experiment, you'll find an uncompleted IRP on the system, and will determine the IRP type, the device at which it's directed, the driver that manages the device, the thread that issued the IRP, and what process the thread belongs to. This experiment is best performed on a 32-bit system with non-local kernel debugging. It will work with local kernel debugging as well, but IRPs may complete during the period between when commands are issued, so some instability of data should be expected.

At any point in time, there are at least a few uncompleted IRPs on a system. This occurs because there are many devices to which applications can issue IRPs that a driver will complete only when a particular event occurs, such as data becoming available. One example is a blocking read from a network endpoint. You can see the outstanding IRPs on a system with the `!irpfind` kernel debugger command (this may take some time; you can stop after some IRPs appear):

[Click here to view code image](#)

```
kd> !irpfind
Scanning large pool allocation table for tag 0x3f707249 (Irp?)
(a5000000 : a5200000)

  Irp      [ Thread ] irpStack: (Mj,Mn)   DevObj   [Driver]           MDL
Process
9515ad68 [aa0c04c0] irpStack: ( e, 5)  8bcb2ca0 [ \Driver\AFD]
0xaa1a3540
8bd5c548 [91deeb80] irpStack: ( e,20)  8bcb2ca0 [ \Driver\AFD]
0x91da5c40

Searching nonpaged pool (80000000 : ffc00000) for tag 0x3f707249
(Irp?)

86264a20 [86262040] irpStack: ( e, 0)  8a7b4ef0 [ \Driver\vm bus]
86278720 [91d96b80] irpStack: ( e,20)  8bcb2ca0 [ \Driver\AFD]
0x86270040
86279e48 [91d96b80] irpStack: ( e,20)  8bcb2ca0 [ \Driver\AFD]
0x86270040
862a1868 [862978c0] irpStack: ( d, 0)  8bca4030 [ \FileSystem\Npfs]
862a24c0 [86297040] irpStack: ( d, 0)  8bca4030 [ \FileSystem\Npfs]
862c3218 [9c25f740] irpStack: ( c, 2)  8b127018 [ \FileSystem\NTFS]
862c4988 [a14bf800] irpStack: ( e, 5)  8bcb2ca0 [ \Driver\AFD]
0xaa1a3540
862c57d8 [a8ef84c0] irpStack: ( d, 0)  8b127018 [ \FileSystem\NTFS]
0xa8e6f040
862c91c0 [99ac9040] irpStack: ( 3, 0)  8a7ace48 [ \Driver\vm bus]
0x9517ac40
862d2d98 [9fd456c0] irpStack: ( e, 5)  8bcb2ca0 [ \Driver\AFD]
0x9fc11780
862d6528 [9aded800] irpStack: ( c, 2)  8b127018 [ \FileSystem\NTFS]
862e3230 [00000000] Irp is complete (CurrentLocation 2 > StackCount
```

```
1)
862ec248 [862e2040] irpStack: ( d, 0) 8bca4030 [ \FileSystem\Npfs]
862f7d70 [91dd0800] irpStack: ( d, 0) 8bca4030 [ \FileSystem\Npfs]
863011f8 [00000000] Irp is complete (CurrentLocation 2 > StackCount
1)
86327008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86328008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86328960 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86329008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
863296d8 [00000000] Irp is complete (CurrentLocation 2 > StackCount
1)
86329960 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
89feeae0 [00000000] irpStack: ( e, 0) 8a765030 [ \Driver\ACPI]
8a6d85d8 [99aa1040] irpStack: ( d, 0) 8b127018 [ \FileSystem\NTFS]
0x00000000
8a6dc828 [8bc758c0] irpStack: ( 4, 0) 8b127018 [ \FileSystem\NTFS]
0x00000000
8a6f42d8 [8bc728c0] irpStack: ( 4,34) 8b0b8030 [ \Driver\disk]
0x00000000
8a6f4d28 [8632e6c0] irpStack: ( 4,34) 8b0b8030 [ \Driver\disk]
0x00000000
8a767d98 [00000000] Irp is complete (CurrentLocation 6 > StackCount
5)
8a788d98 [00000000] irpStack: ( f, 0) 00000000 [00000000: Could not
read device
object or _DEVICE_OBJECT not found
]
8a7911a8 [9fdb4040] irpStack: ( e, 0) 86325768 [ \Driver\DeviceApi]
8b03c3f8 [00000000] Irp is complete (CurrentLocation 2 > StackCount
1)
8b0b8bc8 [863d6040] irpStack: ( e, 0) 8a78f030 [ \Driver\vmbus]
8b0c48c0 [91da8040] irpStack: ( e, 5) 8bcb2ca0 [ \Driver\AFD]
0xaa1a3540
8b118d98 [00000000] Irp is complete (CurrentLocation 9 > StackCount
8)
8b1263b8 [00000000] Irp is complete (CurrentLocation 8 > StackCount
7)
8b174008 [aa0aab80] irpStack: ( 4, 0) 8b127018 [ \FileSystem\NTFS]
0xa15e1c40
8b194008 [aa0aab80] irpStack: ( 4, 0) 8b127018 [ \FileSystem\NTFS]
0xa15e1c40
8b196370 [8b131880] irpStack: ( e,31) 8bcb2ca0 [ \Driver\AFD]
```


Args: 0004b000 00000000 4b3a0000 00000002

[IRP_MJ_WRITE(4), N/A(3)]

14 e0 8b0fc058 00000000 876c36a0-00000000 Success Error

Cancel

\Driver\partmgr

partmgr!PartitionIoCompletion

Args: 4b49ace4 00000000 4b3a0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

14 e0 8b121498 00000000 87531110-8b121a30 Success Error

Cancel

\Driver\partmgr

volmgr!VmpReadWriteCompletionRc

Args: 0004b000 00000000 2bea0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

4 e0 8b121978 00000000 82d103e0-8b1220d9 Success Error

Cancel

\Driver\volmgr

fvevol!FvePassThroughCompleti

Args: 0004b000 00000000 4b49acdf

00000000

[IRP_MJ_WRITE(4), N/A(0)]

4 e0 8b122020 00000000 82801a40-00000000 Success Error

Cancel

\Driver\fvevol

rdyboost!SmdReadWriteCompleti

Args: 0004b000 00000000 2bea0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

4 e1 8b118538 00000000 828637d0-00000000 Success Error

Cancel pending

\Driver\rdyboost

iorate!IoRateReadWriteCompleti

Args: 0004b000 3fffffff 2bea0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

4 e0 8b11ab80 00000000 82da1610-8b1240d8 Success Error

Cancel

\Driver\iorate

volsnap!VspRefCountCompletionRc

Args: 0004b000 00000000 2bea0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

4 e1 8b124020 00000000 87886ada-89aec208 Success Error

Cancel pending

\Driver\volsnap

NTFS!NtfsMasterIrpSyncCompleti

Args: 0004b000 00000000 2bea0000

00000002

[IRP_MJ_WRITE(4), N/A(0)]

4 e0 8b127018 a6de4bb8 871227b2-9ef8eba8 Success Error

Cancel

\FileSystem\NTFS

FLTMGR!FltpPassThroughComp

Args: 0004b000 00000000 00034000

00000000

[IRP_MJ_WRITE(4), N/A(0)]

4 1 8b12a3a0 a6de4bb8 00000000-00000000 pending
\FileSystem\FltMgr

Args: 0004b000 00000000 00034000

00000000

Irp Extension present at 0x8a6f4fb4:

This is a monstrous IRP with 15 stack locations (6 is current, shown in bold above, and is also specified by the debugger with the > character). The major and minor functions are shown for each stack location along with information on the device object and completion routines addresses.

The next step is to see what device object the IRP is targeting by executing the !devobj command on the device object address in the active stack location:

[Click here to view code image](#)

kd> !devobj **8b0b8030**

Device object (8b0b8030) is for:

DR0 \Driver\disk DriverObject 8b0a7e30
Current Irp 00000000 RefCount 1 Type 00000007 Flags 01000050
Vpb 8b0fc420 SecurityDescriptor 87da1b58 DevExt 8b0b80e8 DevObjExt
8b0b8578 Dope 8b0fc3d0
ExtensionFlags (0x00000800) DOE_DEFAULT_SD_PRESENT
Characteristics (0x00000100) FILE_DEVICE_SECURE_OPEN
AttachedDevice (Upper) 8b0fc058 \Driver\partmgr
AttachedTo (Lower) 8b0a4d10 \Driver\storflt
Device queue is not busy.

Finally, you can see details about the thread and process that issued the IRP by using the !thread command:

[Click here to view code image](#)

kd> !thread **8632e6c0**

THREAD 8632e6c0 Cid 0004.0058 Teb: 00000000 Win32Thread: 00000000
WAIT:

(Executive) KernelMode Non-Alertable

89aec20c NotificationEvent

IRP List:

8a6f4d28: (0006,02d4) Flags: 00060043 Mdl: 8b14b250

Not impersonating

DeviceMap 87c025b0

Owning Process 86264280

Image: **System**

Attached Process N/A

Image: N/A

Wait Start TickCount 8083

Ticks: 1 (0:00:00:00.015)

Context Switch Count 2223

IdealProcessor: 0

UserTime 00:00:00.000

KernelTime 00:00:00.046

Win32 Start Address nt!ExpWorkerThread (0x81e68710)
Stack Init 89aecca0 Current 89aebeb4 Base 89aed000 Limit 89aea000
Call 00000000
Priority 13 BasePriority 13 PriorityDecrement 0 IoPriority 2
PagePriority 5

I/O request to a single-layered hardware-based driver

This section traces I/O requests to a single-layered kernel-mode device driver. [Figure 6-14](#) shows a typical IRP processing scenario for such a driver.

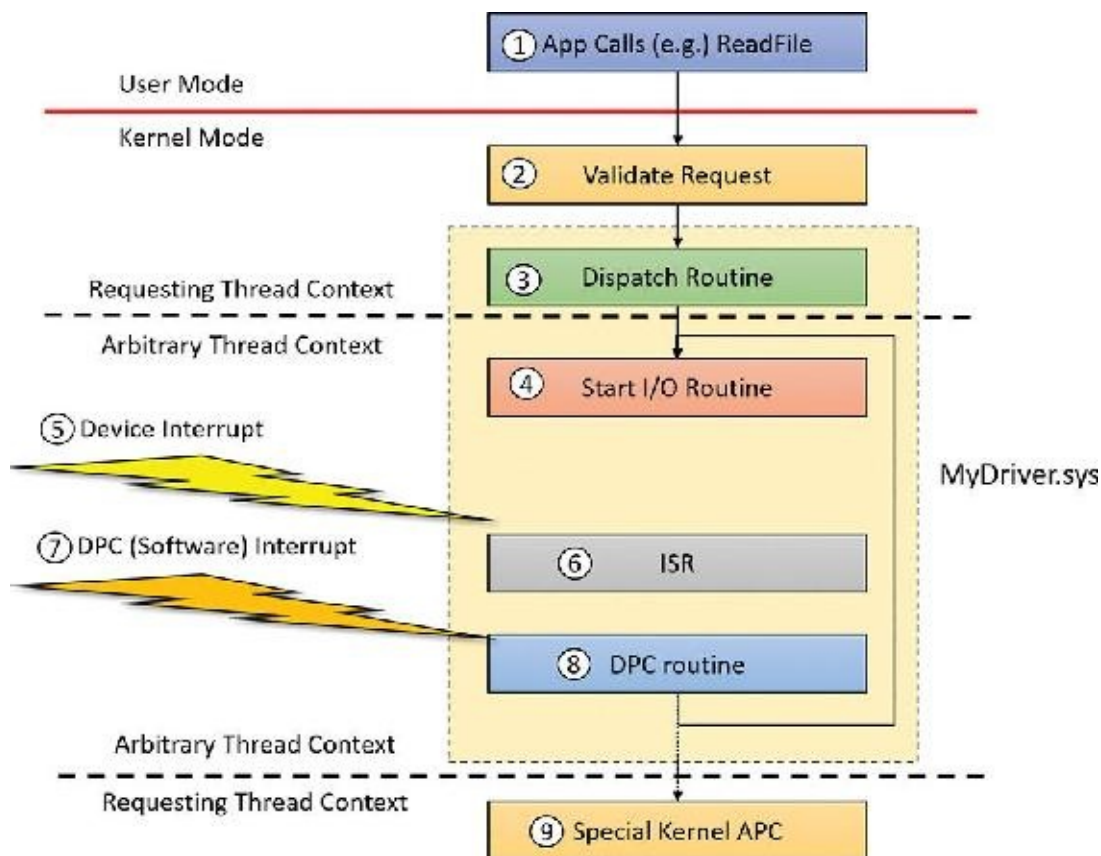


FIGURE 6-14 Typical single layer I/O request processing for hardware drivers.

Before we dig into the various steps outlined in [Figure 6-14](#), some general comments are in order:

- There are two types of horizontal divider lines. The first (solid line) is the usual user-mode/kernel-mode divider. The second (dotted line) separates code that runs in the requesting thread context versus the arbitrary thread context. These contexts are defined as follows:
 - The requesting thread context region indicates that the executing thread is the original one that requested the I/O operation. This is important because if the thread is the one that made the original call, it means the process context is the original process, and so the user-mode address space that contains the user's buffer supplied to the I/O operation is directly accessible.
 - The arbitrary thread context region indicates that the thread running those functions can be any thread. More specifically, it's most likely *not* the requesting thread, and so the user-mode process address space visible is not likely to be the original one. In this context, accessing the user's buffer with a user-mode address can be disastrous. You'll see in the next section how this issue is handled.



Note

The explanations for the steps outlined in [Figure 6-14](#) will prove why the divider lines reside where they are.

- The large rectangle consisting of the four blocks (labeled Dispatch Routine, Start I/O Routine, ISR, and DPC Routine) represents the driver-provided code. All other blocks are provided by the system.
- The figure assumes the hardware device can handle one operation at a time, which is true of many types of devices. Even if the device can handle multiple requests, the basic flow of operations is still the same.

Here is the sequence of events as outlined in [Figure 6-14](#):

1. A client application calls a Windows API such as `ReadFile`. `ReadFile` calls the native `NtReadFile` (in `Ntdll.dll`), which makes the thread transition to kernel mode to the executive `NtReadFile` (these steps have already been discussed earlier in this chapter).
2. The I/O manager, in its `NtReadFile` implementation, performs some sanity checks on the request, such as whether the buffer provided by the client is accessible with the right page protection. Next, the I/O manager locates the associated driver (using the file handle provided), allocates and initializes an IRP, and calls the driver into the appropriate dispatch routine (in this case, corresponding to the `IRP_MJ_READ` index) using `IoCallDriver` with the IRP.
3. This is the first time the driver sees the IRP. This call is usually invoked using the requesting thread; the only way for that *not* to happen is if an upper filter held on to the IRP and called `IoCallDriver` later from a different thread. For the sake of this discussion, we'll assume this is not the case (and in most cases involving hardware devices, this does not happen; even if there are upper filters, they do some processing and call the lower driver immediately from the same thread). The dispatch read callback in the driver has two tasks on its hand: first, it should perform more checking that the I/O manager can't do because it has no idea what the request really means. For example, the driver could check if the buffer provided to a read or write operation is large enough; or for a `DeviceIoControl` operation, the driver would check whether the I/O control code provided is a supported one. If any such check fails, the driver completes the IRP (`IoCompleteRequest`) with the failed status and returns immediately. If the checks turn up OK, the driver calls its Start I/O routine to initiate the operation. However, if the hardware device is currently busy (handling a previous IRP), then the IRP should be inserted into a queue managed by the driver and a `STATUS_PENDING` is returned without completing the IRP. The I/O manager caters for such a scenario with the `IoStartPacket` function, that checks a busy bit in the device object and, if the device is busy, inserts the IRP into a queue (also part of the device object structure). If the device is not busy, it sets the device bit as busy and calls the registered Start I/O routine (recall that there is such a member in the driver object that would have been initialized in `DriverEntry`). Even if a driver chooses not to use `IoStartPacket`, it would still follow similar logic.
4. If the device is not busy, the Start I/O routine is called from the dispatch routine directly—meaning it's still the requesting thread that is making the call. [Figure 6-14](#), however, shows that the Start I/O routine is called in an arbitrary thread context; this will be proven to be true in the general case when we look at the DPC routine in step 8. The purpose of the Start I/O routine is to take the IRP

relevant parameters and use them to program the hardware device (for example, by writing to its ports or registers using HAL hardware access routines such as `WRITE_PORT_UCHAR`, `WRITE_REGISTER_ULONG`, etc.). After the Start I/O completes, the call returns, and no particular code is running in the driver, the hardware is working and “does its thing.” While the hardware device is working, more requests can come in to the device by the same thread (if using asynchronous operations) or other threads that also opened handles to the device. In this case the dispatch routine would realize the device is busy and insert the IRP into the IRP queue (as mentioned, one way to achieve this is with a call to `IoStartPacket`).

5. When the device is done with the current operation, it raises an interrupt. The kernel trap handler saves the CPU context for whatever thread was running on the CPU that was selected to handle the interrupt, raises the IRQL of that CPU to the IRQL associated with the interrupt (DIRQL) and jumps to the registered ISR for the device.
6. The ISR, running at Device IRQL (above 2) does as little work as possible, telling the device to stop the interrupt signal and getting the status or other required information from the hardware device. As its last act, the ISR queues a DPC for further processing at a lower IRQL. The advantage of using a DPC to perform most of the device servicing is that any blocked interrupt whose IRQL lies between the Device IRQL and the DPC/dispatch IRQL (2) is allowed to occur before the lower-priority DPC processing occurs. Intermediate-level interrupts are thus serviced more promptly than they otherwise would be, and this reduces latency on the system.
7. After the interrupt is dismissed, the kernel notices that the DPC queue is not empty and so uses a software interrupt at IRQL DPC_LEVEL (2) to jump to the DPC processing loop.
8. Eventually, the DPC is de-queued and executes at IRQL 2, typically performing two main operations:
 - It gets the next IRP in the queue (if any) and starts the new operation for the device. This is done first to prevent the device from being idle for too long. If the dispatch routine used `IoStartPacket`, then the DPC routine would call its counterpart, `IoStartNextPacket`, which does just that. If an IRP is available, the Start I/O routine is called from the DPC. This is why in the general case, the Start I/O routine is called in an arbitrary thread context. If there are no IRPs in the queue, the device is marked not busy—that is, ready for the next request that comes in.
 - It completes the IRP, whose operation has just finished by the driver by calling `IoCompleteRequest`. From that point, the driver is no longer responsible for the IRP and it shouldn't be touched, as it can be freed at any moment after the call. `IoCompleteRequest` calls any completion routines that have been registered. Finally, the I/O manager frees the IRP (it's actually using a completion routine of its own to do that).
9. The original requesting thread needs to be notified of the completion. Because the current thread executing the DPC is arbitrary, it's not the original thread with its original process address space. To execute code in the context of the requesting thread, a special kernel APC is issued to the thread. An APC is a function that is forced to execute in the context of a particular thread. When the requesting thread gets CPU time, the special kernel APC executes first (at IRQL APC_LEVEL=1). It does what's needed, such as releasing the thread from waiting, signaling an event that was registered in an asynchronous operation, and so on. (For more on APCs, see Chapter 8 in Part 2.)

A final note about I/O completion: the asynchronous I/O functions `ReadFileEx` and `WriteFileEx` allow a caller to supply a callback function as a parameter. If the caller does so, the I/O manager queues a user mode APC to the caller's thread APC queue as the last step of I/O completion. This feature allows a

caller to specify a subroutine to be called when an I/O request is completed or canceled. User-mode APC completion routines execute in the context of the requesting thread and are delivered only when the thread enters an alertable wait state (by calling functions such as `SleepEx`, `WaitForSingleObjectEx`, or `WaitForMultipleObjectsEx`).

User address space buffer access

As shown in [Figure 6-14](#), there are four main driver functions involved in processing an IRP. Some or all of these routines may need to access the buffer in user space provided by the client application. When an application or a device driver indirectly creates an IRP by using the `NtReadFile`, `NtWriteFile`, or `NtDeviceIoControlFile` system services (or the Windows API functions corresponding to these services, which are `ReadFile`, `WriteFile`, and `DeviceIoControl`), the pointer to the user's buffer is provided in the `UserBuffer` member of the IRP body. However, accessing this buffer directly can be done only in the requesting thread context (the client's process address space is visible) and in IRQL 0 (paging can be handled normally).

As discussed in the previous section, only the dispatch routine meets the criteria of running in the requesting thread context and in IRQL 0. And even this is not always the case—it's possible for an upper filter to hold on to the IRP and not pass it down immediately, possibly passing it down later on using a different thread, and could even be done when the CPU IRQL is 2 or higher.

The other three functions (Start I/O, ISR, DPC) clearly run on an arbitrary thread (could be any thread), and with IRQL 2 (DIRQL for the ISR). Accessing the user's buffer directly from any of these routine is mostly fatal. Here's why:

- Because the IRQL is 2 or higher, paging is not allowed. Since the user's buffer (or part of it) may be paged out, accessing the non-resident memory would crash the system.
- Because the thread executing these functions could be any thread, and thus a random process address space would be visible, the original user's address has no meaning and would likely lead to an access violation, or worse—accessing data from some random process (the parent process of whatever thread was running at the time).

Clearly, there must be a safe way to access the user's buffer in any of these routines. The I/O manager provides two options, for which it does the heavy lifting. These are known as Buffered I/O and Direct I/O. A third option, which is not really an option, is called Neither I/O, in which the I/O manager does nothing special and lets the driver handle the problem on its own.

A driver selects the method in the following way:

- For read and write requests (`IRP_MJ_READ` and `IRP_MJ_WRITE`), it sets the `Flags` member (with an OR boolean operation so as not to disturb other flags) of the device object (`DEVICE_OBJECT`) to `DO_BUFFERED_IO` (for buffered I/O) or `DO_DIRECT_IO` (for direct I/O). If neither flag is set, neither I/O is implied. (`DO` is short for *device object*.)
- For device I/O control requests (`IRP_MJ_DEVICE_CONTROL`), each control code is constructed using the `CTL_CODE` macro, where some of the bits indicate the buffering method. This means the buffering method can be set on a control code-by-control code basis, which is very useful.

The following sections describe each buffering method in detail.

Buffered I/O With buffered I/O, the I/O manager allocates a mirror buffer that is the same size as the user's buffer in non-paged pool and stores the pointer to the new buffer in the `AssociatedIrp.SystemBuffer` member of the IRP body. [Figure 6-15](#) shows the main stages in buffered I/O for a read operation (write is similar).

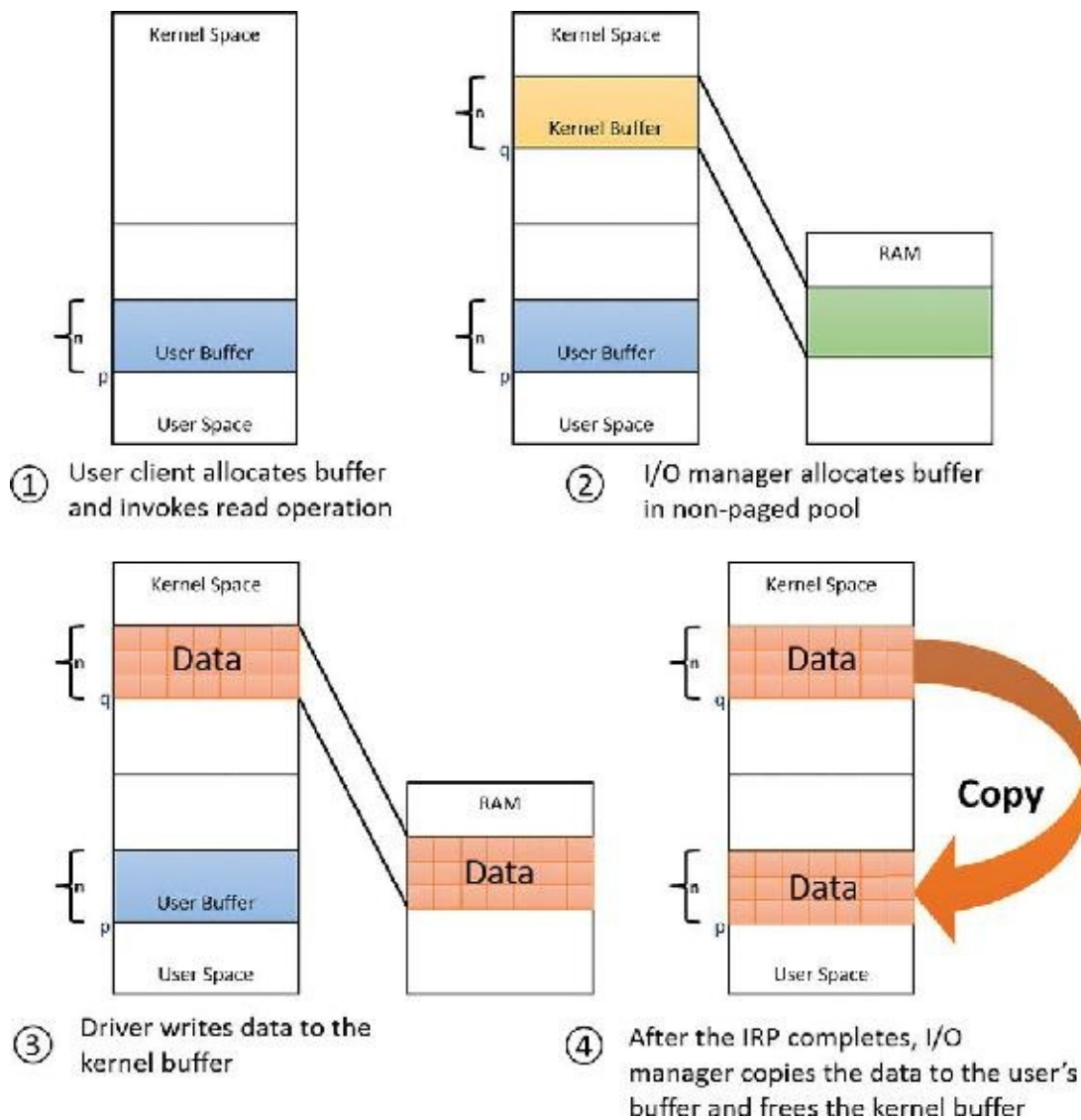


FIGURE 6-15 Buffered I/O.

The driver can access the system buffer (address q in [Figure 6-15](#)) from any thread and any IRQL:

- The address is in system space, meaning it's valid in any process context.
- The buffer is allocated from non-paged pool, so a page fault will not happen.

For write operations, the I/O manager copies the caller's buffer data into the allocated buffer when creating the IRP. For read operations, the I/O manager copies data from the allocated buffer to the user's buffer when the IRP completes (using a special kernel APC) and then frees the allocated buffer.

Buffered I/O clearly is very simple to use because the I/O manager does practically everything. Its main downside is that it always requires copying, which is inefficient for large buffers. Buffered I/O is commonly used when the buffer size is no larger than one page (4 KB) and when the device does not support direct memory access (DMA), because DMA is used to transfer data from a device to RAM or vice versa without CPU intervention—but with buffered I/O, there is always copying done with the CPU, which makes DMA pointless.

Direct I/O Direct I/O provides a way for a driver to access the user's buffer directly without any need for copying. [Figure 6-16](#) shows the main stages in direct I/O for a read or write operation.

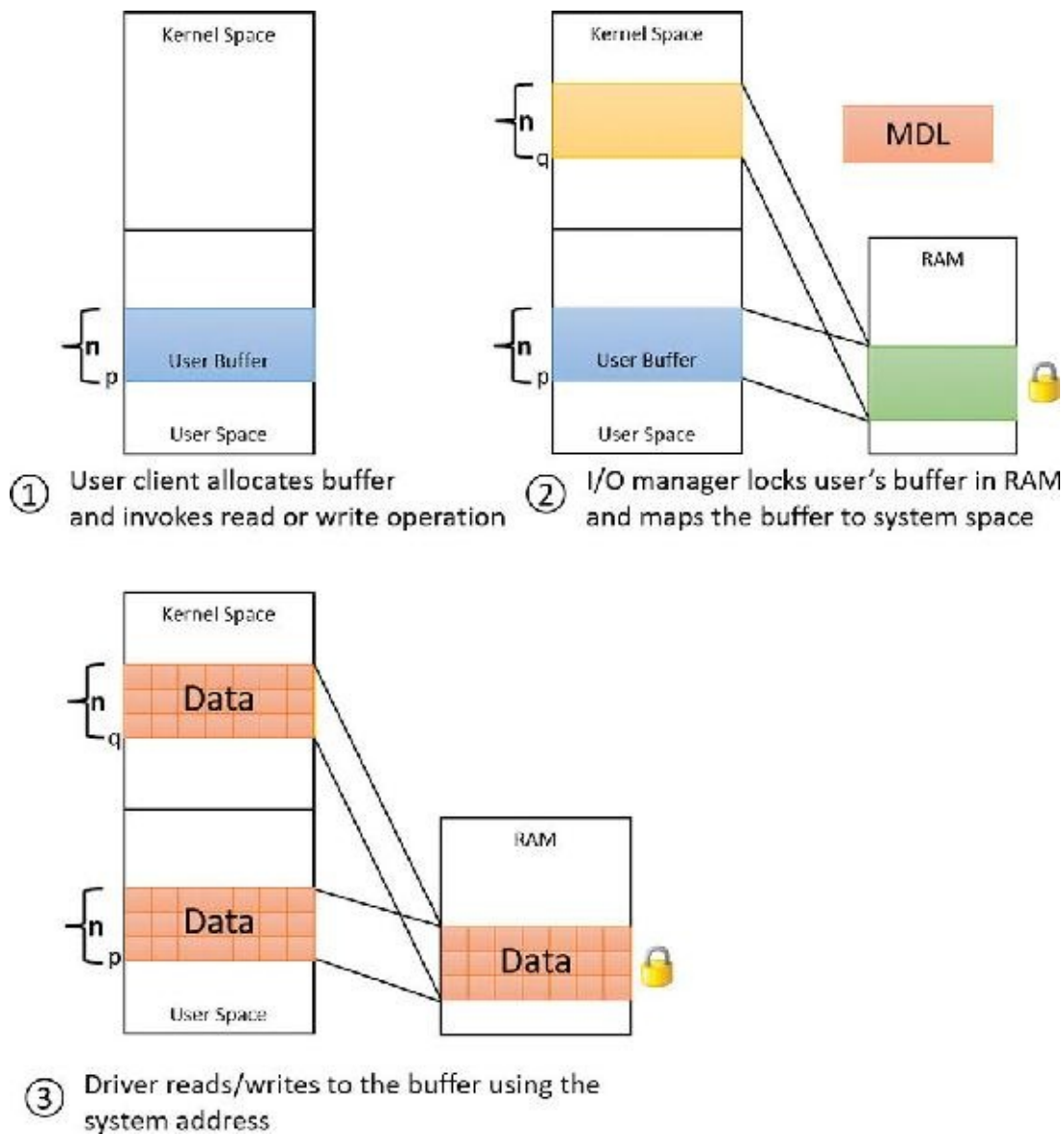


FIGURE 6-16 Direct I/O.

When the I/O manager creates the IRP, it locks the user's buffer into memory (that is, makes it non-pageable) by calling the `MmProbeAndLockPages` function (documented in the WDK). The I/O manager stores a description of the memory in the form of a memory descriptor list (MDL), which is a structure that describes the physical memory occupied by a buffer. Its address is stored in the `MdlAddress` member of the IRP body. Devices that perform DMA require only physical descriptions of buffers, so an MDL is sufficient for the operation of such devices. If a driver must access the contents of a buffer, however, it can map the buffer into the system's address space using the `MmGetSystemAddressForMdlSafe` function, passing in the provided MDL. The resulting pointer (q in [Figure 6-16](#)) is safe to use in any thread context (it's a system address) and in any IRQL (the buffer cannot be paged out). The user's buffer is effectively double-mapped, where the user's direct address (p in [Figure 6-16](#)) is usable only from the original process context, but the second mapping into system space is usable in any context. Once the IRP is complete, the I/O manager unlocks the buffer (making it pageable again) by calling `MmUnlockPages` (documented in the WDK).

Direct I/O is useful for large buffers (more than one page) because no copying is done, especially for DMA transfers (for the same reason).

Neither I/O With neither I/O, the I/O manager doesn't perform any buffer management. Instead, buffer management is left to the discretion of the device driver, which can choose to manually perform the steps the I/O manager performs with the other buffer-management types. In some cases, accessing the buffer in

the dispatch routine is sufficient, so the driver may get away with neither I/O. The main advantage of neither I/O is its zero overhead.

Drivers that use neither I/O to access buffers that might be located in user space must take special care to ensure that buffer addresses are valid and do not reference kernel-mode memory. Scalar values, however, are perfectly safe to pass, although very few drivers have only a scalar value to pass around. Failure to do so could result in crashes or in security vulnerabilities, where applications have access to kernel-mode memory or can inject code into the kernel. The `ProbeForRead` and `ProbeForWrite` functions that the kernel makes available to drivers verify that a buffer resides entirely in the user-mode portion of the address space. To avoid a crash from referencing an invalid user-mode address, drivers can access user-mode buffers protected with structured exception handling (SEH), expressed with `__try/__except` blocks in C/C++, that catch any invalid memory faults and translate them into error codes to return to the application. (See Chapter 8 in Part 2 for more information on SEH.) Additionally, drivers should also capture all input data into a kernel buffer instead of relying on user-mode addresses because the caller could always modify the data behind the driver's back, even if the memory address itself is still valid.

Synchronization

Drivers must synchronize their access to global driver data and hardware registers for two reasons:

- The execution of a driver can be preempted by higher-priority threads and time-slice (or quantum) expiration or can be interrupted by higher IRQL interrupts.
- On multiprocessor systems (the norm), Windows can run driver code simultaneously on more than one processor.

Without synchronization, corruption could occur—for example, device-driver code running at passive IRQL (0) (say, a dispatch routine) when a caller initiates an I/O operation can be interrupted by a device interrupt, causing the device driver's ISR to execute while its own device driver is already running. If the device driver was modifying data that its ISR also modifies—such as device registers, heap storage, or static data—the data can become corrupted when the ISR executes.

To avoid this situation, a device driver written for Windows must synchronize its access to any data that can be accessed at more than one IRQL. Before attempting to update shared data, the device driver must lock out all other threads (or, in the case of a multiprocessor system, CPUs) to prevent them from updating the same data structure.

On a single-CPU system, synchronizing between two or more functions that run at different IRQLs is easy enough. Such function just needs to raise the IRQL (`KeRaiseIrql`) to the highest IRQL these functions execute in. For example, to synchronize between a dispatch routine (IRQL 0) and a DPC routine (IRQL 2), the dispatch routine needs to raise IRQL to 2 before accessing the shared data. If synchronization between a DPC and ISR is required, the DPC would raise IRQL to the Device IRQL (this information is provided to the driver when the PnP manager informs the driver of the hardware resources a device is connected to.) On multiprocessing systems, raising IRQL is not enough because the other routine—for example, ISR—could be serviced on another CPU (remember that IRQL is a CPU attribute, and not a global system attribute).

To allow high IRQL synchronization across CPUs, the kernel provides a specialized synchronization object: the spinlock. Here, we'll take a brief look at spinlocks as they apply to driver synchronization. (A full treatment of spinlocks is reserved for Chapter 8 in Part 2.) In principle, a spinlock resembles a mutex (also discussed in detail in Chapter 8 in Part 2) in the sense that it allows one piece of code to access shared data, but it works and is used quite differently. [Table 6-3](#) summarizes the differences between

mutexes and spinlocks.

	Mutex	Spinlock
Synchronization nature	One thread out of any number of threads that is allowed enters a critical region and accesses shared data.	One CPU out of any number of CPUs that is allowed enters a critical region and accesses shared data.
Usable at IRQL	< DISPATCH_LEVEL (2)	>= DISPATCH_LEVEL (2)
Wait kind	Normal. That is, it does not waste CPU cycles while waiting	Busy. That is, the CPU is constantly testing the spinlock bit until it's free.
Ownership	The owner thread is tracked, and recursive acquisition is allowed.	The CPU owner is not tracked, and recursive acquisition will cause a deadlock.

TABLE 6-3 Mutexes versus spinlocks

A spinlock is just a bit in memory that is accessed by an atomic test and modify operation. A spinlock may be owned by a CPU or free (unowned). As shown in [Table 6-3](#), spinlocks are necessary when synchronization is needed in high IRQLs (≥ 2), because a mutex can't be used in these cases as a scheduler is needed, but as we've seen the scheduler cannot wake up on a CPU whose IRQL is 2 or higher. This is why waiting for a spinlock is a busy wait operation: The thread cannot go to a normal wait state because that implies the scheduler waking up and switching to another thread on that CPU.

Acquiring a spinlock by a CPU is always a two-step operation. First, the IRQL is raised to the associated IRQL on which synchronization is to occur—that is, the highest IRQL on which the function that needs to synchronize executes. For example, synchronizing between a dispatch routine (IRQL 0) and a DPC (2) would need to raise IRQL to 2; synchronizing between DPC (2) and ISR (DIRQL) would need to raise IRQL to DIRQL (the IRQL for that particular interrupt). Second, the spinlock is attempted acquisition by atomically testing and setting the spinlock bit.



Note

The steps outlined for spinlock acquisition are simplified and omit some details that are not important for this discussion. The complete spinlock story is described in Chapter 8 in Part 2.

The functions that acquire spinlocks determine the IRQL on which to synchronize, as we shall see in a moment.

[Figure 6-17](#) shows a simplified view of the two-step process of acquiring a spinlock.

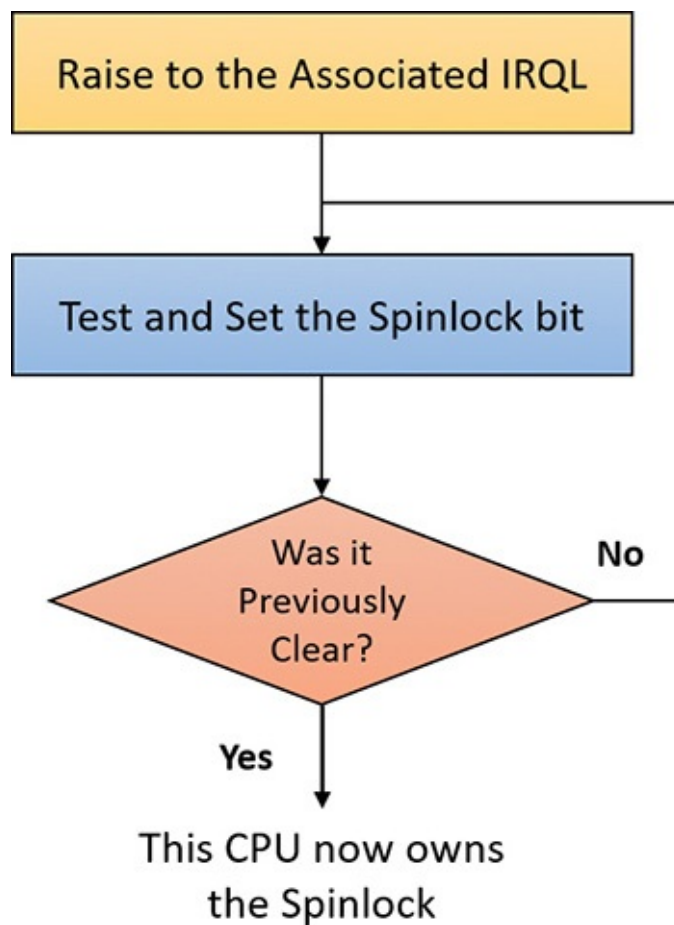


FIGURE 6-17 Spinlock acquisition.

When synchronizing at IRQL 2—for example, between a dispatch routine and a DPC or between a DPC and another DPC (running on another CPU, of course)—the kernel provides the `KeAcquireSpinLock` and `KeReleaseSpinLock` functions (there are other variations that are discussed in Chapter 8 in Part 2). These functions perform the steps in [Figure 6-17](#) where the “associated IRQL” is 2. The driver in this case must allocate a spinlock (`KSPIN_LOCK`, which is just 4 bytes on 32-bit systems and 8 bytes on 64-bit systems), typically in the device extension (where driver-managed data for the device is kept) and initialize it with `KeInitializeSpinLock`.

For synchronizing between any function (such as DPC or a dispatch routine) and the ISR, different functions must be used. Every interrupt object (`KINTERRUPT`) holds inside it a spinlock, which is acquired before the ISR executes (this implies that the same ISR cannot run concurrently on other CPUs). Synchronization in this case would be with that particular spinlock (no need to allocate another one), which can be acquired indirectly with the `KeAcquireInterruptSpinLock` function and released with `KeReleaseInterruptSpinLock`. Another option is to use the `KeSynchronizeExecution` function, which accepts a callback function the driver provides that is called between the acquisition and release of the interrupt spinlock.

By now, you should realize that although ISRs require special attention, any data that a device driver uses is subject to being accessed by the same device driver (one of its functions) running on another processor. Therefore, it’s critical for device-driver code to synchronize its use of any global or shared data or any accesses to the physical device itself.

I/O requests to layered drivers

The “[IRP flow](#)” section showed the general options drivers have for dealing with IRPs, with a focus on a standard WDM device node. The preceding section showed how an I/O request to a simple device controlled by a single device driver is handled. I/O processing for file-based devices or for requests to other layered drivers happens in much the same way, but it’s worthwhile to take a closer look at a request targeted at file-system drivers. [Figure 6-18](#) shows a very simplified illustrative example of how an asynchronous I/O request might travel through layered drivers for non-hardware based devices as primary targets. It uses as an example a disk controlled by a file system.

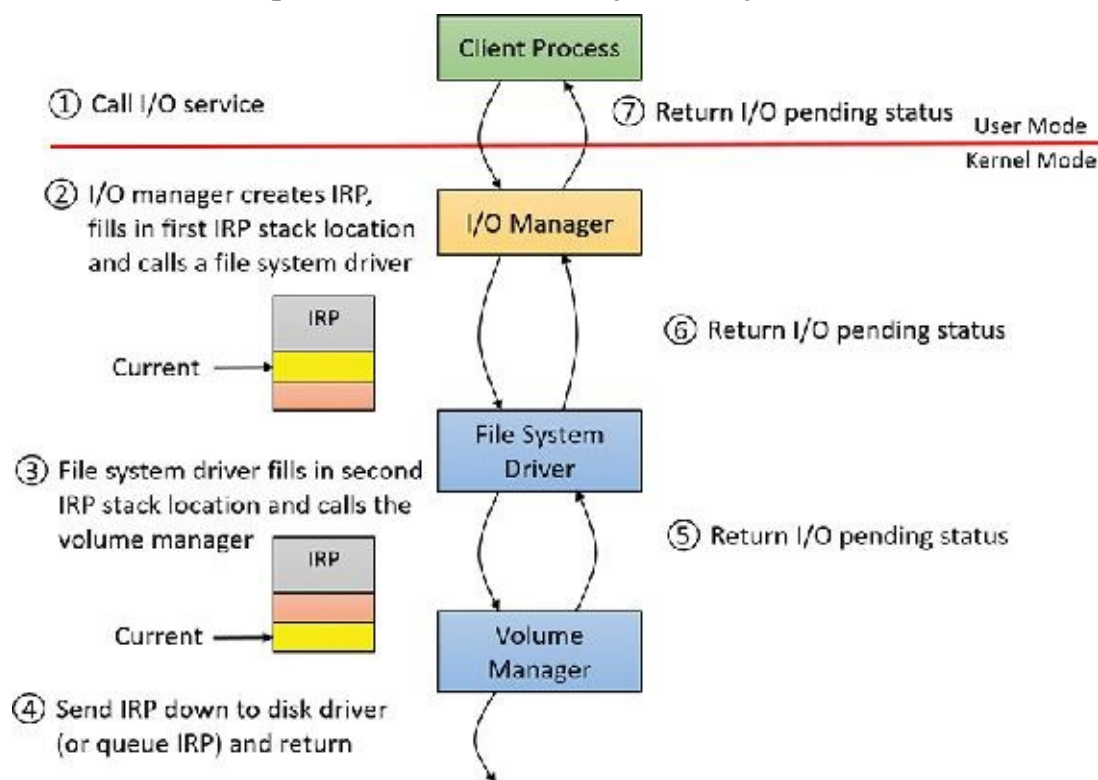


FIGURE 6-18 Queuing an asynchronous request to layered drivers.

Once again, the I/O manager receives the request and creates an IRP to represent it. This time, however, it delivers the packet to a file-system driver. The file-system driver exercises great control over the I/O operation at that point. Depending on the type of request the caller made, the file system can send the same IRP to the disk driver or it can generate additional IRPs and send them separately to the disk driver.

The file system is most likely to reuse an IRP if the request it receives translates into a single straightforward request to a device. For example, if an application issues a read request for the first 512 bytes in a file stored on a volume, the NTFS file system would simply call the volume manager driver, asking it to read one sector from the volume, beginning at the file’s starting location.

After the disk controller’s DMA adapter finishes a data transfer, the disk controller interrupts the host, causing the ISR for the disk controller to run, which requests a DPC callback completing the IRP, as shown in [Figure 6-19](#).

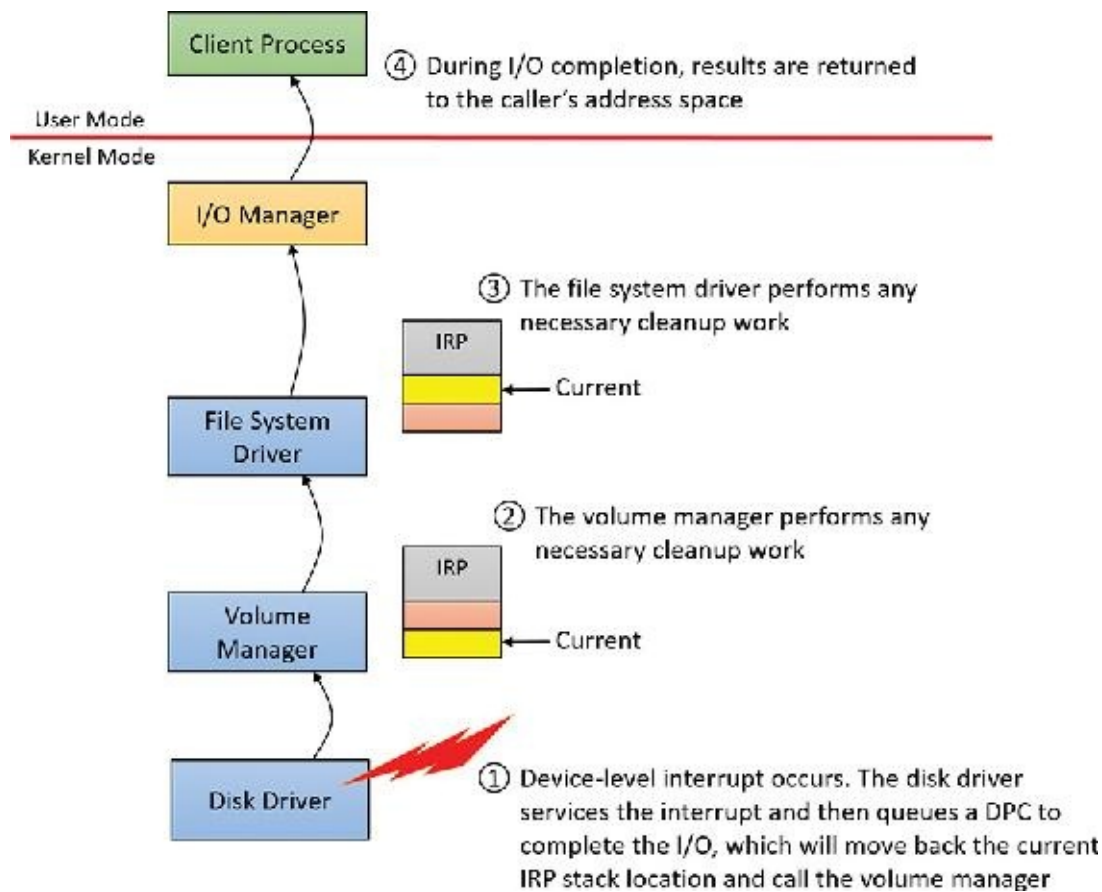


FIGURE 6-19 Completing a layered I/O request.

As an alternative to reusing a single IRP, a file system can establish a group of associated IRPs that work in parallel on a single I/O request. For example, if the data to be read from a file is dispersed across the disk, the file-system driver might create several IRPs, each of which reads some portion of the request from a different sector. This queuing is illustrated in [Figure 6-20](#).

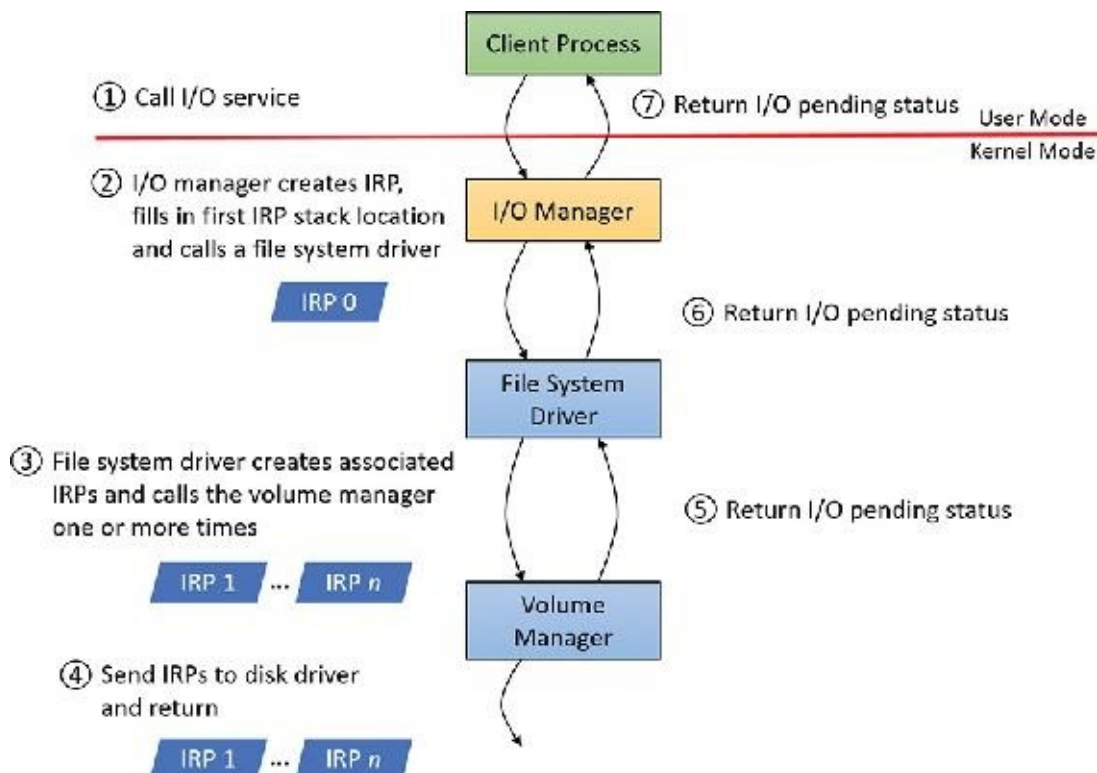


FIGURE 6-20 Queuing associated IRPs.

The file-system driver delivers the associated IRPs to the volume manager, which in turn sends them to

the disk-device driver, which queues them to the disk device. They are processed one at a time, and the file-system driver keeps track of the returned data. When all the associated IRPs complete, the I/O system completes the original IRP and returns to the caller, as shown in [Figure 6-21](#).

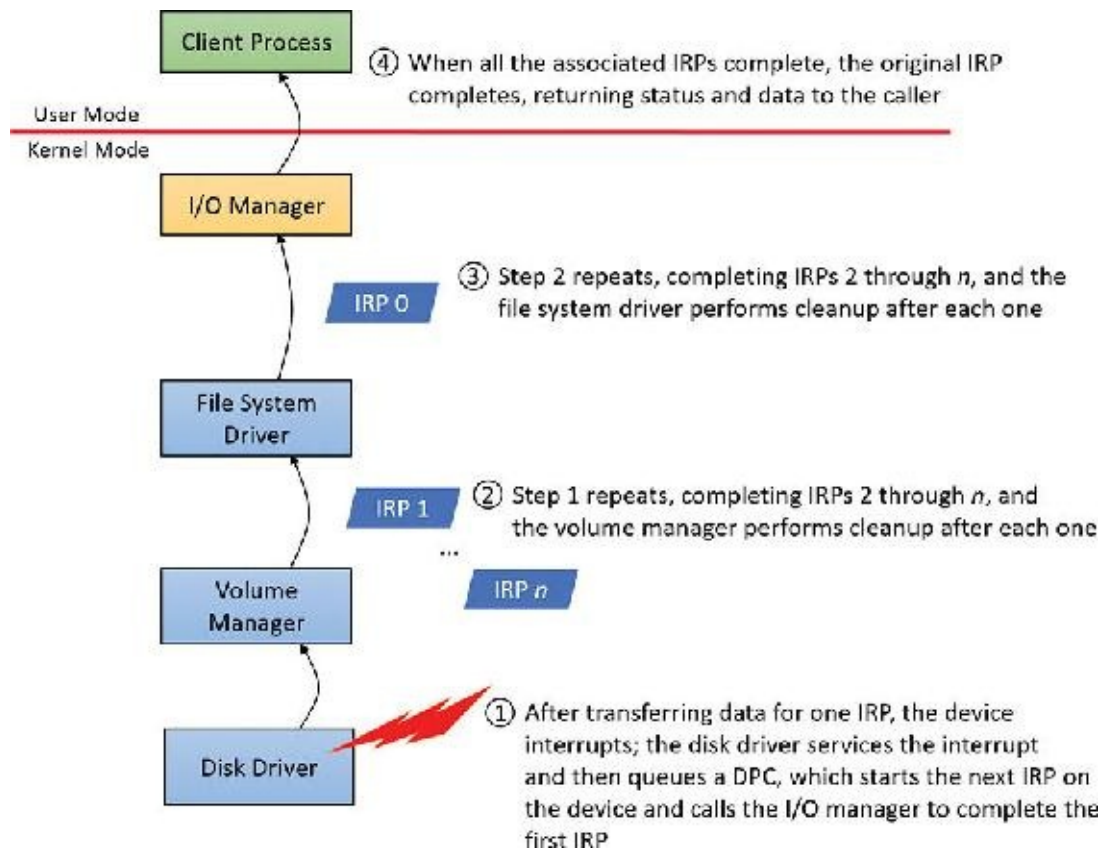


FIGURE 6-21 Completing associated IRPs.



Note

All Windows file-system drivers that manage disk-based file systems are part of a stack of drivers that is at least three layers deep. The file-system driver sits at the top, a volume manager in the middle, and a disk driver at the bottom. In addition, any number of filter drivers can be interspersed above and below these drivers. For clarity, the preceding example of layered I/O requests includes only a file-system driver and the volume-manager driver. See Chapter 12 in Part 2 for more information.

Thread-agnostic I/O

In the I/O models described thus far, IRPs are queued to the thread that initiated the I/O and are completed by the I/O manager issuing an APC to that thread so that process-specific and thread-specific context are accessible by completion processing. Thread-specific I/O processing is usually sufficient for the performance and scalability needs of most applications, but Windows also includes support for *thread-agnostic I/O* via two mechanisms:

- I/O completion ports, which are described at length in the section “[I/O completion ports](#)” later in this chapter
- Locking the user buffer into memory and mapping it into the system address space

With I/O completion ports, the application decides when it wants to check for the completion of I/O. Therefore, the thread that happens to have issued an I/O request is not necessarily relevant because any

other thread can perform the completion request. As such, instead of completing the IRP inside the specific thread's context, it can be completed in the context of any thread that has access to the completion port.

Likewise, with a locked and kernel-mapped version of the user buffer, there's no need to be in the same memory address space as the issuing thread because the kernel can access the memory from arbitrary contexts. Applications can enable this mechanism by using `SetFileIoOverlappedRange` as long as they have the `SeLockMemoryPrivilege`.

With both completion port I/O and I/O on file buffers set by `SetFileIoOverlappedRange`, the I/O manager associates the IRPs with the file object to which they have been issued instead of with the issuing thread. The `!fileobj` extension in WinDbg shows an IRP list for file objects that are used with these mechanisms.

In the next sections, you'll see how thread-agnostic I/O increases the reliability and performance of applications in Windows.

I/O cancellation

While there are many ways in which IRP processing occurs and various methods to complete an I/O request, a great many I/O processing operations actually end in cancellation rather than completion. For example, a device may require removal while IRPs are still active, or the user might cancel a long-running operation to a device—for example, a network operation. Another situation that requires I/O cancellation support is thread and process termination. When a thread exits, the I/Os associated with the thread must be cancelled. This is because the I/O operations are no longer relevant and the thread cannot be deleted until the outstanding I/Os have completed.

The Windows I/O manager, working with drivers, must deal with these requests efficiently and reliably to provide a smooth user experience. Drivers manage this need by registering a cancel routine, by calling `IoSetCancelRoutine`, for their cancellable I/O operations (typically, those operations that are still enqueued and not yet in progress), which is invoked by the I/O manager to cancel an I/O operation. When drivers fail to play their role in these scenarios, users may experience unkillable processes, which have disappeared visually but linger and still appear in Task Manager or Process Explorer.

User-initiated I/O cancellation

Most software uses one thread to handle user interface (UI) input and one or more threads to perform work, including I/O. In some cases, when a user wants to abort an operation that was initiated in the UI, an application might need to cancel outstanding I/O operations. Operations that complete quickly might not require cancellation, but for operations that take arbitrary amounts of time—like large data transfers or network operations—Windows provides support for cancelling both synchronous and asynchronous operations.

- **Cancelling synchronous I/Os** A thread can call `CancelSynchronousIo`. This enables even create (open) operations to be cancelled when supported by a device driver. Several drivers in Windows support this functionality. These include drivers that manage network file systems (for example, MUP, DFS, and SMB), which can cancel open operations to network paths.
- **Cancelling asynchronous I/Os** A thread can cancel its own outstanding asynchronous I/Os by calling `CancelIo`. It can cancel all asynchronous I/Os issued to a specific file handle, regardless of which thread initiated them, in the same process with `CancelIoEx`. `CancelIoEx` also works on operations associated with I/O completion ports through the aforementioned thread-agnostic support in Windows. This is because the I/O system keeps track of a completion port's outstanding

I/Os by linking them with the completion port.

[Figure 6-22](#) and [Figure 6-23](#) show synchronous and asynchronous I/O cancellation. (To a driver, all cancel processing looks the same.)

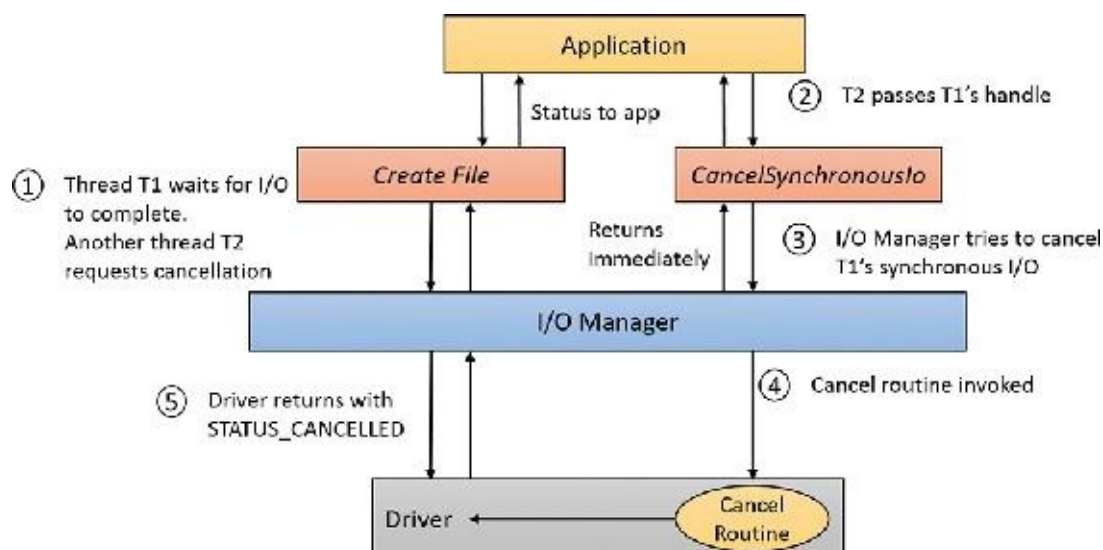


FIGURE 6-22 Synchronous I/O cancellation.

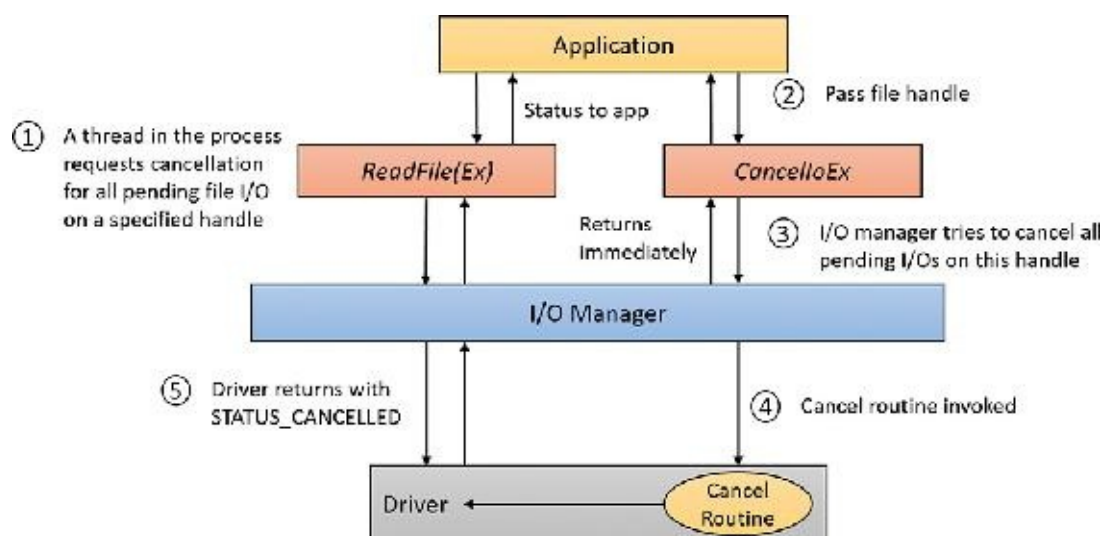


FIGURE 6-23 Asynchronous I/O cancellation.

I/O cancellation for thread termination

The other scenario in which I/Os must be cancelled is when a thread exits, either directly or as a result of its process terminating (which causes the threads of the process to terminate). Because every thread has a list of IRPs associated with it, the I/O manager can walk this list, look for cancellable IRPs, and cancel them. Unlike `CancelIoEx`, which does not wait for an IRP to be cancelled before returning, the process manager will not allow thread termination to proceed until all I/Os have been cancelled. As a result, if a driver fails to cancel an IRP, the process and thread object will remain allocated until the system shuts down.



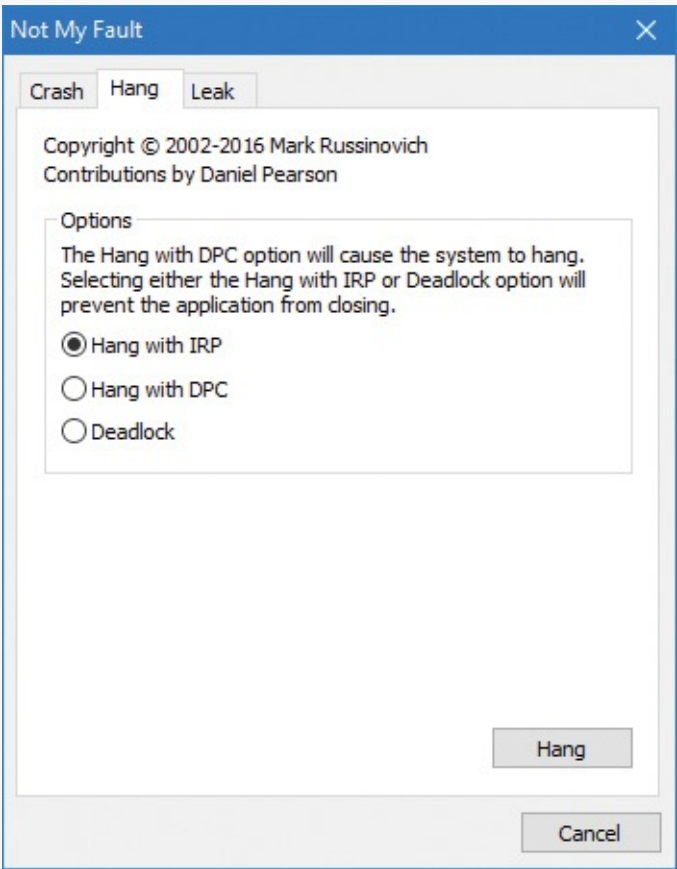
Note

Only IRPs for which a driver sets a cancel routine are cancellable. The process manager waits until all I/Os associated with a thread are either cancelled or completed before deleting the thread.

EXPERIMENT: Debugging an unkillable process

In this experiment, we'll use Notmyfault from Sysinternals to force an unkillable process by causing the Myfault.sys driver, which Notmyfault.exe uses, to indefinitely hold an IRP without having registered a cancel routine for it. (Notmyfault is covered in detail in the "Crash dump analysis" section of Chapter 15, "Crash dump analysis," in Part 2.) Follow these steps:

1. Run Notmyfault.exe.
2. The Not My Fault dialog box appears. Click the **Hang** tab and choose **Hang with IRP**, as shown in the following screenshot. Then click the **Hang** button.



3. You shouldn't see anything happen, and you should be able to click the **Cancel** button to quit the application. However, you should still see the Notmyfault process in Task Manager or

Process Explorer. Attempts to terminate the process will fail because Windows will wait forever for the IRP to complete given that the Myfault driver doesn't register a cancel routine.

4. To debug an issue such as this, you can use WinDbg to look at what the thread is currently doing. Open a local kernel debugger session and start by listing the information about the Notmyfault.exe process with the !process command (notmyfault64 is the 64-bit version):

[Click here to view code image](#)

```
lkd> !process 0 7 notmyfault64.exe
PROCESS fffff8c0b88c823c0
    SessionId: 1 Cid: 2b04 Peb: 4e5c9f4000 ParentCid: 0d40
    DirBase: 3edfa000 ObjectTable:
fffffdf08dd140900 HandleCount: <Data Not
Accessible>
    Image: notmyfault64.exe
    VadRoot fffff8c0b863ed190 Vads 81 Clone 0 Private 493.
Modified 8. Locked
0....
    THREAD fffff8c0b85377300 Cid 2b04.2714 Teb:
00000004e5c808000
Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode Non-
Alertable
    fffff80a4c944018 SynchronizationEvent
    IRP List:
    fffff8c0b84f1d130: (0006,0118) Flags: 00060000 Mdl:
000000000
    Not impersonating
    DeviceMap fffffdf08cf4d7d20
    Owning Process fffff8c0b88c823c0 Image:
notmyfault64.exe
...
    Child-SP RetAddr : Args to Child
: Call Site
    fffffb881'3ecf74a0 fffff802'cfc38a1c : 00000000'00000100
00000000'00000000 00000000'00000000 00000000'00000000 :
nt!KiSwapContext+0x76
    fffffb881'3ecf75e0 fffff802'cfc384bf : 00000000'00000000
00000000'00000000 00000000'00000000 00000000'00000000 :
nt!KiSwapThread+0x17c
    fffffb881'3ecf7690 fffff802'cfc3a287 : 00000000'00000000
00000000'00000000 00000000'00000000 00000000'00000000 :
nt!KiCommitThreadWait+0x14f
    fffffb881'3ecf7730 fffff80a'4c941fce : fffff80a'4c944018
fffff802'00000006 00000000'00000000 00000000'00000000 :
nt!KeWaitForSingleObject+0x377
    fffffb881'3ecf77e0 fffff802'd0067430 : fffff8c0b'88d2b550
00000000'00000001 00000000'00000001 00000000'00000000 :
myfault+0x1fce
```

```
fffffb881'3ecf7820 fffff802'd0066314 : ffff8c0b'00000000
ffff8c0b'88d2b504 00000000'00000000 fffffb881'3ecf7b80 :
nt!IopSynchronousSer
viceTail+0x1a0
fffffb881'3ecf78e0 fffff802'd0065c96 : 00000000'00000000
00000000'00000000 00000000'00000000 00000000'00000000 :
nt!IopXxxControlFile+0x674
fffffb881'3ecf7a20 fffff802'cfd57f93 : ffff8c0b'85377300
fffff802'cfcb9640 00000000'00000000 fffff802'd005b32f :
nt!NtDeviceIoControlFile+0x56
fffffb881'3ecf7a90 00007ffd'c1564f34 : 00000000'00000000
00000000'00000000 00000000'00000000 00000000'00000000 :
nt!KiSystemServiceCopyEnd+0x13 (TrapFrame @ fffffb881'3ecf7b00)
```

- From the stack trace, you can see that the thread that initiated the I/O is now waiting for cancellation or completion. The next step is to use the same debugger extension command used in the previous experiments, `!irp`, and attempt to analyze the problem. Copy the IRP pointer, and examine it with `!irp`:

[Click here to view code image](#)

```
lkd> !irp ffff8c0b84f1d130
Irp is active with 1 stacks 1 is current (= 0xffff8c0b84f1d200)
No Mdl: No System Buffer: Thread ffff8c0b85377300: Irp stack
trace.
      cmd  flg  cl Device      File      Completion-Context
>[IRP_MJ_DEVICE_CONTROL(e), N/A(0)]
      5    0 ffff8c0b886b5590 ffff8c0b88d2b550 00000000-
00000000
      \Driver\MYFAULT
      Args: 00000000 00000000 83360020 00000000
```

- From this output, it is obvious who the culprit driver is: `\Driver\MYFAULT`, or `Myfault.sys`. The name of the driver highlights the fact that the only way this situation can occur is through a driver problem—not a buggy application. Unfortunately, although you now know which driver caused the problem, there isn't much you can do about it apart from rebooting the system. This is necessary because Windows can never safely assume it is OK to ignore the fact that cancellation hasn't yet occurred. The IRP could return at any time and cause corruption of system memory.



Tip

If you encounter this situation in practice, you should check for a newer version of the driver, which might include a fix for the bug.

I/O completion ports

Writing a high-performance server application requires implementing an efficient threading model. Having either too few or too many server threads to process client requests can lead to performance problems. For example, if a server creates a single thread to handle all requests, clients can become starved because the server will be tied up processing one request at a time. A single thread could simultaneously process multiple requests, switching from one to another as I/O operations are started. However, this architecture introduces significant complexity and can't take advantage of systems with more than one logical processor. At the other extreme, a server could create a big pool of threads so that virtually every client request is processed by a dedicated thread. This scenario usually leads to thread-thrashing, in which lots of threads wake up, perform some CPU processing, block while waiting for I/O, and then, after request processing is completed, block again waiting for a new request. If nothing else, having too many threads results in excessive context switching, caused by the scheduler having to divide processor time among multiple active threads; such a scheme will not scale.

The goal of a server is to incur as few context switches as possible by having its threads avoid unnecessary blocking, while at the same time maximizing parallelism by using multiple threads. The ideal is for there to be a thread actively servicing a client request on every processor and for those threads not to block when they complete a request if additional requests are waiting. For this optimal process to work correctly, however, the application must have a way to activate another thread when a thread processing a client request blocks I/O (such as when it reads from a file as part of the processing).

The `IoCompletion` object

Applications use the `IoCompletion` executive object, which is exported to the Windows API as a completion port, as the focal point for the completion of I/O associated with multiple file handles. Once a file is associated with a completion port, any asynchronous I/O operations that complete on the file result in a completion packet being queued to the completion port. A thread can wait for any outstanding I/Os to complete on multiple files simply by waiting for a completion packet to be queued to the completion port. The Windows API provides similar functionality with the `WaitForMultipleObjects` API function, but completion ports have one important advantage: concurrency. *Concurrency* refers to the number of threads that an application has actively servicing client requests, which is controlled with the aid of the system.

When an application creates a completion port, it specifies a concurrency value. This value indicates the maximum number of threads associated with the port that should be running at any given time. As stated earlier, the ideal is to have one thread active at any given time for every processor in the system. Windows uses the concurrency value associated with a port to control how many threads an application has active. If the number of active threads associated with a port equals the concurrency value, a thread that is waiting on the completion port won't be allowed to run. Instead, an active thread will finish processing its current request, after which it will check whether another packet is waiting at the port. If one is, the thread simply grabs the packet and goes off to process it. When this happens, there is no context switch, and the CPUs are utilized nearly to their full capacity.

Using completion ports

Figure 6-24 shows a high-level illustration of completion-port operation. A completion port is created with a call to the `CreateIoCompletionPort` Windows API function. Threads that block on a completion port become associated with the port and are awakened in last in, first out (LIFO) order so that the thread that blocked most recently is the one that is given the next packet. Threads that block for long periods of time can have their stacks swapped out to disk, so if there are more threads associated with a port than there is work to process, the in-memory footprints of threads blocked the longest are minimized.

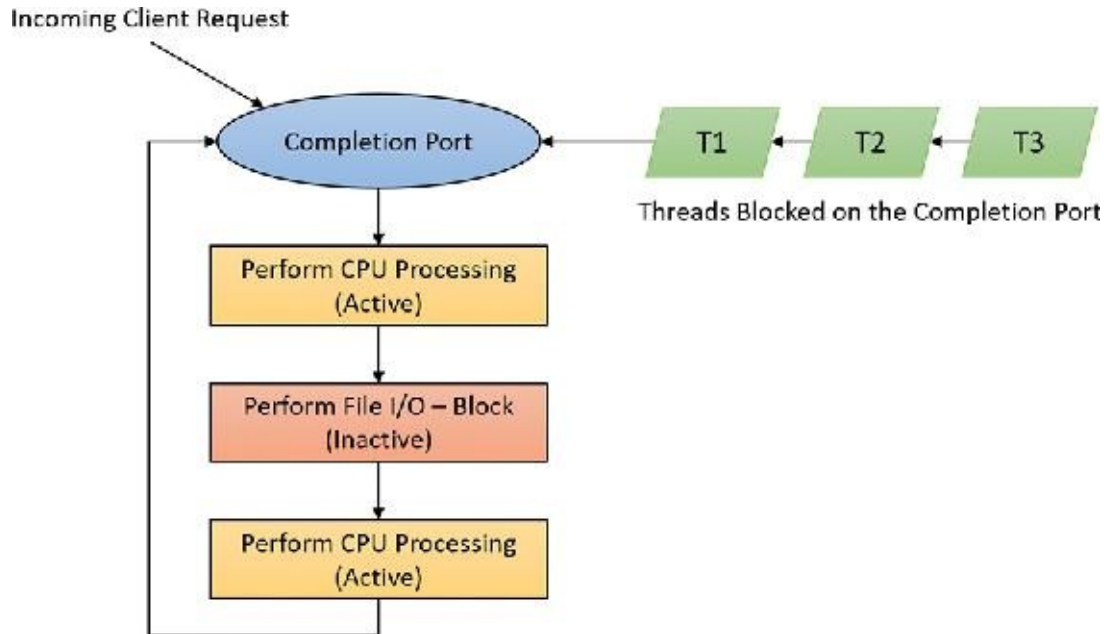


FIGURE 6-24 I/O completion-port operation.

A server application will usually receive client requests via network endpoints that are identified by file handles. Examples include Windows Sockets 2 (Winsock2) sockets or named pipes. As the server creates its communications endpoints, it associates them with a completion port and its threads wait for incoming requests by calling `GetQueuedCompletionStatus(Ex)` on the port. When a thread is given a packet from the completion port, it will go off and start processing the request, becoming an active thread. A thread will block many times during its processing, such as when it needs to read or write data to a file on disk or when it synchronizes with other threads. Windows detects this activity and recognizes that the completion port has one less active thread. Therefore, when a thread becomes inactive because it blocks, a thread waiting on the completion port will be awakened if there is a packet in the queue.

Microsoft's guidelines are to set the concurrency value roughly equal to the number of processors in a system. Keep in mind that it's possible for the number of active threads for a completion port to exceed the concurrency limit. Consider a case in which the limit is specified as 1:

1. A client request comes in and a thread is dispatched to process the request, becoming active.
2. A second request arrives, but a second thread waiting on the port isn't allowed to proceed because the concurrency limit has been reached.
3. The first thread blocks, waiting for a file I/O, so it becomes inactive.
4. The second thread is released.
5. While the second thread is still active, the first thread's file I/O is completed, making it active again. At that point—and until one of the threads blocks—the concurrency value is 2, which is

higher than the limit of 1. Most of the time, the count of active threads will remain at or just above the concurrency limit.

The completion port API also makes it possible for a server application to queue privately defined completion packets to a completion port by using the `PostQueuedCompletionStatus` function. A server typically uses this function to inform its threads of external events, such as the need to shut down gracefully.

Applications can use thread-agnostic I/O, described earlier, with I/O completion ports to avoid associating threads with their own I/Os and associating them with a completion port object instead. In addition to the other scalability benefits of I/O completion ports, their use can minimize context switches. Standard I/O completions must be executed by the thread that initiated the I/O, but when an I/O associated with an I/O completion port completes, the I/O manager uses any waiting thread to perform the completion operation.

I/O completion port operation

Windows applications create completion ports by calling the `CreateIoCompletionPort` Windows API and specifying a `NULL` completion port handle. This results in the execution of the `NtCreateIoCompletion` system service. The executive's `IoCompletion` object contains a kernel synchronization object called a *kernel queue*. Thus, the system service creates a completion port object and initializes a queue object in the port's allocated memory. (A pointer to the port also points to the queue object because the queue is the first member of the completion port.) A kernel queue object has a concurrency value that is specified when a thread initializes it, and in this case the value that is used is the one that was passed to `CreateIoCompletionPort`. `KeInitializeQueue` is the function that `NtCreateIoCompletion` calls to initialize a port's queue object.

When an application calls `CreateIoCompletionPort` to associate a file handle with a port, the `NtSetInformationFile` system service is executed with the file handle as the primary parameter. The information class that is set is `FileCompletionInformation`, and the completion port's handle and the `CompletionKey` parameter from `CreateIoCompletionPort` are the data values. `NtSetInformationFile` dereferences the file handle to obtain the file object and allocates a completion context data structure.

Finally, `NtSetInformationFile` sets the `CompletionContext` field in the file object to point at the context structure. When an asynchronous I/O operation completes on a file object, the I/O manager checks whether the `CompletionContext` field in the file object is non-`NULL`. If it is, the I/O manager allocates a completion packet and queues it to the completion port by calling `KeInsertQueue` with the port as the queue on which to insert the packet (this works because the completion port object and queue object have the same address).

When a server thread invokes `GetQueuedCompletionStatus`, the `NtRemoveIoCompletion` system service is executed. After validating parameters and translating the completion port handle to a pointer to the port, `NtRemoveIoCompletion` calls `IoRemoveIoCompletion`, which eventually calls `KeRemoveQueueEx`. For high-performance scenarios, it's possible that multiple I/Os may have been completed, and although the thread will not block, it will still call into the kernel each time to get one item. The `GetQueuedCompletionStatus` or `GetQueuedCompletionStatusEx` API allows applications to retrieve more than one I/O completion status at the same time, reducing the number of user-to--kernel roundtrips and maintaining peak efficiency. Internally, this is implemented through the `NtRemoveIoCompletionEx` function. This calls `IoRemoveIoCompletion` with a count of queued items, which is passed on to

KeRemoveQueueEx.

As you can see, `KeRemoveQueueEx` and `KeInsertQueue` are the engine behind completion ports. They are the functions that determine whether a thread waiting for an I/O completion packet should be activated. Internally, a queue object maintains a count of the current number of active threads and the maximum number of active threads. If the current number equals or exceeds the maximum when a thread calls `KeRemoveQueueEx`, the thread will be put (in LIFO order) onto a list of threads waiting for a turn to process a completion packet. The list of threads hangs off the queue object. A thread's control block data structure (KTHREAD) has a pointer in it that references the queue object of a queue that it's associated with; if the pointer is NULL, the thread isn't associated with a queue.

Windows keeps track of threads that become inactive because they block on something other than the completion port by relying on the queue pointer in a thread's control block. The scheduler routines that possibly result in a thread blocking (such as `KeWaitForSingleObject`, `KeDelayExecutionThread`, and so on) check the thread's queue pointer. If the pointer isn't NULL, the functions call `KiActivate-WaiterQueue`, a queue-related function that decrements the count of active threads associated with the queue. If the resulting number is less than the maximum and at least one completion packet is in the queue, the thread at the front of the queue's thread list is awakened and given the oldest packet. Conversely, whenever a thread that is associated with a queue wakes up after blocking, the scheduler executes the `KiUnwaitThread` function, which increments the queue's active count.

The `PostQueuedCompletionStatus` Windows API function results in the execution of the `NtSet-IoCompletion` system service. This function simply inserts the specified packet onto the completion port's queue by using `KeInsertQueue`.

[Figure 6-25](#) shows an example of a completion port object in operation. Even though two threads are ready to process completion packets, the concurrency value of 1 allows only one thread associated with the completion port to be active, and so the two threads are blocked on the completion port.

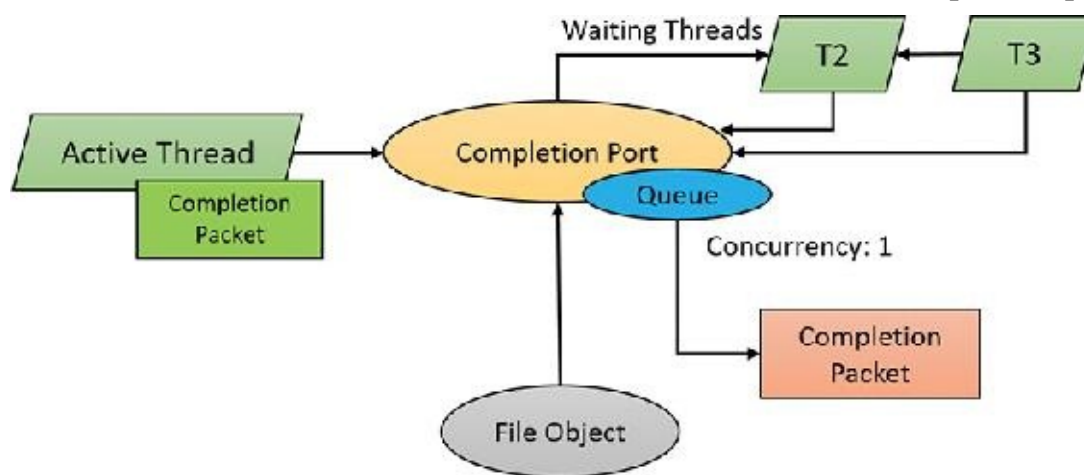


FIGURE 6-25 I/O completion port object in operation.

You can fine-tune the exact notification model of the I/O completion port through the `SetFile-CompletionNotificationModes` API, which allows application developers to take advantage of additional, specific improvements that usually require code changes but can offer even more throughput. Three notification-mode optimizations are supported, which are listed in [Table 6-4](#). Note that these modes are per file handle and cannot be changed after being set.

Notification Mode	Meaning
Skip completion port on success (FILE_SKIP_COMPLETION_PORT_ON_SUCCESS=1)	If the following three conditions are true, the I/O manager does not queue a completion entry to the port when it would ordinarily do so. First, a completion port must be associated with the file handle. Second, the file must be opened for asynchronous I/O. Third, the request must return success immediately without returning ERROR_PENDING.
Skip set event on handle (FILE_SKIP_SET_EVENT_ON_HANDLE=2)	The I/O manager does not set the event for the file object if a request returns with a success code or the error returned is ERROR_PENDING and the function that is called is not a synchronous function. If an explicit event is provided for the request, it is still signaled.
Skip set user event on fast I/O (FILE_SKIP_SET_USER_EVENT_ON_FAST_IO=4)	The I/O manager does not set the explicit event provided for the request if a request takes the fast I/O path and returns with a success code or the error returned is ERROR_PENDING and the function that is called is not a synchronous function.

TABLE 6-4 I/O completion port notification modes

I/O prioritization

Without I/O priority, background activities like search indexing, virus scanning, and disk defragmenting can severely impact the responsiveness of foreground operations. For example, a user who launches an application or opens a document while another process is performing disk I/O will experience delays as the foreground task waits for disk access. The same interference also affects the streaming playback of multimedia content like music from a disk.

Windows includes two types of I/O prioritization to help foreground I/O operations get preference: priority on individual I/O operations and I/O bandwidth reservations.

I/O priorities

The Windows I/O manager internally includes support for five I/O priorities, as shown in [Table 6-5](#), but only three of the priorities are used. (Future versions of Windows may support High and Low.)

I/O Priority	Usage
Critical	Memory manager
High	Not used
Normal	Normal application I/O
Low	Not used
Very Low	Scheduled tasks, SuperFetch, defragmenting, content indexing, background activities

TABLE 6-5 I/O priorities

I/O has a default priority of Normal, and the memory manager uses Critical when it wants to write dirty memory data out to disk under low-memory situations to make room in RAM for other data and code. The Windows Task Scheduler sets the I/O priority for tasks that have the default task priority to Very Low. The priority specified by applications that perform background processing is Very Low. All the Windows background operations, including Windows Defender scanning and desktop search indexing, use Very Low I/O priority.

Prioritization strategies

Internally, the five I/O priorities are divided into two I/O prioritization modes, called *strategies*. These are the hierarchy prioritization and the idle prioritization strategies. Hierarchy prioritization deals with all the I/O priorities except Very Low. It implements the following strategy:

- All critical-priority I/O must be processed before any high-priority I/O.

- All high-priority I/O must be processed before any normal-priority I/O.
- All normal-priority I/O must be processed before any low-priority I/O.
- All low-priority I/O is processed after any higher-priority I/O.

As each application generates I/Os, IRPs are put on different I/O queues based on their priority, and the hierarchy strategy decides the ordering of the operations.

The idle prioritization strategy, on the other hand, uses a separate queue for non-idle priority I/O. Because the system processes all hierarchy prioritized I/O before idle I/O, it's possible for the I/Os in this queue to be starved, as long as there's even a single non-idle I/O on the system in the hierarchy priority strategy queue.

To avoid this situation, as well as to control back-off (the sending rate of I/O transfers), the idle strategy uses a timer to monitor the queue and guarantee that at least one I/O is processed per unit of time (typically, half a second). Data written using non-idle I/O priority also causes the cache manager to write modifications to disk immediately instead of doing it later and to bypass its read-ahead logic for read operations that would otherwise preemptively read from the file being accessed. The prioritization strategy also waits for 50 milliseconds after the completion of the last non-idle I/O in order to issue the next idle I/O. Otherwise, idle I/Os would occur in the middle of non-idle streams, causing costly seeks.

Combining these strategies into a virtual global I/O queue for demonstration purposes, a snapshot of this queue might look similar to [Figure 6-26](#). Note that within each queue, the ordering is first-in, first-out (FIFO). The order in the figure is shown only as an example.

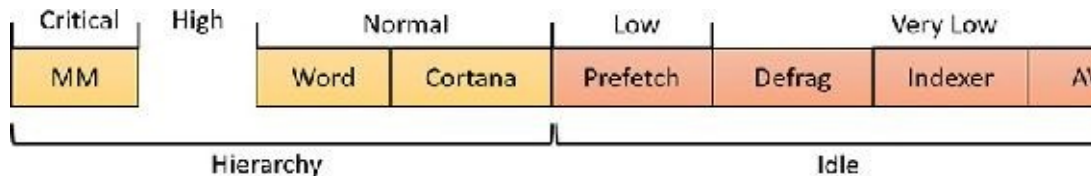


FIGURE 6-26 Sample entries in a global I/O queue.

User-mode applications can set I/O priority on three different objects. The functions `SetPriorityClass` (with the `PROCESS_MODE_BACKGROUND_BEGIN` value) and `SetThreadPriority` (with the `THREAD_MODE_BACKGROUND_BEGIN` value), set the priority for all the I/Os that are generated by either the entire process or specific threads (the priority is stored in the IRP of each request). These functions work only on the current process or thread and lower the I/O priority to Very Low. In addition, these also lower the scheduling priority to 4 and the memory priority to 1. The function `SetFileInformationByHandle` can set the priority for a specific file object (the priority is stored in the file object). Drivers can also set I/O priority directly on an IRP by using the `IoSetIoPriorityHint` API.



Note

The I/O priority field in the IRP and/or file object is a *hint*. There is no guarantee that the I/O priority will be respected or even supported by the different drivers that are part of the storage stack.

The two prioritization strategies are implemented by two different types of drivers. The hierarchy strategy is implemented by the storage *port* drivers, which are responsible for all I/Os on a specific port, such as ATA, SCSI, or USB. Only the ATA port driver (`Ataport.sys`) and USB port driver (`Usbstor.sys`)

implement this strategy, while the SCSI and storage port drivers (Scsiport.sys and Storport.sys) do not.



Note

All port drivers check specifically for Critical priority I/Os and move them ahead of their queues, even if they do not support the full hierarchy mechanism. This mechanism is in place to support critical memory manager paging I/Os to ensure system reliability.

This means that consumer mass storage devices such as IDE or SATA hard drives and USB flash disks will take advantage of I/O prioritization, while devices based on SCSI, Fibre Channel, and iSCSI will not.

On the other hand, it is the system storage class device driver (Classpn.sys) that enforces the idle strategy, so it automatically applies to I/Os directed at all storage devices, including SCSI drives. This separation ensures that idle I/Os will be subject to back-off algorithms to ensure a reliable system during operation under high idle I/O usage and so that applications that use them can make forward progress. Placing support for this strategy in the Microsoft-provided class driver avoids performance problems that would have been caused by lack of support for it in legacy third-party port drivers.

[Figure 6-27](#) displays a simplified view of the storage stack that shows where each strategy is implemented. See Chapter 12 in Part 2 for more information on the storage stack.

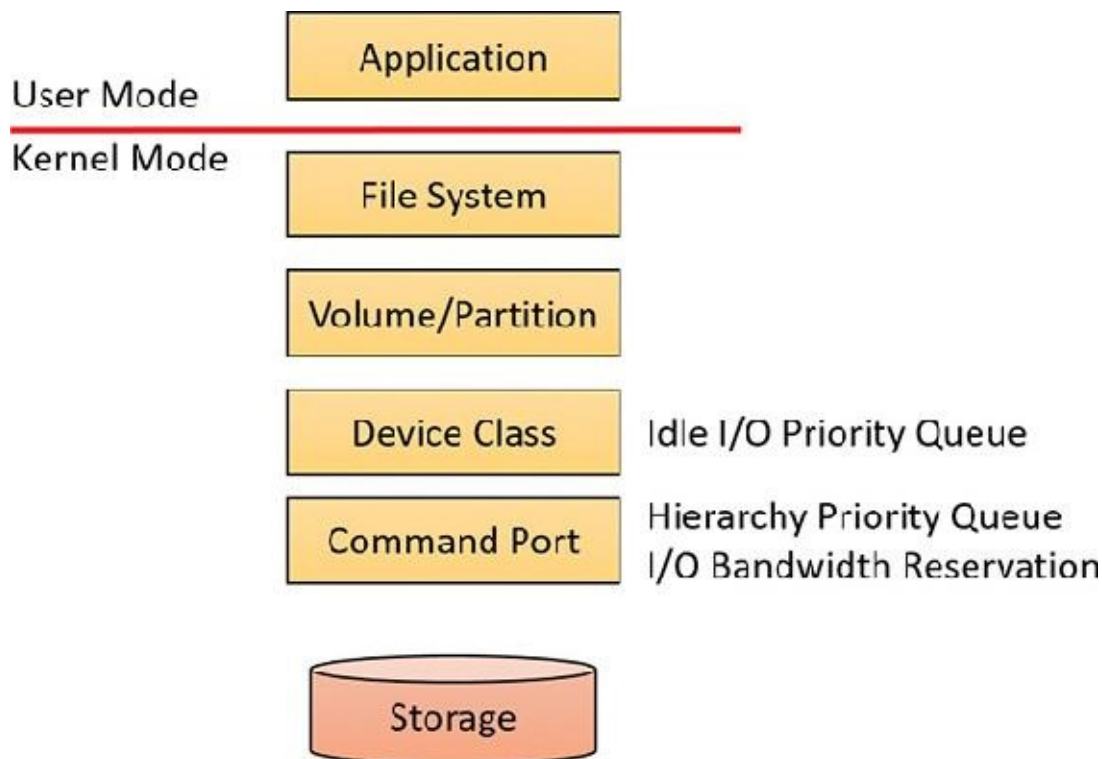


FIGURE 6-27 Implementation of I/O prioritization across the storage stack.

I/O priority inversion avoidance

To avoid I/O priority inversion, in which a high I/O priority thread is starved by a low I/O priority thread, the executive resource (ERESOURCE) locking functionality uses several strategies. The ERESOURCE was picked for the implementation of I/O priority inheritance specifically because of its heavy use in file system and storage drivers, where most I/O priority inversion issues can appear. (See Chapter 8 in Part 2 for more on executive resources.)

If an ERESOURCE is being acquired by a thread with low I/O priority, and there are currently waiters on the ERESOURCE with normal or higher priority, the current thread is temporarily boosted to normal I/O priority by using the PsBoostThreadIo API, which increments the IoBoostCount in the ETHREAD structure. It also notifies Autoboot if the thread I/O priority was boosted or the boost was removed. (Refer to [Chapter 4](#) for more on Autoboot.)

It then calls the IoBoostThreadIoPriority API, which enumerates all the IRPs queued to the target thread (recall that each thread has a list of pending IRPs) and checks which ones have a lower priority than the target priority (normal in this case), thus identifying pending idle I/O priority IRPs. In turn, the device object responsible for each of those IRPs is identified, and the I/O manager checks whether a priority callback has been registered, which driver developers can do through the IoRegisterPriority-Callback API and by setting the DO_PRIORITY_CALLBACK_ENABLED flag on their device object. Depending on whether the IRP was a paging I/O, this mechanism is called *threaded boost* or *paging boost*. Finally, if no matching IRPs were found, but the thread has at least some pending IRPs, all are boosted regardless of device object or priority, which is called *blanket boosting*.

I/O priority boosts and bumps

Windows uses a few other subtle modifications to normal I/O paths to avoid starvation, inversion, or otherwise unwanted scenarios when I/O priority is being used. Typically, these modifications are done by boosting I/O priority when needed. The following scenarios exhibit this behavior:

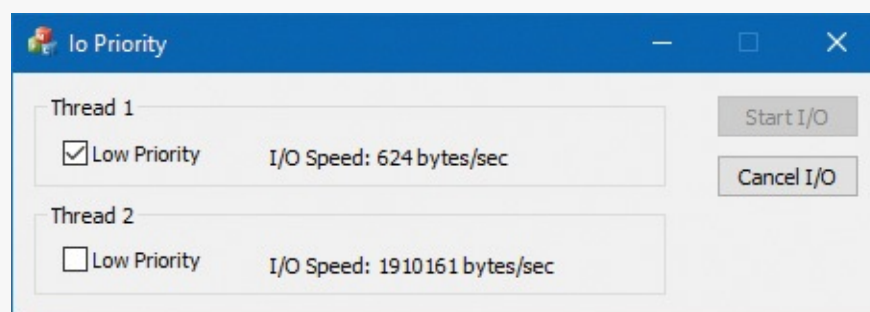
- When a driver is being called with an IRP targeted to a particular file object, Windows makes sure that if the request comes from kernel mode, the IRP uses normal priority even if the file object has a lower I/O priority hint. This is called a *kernel bump*.
- When reads or writes to the paging file are occurring (through IoPageRead and IoPageWrite), Windows checks whether the request comes from kernel mode and is not being performed on behalf of Superfetch (which always uses idle I/O). In this case, the IRP uses normal priority even if the current thread has a lower I/O priority. This is called a *paging bump*.

The following experiment will show you an example of Very Low I/O priority and how you can use Process Monitor to look at I/O priorities on different requests.

EXPERIMENT: Very low versus normal I/O throughput

You can use the IO Priority sample application (included in this book's utilities) to look at the throughput difference between two threads with different I/O priorities. Follow these steps:

1. Launch **IoPriority.exe**.
2. In the dialog box, under Thread 1, check the **Low Priority** check box.
3. Click the **Start I/O** button. You should notice a significant difference in speed between the two threads, as shown in the following screenshot:





Note

If both threads run at low priority and the system is relatively idle, their throughput will be roughly equal to the throughput of a single normal I/O priority in the example. This is because low-priority I/Os are not artificially throttled or otherwise hindered if there isn't any competition from higher-priority I/O.

4. Open the process in Process Explorer and look at the low I/O priority thread to see the priorities:

TID	CPU	Cycles Delta	Start Address
9880			ntdll.dll!TppWorkerThread
10372			ntdll.dll!TppWorkerThread
9012			ntdll.dll!TppWorkerThread
8164	4.71	466,549,279	loPriority.exe!lwWinMainCRTStartup
11048	9.73	963,727,442	loPriority.exe!CioPriorityDlg::IoThread
4464	< 0.01	154,972	loPriority.exe!CioPriorityDlg::IoThread

Thread ID: 4464

Start Time: 04:42:07 PM 05-Oct-16

State: Wait:Executive

Kernel Time: 0:00:00.000

User Time: 0:00:00.000

Context Switches: 1,009

Cycles: 30,637,564

Base Priority: 4

Dynamic Priority: 6

I/O Priority: Very Low

Memory Priority: 1

Ideal Processor: 2

5. You can also use Process Monitor to trace IO Priority's I/Os and look at their I/O priority hint. To do so, launch Process Monitor, configure a filter for IoPriority.exe, and repeat the experiment. In this application, each thread reads from a file named *_File_* concatenated with the thread ID.
6. Scroll down until you see a write to File_1. You should see output similar to the following:

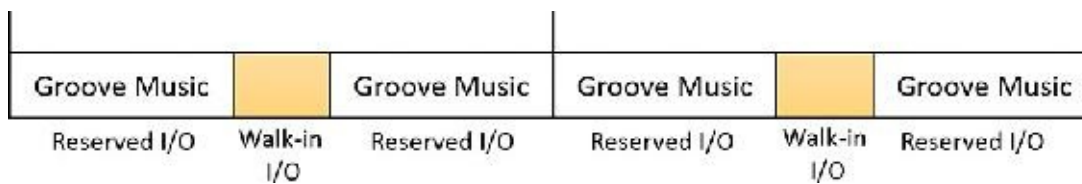


FIGURE 6-28 Effect of I/O requests during bandwidth reservation.

Like the hierarchy prioritization strategy, bandwidth reservation is implemented at the port driver level, which means it is available only for IDE, SATA, or USB-based mass-storage devices.

Container notifications

Container notifications are specific classes of events that drivers can register for through an asynchronous callback mechanism by using the `IoRegisterContainerNotification` API and selecting the notification class that interests them. Thus far, one such class is implemented in Windows: `IoSessionStateNotification`. This class allows drivers to have their registered callback invoked whenever a change in the state of a given session is registered. The following changes are supported:

- A session is created or terminated.
- A user connects to or disconnects from a session.
- A user logs on to or logs off from a session.

By specifying a device object that belongs to a specific session, the driver callback will be active only for that session. In contrast, by specifying a global device object (or no device object at all), the driver will receive notifications for all events on a system. This feature is particularly useful for devices that participate in the Plug and Play device redirection functionality that is provided through Terminal Services, which allows a remote device to be visible on the connecting host's Plug and Play manager bus as well (such as audio or printer device redirection). Once the user disconnects from a session with audio playback, for example, the device driver needs a notification in order to stop redirecting the source audio stream.

Driver Verifier

Driver Verifier is a mechanism that can be used to help find and isolate common bugs in device drivers or other kernel-mode system code. Microsoft uses Driver Verifier to check its own device drivers as well as all device drivers that vendors submit for WHQL testing. Doing so ensures that the drivers submitted are compatible with Windows and free from common driver errors. (Although not described in this book, there is also a corresponding Application Verifier tool that has resulted in quality improvements for user-mode code in Windows.)



Note

Although Driver Verifier serves primarily as a tool to help device driver developers discover bugs in their code, it is also a powerful tool for system administrators experiencing crashes. Chapter 15 in Part 2 describes its role in crash analysis troubleshooting.

Driver Verifier consists of support in several system components: the memory manager, I/O manager, and HAL all have driver verification options that can be enabled. These options are configured using the

Driver Verifier Manager (%SystemRoot%\System32\Verifier.exe). When you run Driver Verifier with no command-line arguments, it presents a wizard-style interface, as shown in [Figure 6-29](#). (You can also enable and disable Driver Verifier, as well as display current settings, by using its command-line interface. From a command prompt, type **verifier /?** to see the switches.)

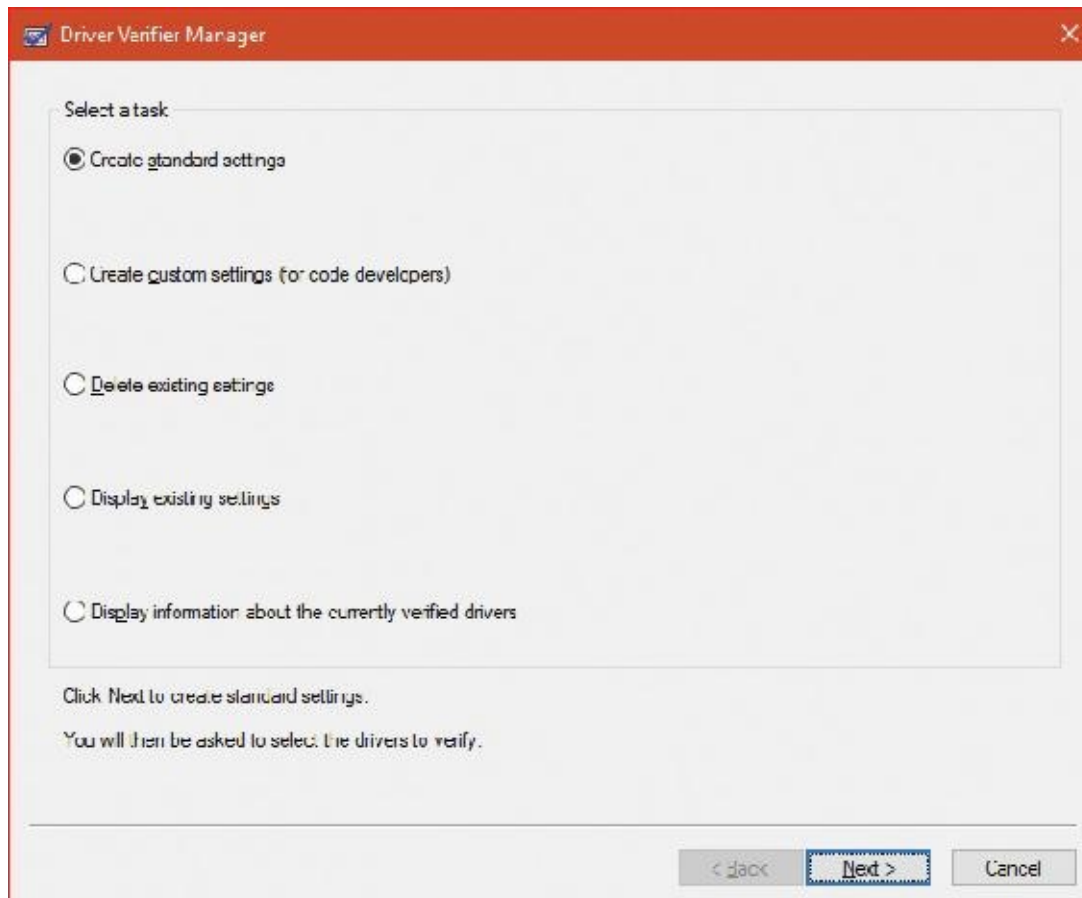


FIGURE 6-29 Driver Verifier Manager.

Driver Verifier Manager distinguishes between two sets of settings: standard and additional. This is somewhat arbitrary, but the standard settings represent the more common options that should be probably selected for every driver being tested, while the additional settings represent those settings that are less common or specific to some types of drivers. Selecting **Create Custom Settings** from the main wizard's page shows all options with a column indicating which is standard and which is additional, as shown in [Figure 6-30](#).

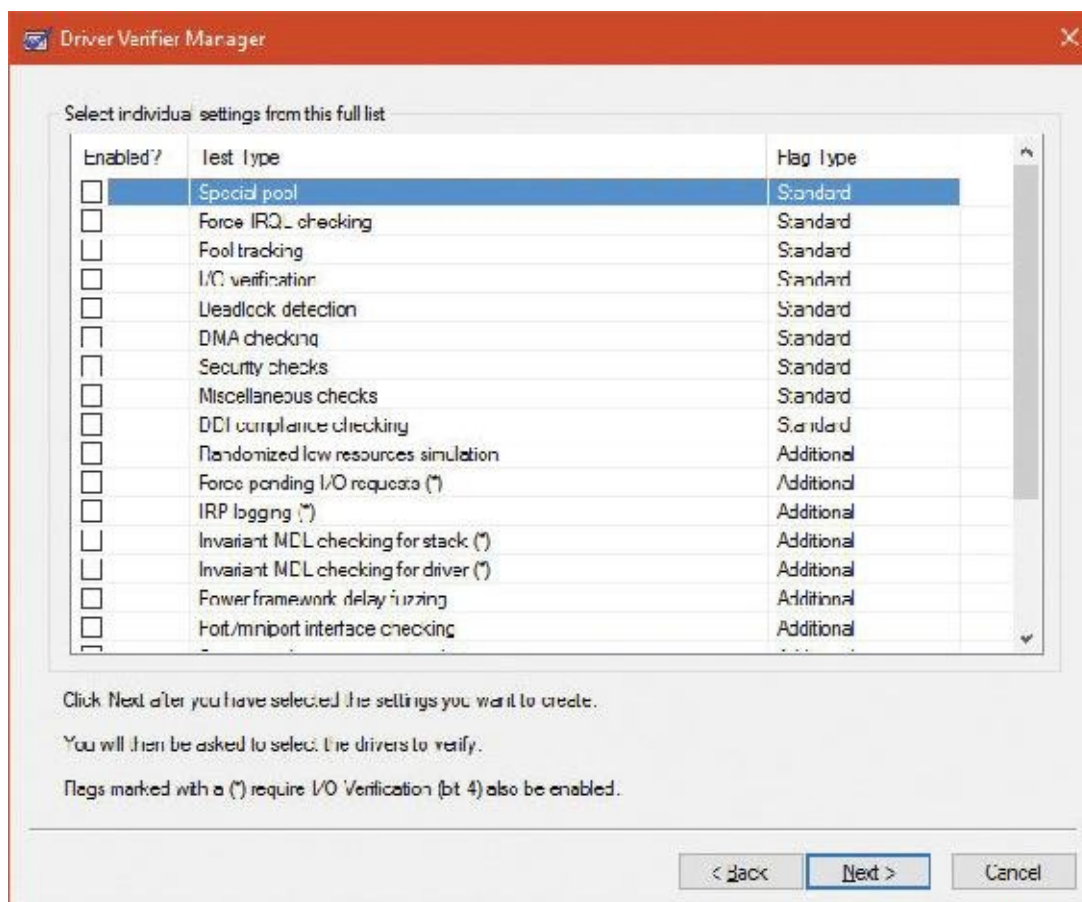


FIGURE 6-30 Driver Verifier settings.

Regardless of which options are selected, Driver Verifier always monitors drivers selected for verification, looking for a number of illegal and boundary operations, including calling kernel-memory pool functions at invalid IRQL, double-freeing memory, releasing spinlocks inappropriately, not freeing timers, referencing a freed object, delaying shutdown for longer than 20 minutes, and requesting a zero-size memory allocation.

Driver Verifier settings are stored in the registry under the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key. The `VerifyDriverLevel` value contains a bitmask that represents the verification options that are enabled. The `VerifyDrivers` value contains the names of the drivers to monitor. (These values won't exist in the registry until you select drivers to verify in the Driver Verifier Manager.) If you choose to verify all drivers (which you should never do, since this will cause considerable system slowdown), `VerifyDrivers` is set to an asterisk (*) character. Depending on the settings you have made, you might need to reboot the system for the selected verification to occur.

Early in the boot process, the memory manager reads the Driver Verifier registry values to determine which drivers to verify and which Driver Verifier options you enabled. (Note that if you boot in safe mode, any Driver Verifier settings are ignored.) Subsequently, if you've selected at least one driver for verification, the kernel checks the name of every device driver it loads into memory against the list of drivers you've selected for verification. For every device driver that appears in both places, the kernel invokes the `VfLoadDriver` function, which calls other internal `Vf*` functions to replace the driver's references to a number of kernel functions with references to Driver Verifier–equivalent versions of those functions. For example, `ExAllocatePool` is replaced with a call to `VerifierAllocatePool`. The windowing system driver (`Win32k.sys`) also makes similar changes to use Driver Verifier–equivalent functions.

I/O-related verification options

The various I/O-related verification options are as follows:

- **I/O Verification** When this option is selected, the I/O manager allocates IRPs for verified drivers from a special pool and their usage is tracked. In addition, the Driver Verifier crashes the system when an IRP is completed that contains an invalid status or when an invalid device object is passed to the I/O manager. This option also monitors all IRPs to ensure that drivers mark them correctly when completing them asynchronously, that they manage device-stack locations correctly, and that they delete device objects only once. In addition, the Verifier randomly stresses drivers by sending them fake power management and WMI IRPs, changing the order in which devices are enumerated, and adjusting the status of PnP and power IRPs when they complete to test for drivers that return incorrect status from their dispatch routines. Finally, the Verifier also detects incorrect re-initialization of remove locks while they are still being held due to pending device removal.
- **DMA Checking** DMA is a hardware-supported mechanism that allows devices to transfer data to or from physical memory without involving the CPU. The I/O manager provides several functions that drivers use to initiate and control DMA operations, and this option enables checks for the correct use of the functions and buffers that the I/O manager supplies for DMA operations.
- **Force Pending I/O Requests** For many devices, asynchronous I/Os complete immediately, so drivers may not be coded to properly handle the occasional asynchronous I/O. When this option is enabled, the I/O manager randomly returns `STATUS_PENDING` in response to a driver's calls to `IoCallDriver`, which simulates the asynchronous completion of an I/O.
- **IRP Logging** This option monitors a driver's use of IRPs and makes a record of IRP usage, which is stored as WMI information. You can then use the `Dc2wmiparser.exe` utility in the WDK to convert these WMI records to a text file. Note that only 20 IRPs for each device will be recorded—each subsequent IRP will overwrite the least recently added entry. After a reboot, this information is discarded, so `Dc2wmiparser.exe` should be run if the contents of the trace are to be analyzed later.

Memory-related verification options

The following are memory-related verification options supported by Driver Verifier. (Some are also related to I/O operations.)

Special Pool

Selecting the Special Pool option causes the pool allocation routines to bracket pool allocations with an invalid page so that references before or after the allocation will result in a kernel-mode access violation, thus crashing the system with the finger pointed at the buggy driver. Special pool also causes some additional validation checks to be performed when a driver allocates or frees memory. With special pool enabled, the pool allocation routines allocate a region of kernel memory for Driver Verifier to use. Driver Verifier redirects memory allocation requests that drivers under verification make to the special pool area rather than to the standard kernel-mode memory pools. When a device driver allocates memory from special pool, Driver Verifier rounds up the allocation to an even-page boundary. Because Driver Verifier brackets the allocated page with invalid pages, if a device driver attempts to read or write past the end of the buffer, the driver will access an invalid page, and the memory manager will raise a kernel-mode access violation.

[Figure 6-31](#) shows an example of the special pool buffer that Driver Verifier allocates to a device driver when Driver Verifier checks for overrun errors.

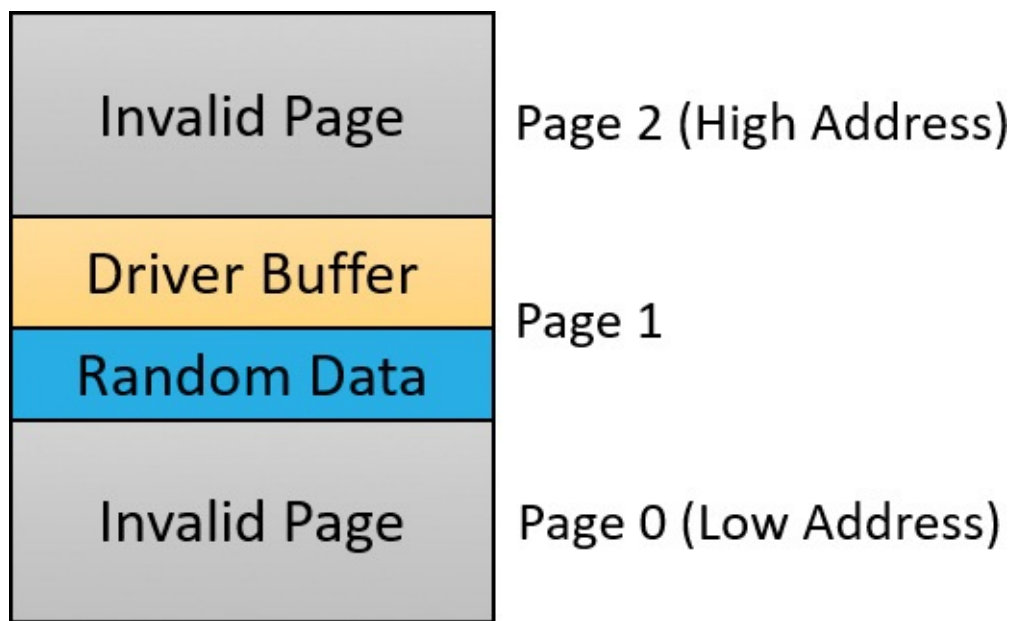


FIGURE 6-31 Layout of special pool allocations.

By default, Driver Verifier performs overrun detection. It does this by placing the buffer that the device driver uses at the end of the allocated page and filling the beginning of the page with a random pattern. Although the Driver Verifier Manager doesn't let you specify underrun detection, you can set this type of detection manually by adding the DWORD registry value `PoolTagOverruns` to the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key and setting it to 0 (or by running the `Gflags.exe` utility and selecting the **Verify Start** option in the Kernel Special Pool Tag section instead of the default option, **Verify End**). When Windows enforces underrun detection, Driver Verifier allocates the driver's buffer at the beginning of the page rather than at the end.

The overrun-detection configuration includes some measure of underrun detection as well. When the driver frees its buffer to return the memory to Driver Verifier, Driver Verifier ensures that the pattern preceding the buffer hasn't changed. If the pattern is modified, the device driver has underrun the buffer and written to memory outside the buffer.

Special pool allocations also check to ensure that the processor IRQL at the time of an allocation and deallocation is legal. This check catches an error that some device drivers make: allocating pageable memory from an IRQL at DPC/dispatch level or above.

You can also configure special pool manually by adding the DWORD registry value `PoolTag` in the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key, which represents the allocation tags the system uses for special pool. Thus, even if Driver Verifier isn't configured to verify a particular device driver, if the tag the driver associates with the memory it allocates matches what is specified in the `PoolTag` registry value, the pool allocation routines will allocate the memory from special pool. If you set the value of `PoolTag` to `0x2a` or to the wildcard (*), all memory that drivers allocate will be from special pool, provided there's enough virtual and physical memory (drivers will revert to allocating from regular pool if there aren't enough free pages).

Pool tracking

If pool tracking is enabled, the memory manager checks at driver unload time whether the driver freed all the memory allocations it made. If it didn't, it crashes the system, indicating the buggy driver. Driver Verifier also shows general pool statistics on the Driver Verifier Manager's Pool Tracking tab (accessible from the main wizard UI by selecting **Display Information About the Currently Verified Drivers** and selecting **Next** twice). You can also use the `!verifier` kernel debugger command. This command shows more information than Driver Verifier and is useful to driver writers.

Pool tracking and special pool cover not only explicit allocation calls, such as `ExAllocatePoolWithTag`, but also calls to other kernel APIs that implicitly allocate memory from pools: `IoAllocateMdl`, `IoAllocateIrp`, and other IRP allocation calls; various `Rtl` string APIs; and `IoSetCompletionRoutineEx`.

Another driver verified function enabled by the Pool Tracking option pertains to pool quota charges. The call to `ExAllocatePoolWithQuotaTag` charges the current process's pool quota for the number of bytes allocated. If such a call is made from a DPC routine, the process that is charged is unpredictable because DPC routines may execute in the context of any process. The Pool Tracking option checks for calls to this routine from the DPC routine context.

Driver Verifier can also perform locked memory page tracking, which additionally checks for pages that have been left locked after an I/O operation completes and generates a `DRIVER_LEFT_LOCKED_PAGES_IN_PROCESS` crash code instead of `PROCESS_HAS_LOCKED_PAGES`—the former indicates the driver responsible for the error as well as the function responsible for the locking of the pages.

Force IRQL Checking

One of the most common device driver bugs occurs when a driver accesses pageable data or code when the processor on which the device driver is executing is at an elevated IRQL. The memory manager can't service a page fault when the IRQL is DPC/dispatch level or above. The system often doesn't detect instances of a device driver accessing pageable data when the processor is executing at a high IRQL level because the pageable data being accessed happens to be physically resident at the time. At other times, however, the data might be paged out, which results in a system crash with the stop code `IRQL_NOT_LESS_OR_EQUAL` (that is, the IRQL wasn't less than or equal to the level required for the operation attempted—in this case, accessing pageable memory).

Although testing device drivers for this kind of bug is usually difficult, Driver Verifier makes it easy. If you select the Force IRQL Checking option, Driver Verifier forces all kernel-mode pageable code and data out of the system working set whenever a device driver under verification raises the IRQL. The internal function that does this is `MiTrimAllSystemPagableMemory`. With this setting enabled, whenever a device driver under verification accesses pageable memory when the IRQL is elevated, the system instantly detects the violation, and the resulting system crash identifies the faulty driver.

Another common driver crash that results from incorrect IRQL usage occurs when synchronization objects are part of data structures that are paged and then waited on. Synchronization objects should never be paged because the dispatcher needs to access them at an elevated IRQL, which would cause a crash. Driver Verifier checks whether any of the following structures are present in pageable memory: `KTIMER`, `KMUTEX`, `KSPIN_LOCK`, `KEVENT`, `KSEMAPHORE`, `ERESOURCE`, and `FAST_MUTEX`.

Low Resources Simulation

Enabling Low Resources Simulation causes Driver Verifier to randomly fail memory allocations that verified device drivers perform. In the past, developers wrote many device drivers under the assumption that kernel memory would always be available, and that if memory ran out, the device driver didn't have to worry about it because the system would crash anyway. However, because low-memory conditions can occur temporarily, and today's mobile devices are not as powerful as larger machines, it's important that device drivers properly handle allocation failures that indicate kernel memory is exhausted.

The driver calls that will be injected with random failures include the functions `ExAllocatePool*`, `MmProbeAndLockPages`, `MmMapLockedPagesSpecifyCache`, `MmMapIoSpace`, `MmAllocateContiguous-Memory`, `MmAllocatePagesForMdl`, `IoAllocateIrp`, `IoAllocateMdl`, `IoAllocateWorkItem`, `IoAllocateErrorLogEntry`, `IOSetCompletionRoutineEx`, and various `Rtl` string APIs that allocate from the pool. Driver Verifier also fails some allocations made by kernel GDI functions (see the WDK documentation for a complete list). Additionally, you can specify the following:

- **The probability that allocation will fail** This is 6 percent by default.
- **Which applications should be subject to the simulation** All are by default.
- **Which pool tags should be affected** All are by default.
- **What delay should be used before fault injection starts** The default is 7 minutes after the system boots, which is enough time to get past the critical initialization period in which a low-memory condition might prevent a device driver from loading.

You can change these customizations with command line options to `verifier.exe`.

After the delay period, Driver Verifier starts randomly failing allocation calls for device drivers it is verifying. If a driver doesn't correctly handle allocation failures, this will likely show up as a system crash.

Systematic Low Resources Simulation

Similar to the Low Resources Simulation option, this option fails certain calls to the kernel and *Ndis.Sys* (for network drivers), but does so in a systematic way, by examining the call stack at the point of failure injection. If the driver handles the failure correctly, that call stack will not be failure injected again. This allows the driver writer to see issues in a systematic way, fix a reported issue, and then move on to the next. Examining call stacks is a relatively expensive operation, therefore verifying more than a single driver at a time with this setting is not recommended.

Miscellaneous checks

Some of the checks that Driver Verifier calls *miscellaneous* allow it to detect the freeing of certain system structures in the pool that are still active. For example, Driver Verifier will check for:

- **Active work items in freed memory** A driver calls `ExFreePool` to free a pool block in which one or more work items queued with `IoQueueWorkItem` are present.
- **Active resources in freed memory** A driver calls `ExFreePool` before calling `ExDelete-Resource` to destroy an `ERESOURCE` object.
- **Active look-aside lists in freed memory** A driver calls `ExFreePool` before calling `ExDelete- NPagedLookasideList` or `ExDeletePagedLookasideList` to delete the look-aside list.

Finally, when verification is enabled, Driver Verifier performs certain automatic checks that cannot be

individually enabled or disabled. These include the following:

- Calling `MmProbeAndLockPages` or `MmProbeAndLockProcessPages` on an MDL having incorrect flags. For example, it is incorrect to call `MmProbeAndLockPages` for an MDL that was set up by calling `MmBuildMdlForNonPagedPool`.
- Calling `MmMapLockedPages` on an MDL having incorrect flags. For example, it is incorrect to call `MmMapLockedPages` for an MDL that is already mapped to a system address. Another example of incorrect driver behavior is calling `MmMapLockedPages` for an MDL that was not locked.
- Calling `MmUnlockPages` or `MmUnmapLockedPages` on a partial MDL (created by using `IoBuildPartialMdl`).
- Calling `MmUnmapLockedPages` on an MDL that is not mapped to a system address.
- Allocating synchronization objects such as events or mutexes from `NonPagedPoolSession` memory.

Driver Verifier is a valuable addition to the arsenal of verification and debugging tools available to device driver writers. Many device drivers that first ran with Driver Verifier had bugs that Driver Verifier was able to expose. Thus, Driver Verifier has resulted in an overall improvement in the quality of all kernel-mode code running in Windows.

The Plug and Play manager

The PnP manager is the primary component involved in supporting the ability of Windows to recognize and adapt to changing hardware configurations. A user doesn't need to understand the intricacies of hardware or manual configuration to install and remove devices. For example, it's the PnP manager that enables a running Windows laptop that is placed on a docking station to automatically detect additional devices located in the docking station and make them available to the user.

Plug and Play support requires cooperation at the hardware, device driver, and operating system levels. Industry standards for the enumeration and identification of devices attached to buses are the foundation of Windows Plug and Play support. For example, the USB standard defines the way that devices on a USB bus identify themselves. With this foundation in place, Windows Plug and Play support provides the following capabilities:

- The PnP manager automatically recognizes installed devices, a process that includes enumerating devices attached to the system during a boot and detecting the addition and removal of devices as the system executes.
- Hardware resource allocation is a role the PnP manager fills by gathering the hardware resource requirements (interrupts, I/O memory, I/O registers, or bus-specific resources) of the devices attached to a system and, in a process called *resource arbitration*, optimally assigning resources so that each device meets the requirements necessary for its operation. Because hardware devices can be added to the system after boot-time resource assignment, the PnP manager must also be able to reassign resources to accommodate the needs of dynamically added devices.
- Loading appropriate drivers is another responsibility of the PnP manager. The PnP manager determines, based on the identification of a device, whether a driver capable of managing the device is installed on the system, and if one is, it instructs the I/O manager to load it. If a suitable driver isn't installed, the kernel-mode PnP manager communicates with the user-mode PnP manager to install the device, possibly requesting the user's assistance in locating a suitable driver.

- The PnP manager also implements application and driver mechanisms for the detection of hardware configuration changes. Applications or drivers sometimes require a specific hardware device to function, so Windows includes a means for them to request notification of the presence, addition, or removal of devices.
- It provides a place for storing device state, and it participates in system setup, upgrade, migration, and offline image management.
- It supports network connected devices, such as network projectors and printers, by allowing specialized bus drivers to detect the network as a bus and create device nodes for the devices running on it.

Level of Plug and Play support

Windows aims to provide full support for Plug and Play, but the level of support possible depends on the attached devices and installed drivers. If a single device or driver doesn't support Plug and Play, the extent of Plug and Play support for the system can be compromised. In addition, a driver that doesn't support Plug and Play might prevent other devices from being usable by the system. [Table 6-6](#) shows the outcome of various combinations of devices and drivers that can and can't support Plug and Play.

Type of Device	Plug-and-Play Driver	Non-Plug and Play Driver
Plug and play	Full plug and play	No plug and play
Non-plug and play	Possible partial plug and play	No plug and play

TABLE 6-6 Device and driver plug-and-play capability

A device that isn't Plug and Play-compatible is one that doesn't support automatic detection, such as a legacy ISA sound card. Because the operating system doesn't know where the hardware physically lies, certain operations—such as laptop undocking, sleep, and hibernation—are disallowed. However, if a Plug and Play driver is manually installed for the device, the driver can at least implement PnP manager-directed resource assignment for the device.

Drivers that aren't Plug and Play-compatible include legacy drivers, such as those that ran on Windows NT 4. Although these drivers might continue to function on later versions of Windows, the PnP manager can't reconfigure the resources assigned to such devices in the event that resource reallocation is necessary to accommodate the needs of a dynamically added device. For example, a device might be able to use I/O memory ranges A and B, and during the boot, the PnP manager assigns it range A. If a device that can use only A is attached to the system later, the PnP manager can't direct the first device's driver to reconfigure itself to use range B. This prevents the second device from obtaining required resources, which results in the device being unavailable for use by the system. Legacy drivers also impair a machine's ability to sleep or hibernate. (See the section "[The power manager](#)" later in this chapter for more details.)

Device enumeration

Device enumeration occurs when the system boots, resumes from hibernation, or is explicitly instructed to do so (for example, by clicking Scan for Hardware Changes in the Device Manager UI). The PnP manager builds a device tree (described momentarily) and compares it to its known stored tree from a previous enumeration, if any. For a boot or resume from hibernation, the stored device tree is empty. Newly discovered devices and removed devices require special treatment, such as loading appropriate drivers (for a newly discovered device) and notifying drivers of a removed device.

The PnP manager begins device enumeration with a virtual bus driver called *Root*, which represents the entire computer system and acts as the bus driver for non-Plug and Play drivers and the HAL. The HAL acts as a bus driver that enumerates devices directly attached to the motherboard as well as system components such as batteries. Instead of actually enumerating, however, the HAL relies on the hardware description the Setup process recorded in the registry to detect the primary bus (in most cases, a PCI bus) and devices such as batteries and fans.

The primary bus driver enumerates the devices on its bus, possibly finding other buses, for which the PnP manager initializes drivers. Those drivers in turn can detect other devices, including other subsidiary buses. This recursive process of enumeration, driver loading (if the driver isn't already loaded), and further enumeration proceeds until all the devices on the system have been detected and configured.

As the bus drivers report detected devices to the PnP manager, the PnP manager creates an internal tree called a *device tree* that represents the relationships between devices. Nodes in the tree are called *device nodes*, or *devnodes*. A devnode contains information about the device objects that represent the device as well as other Plug and Play-related information stored in the devnode by the PnP manager. [Figure 6-32](#) shows an example of a simplified device tree. A PCI bus serves as the system's primary bus, which USB, ISA, and SCSI buses are connected to.

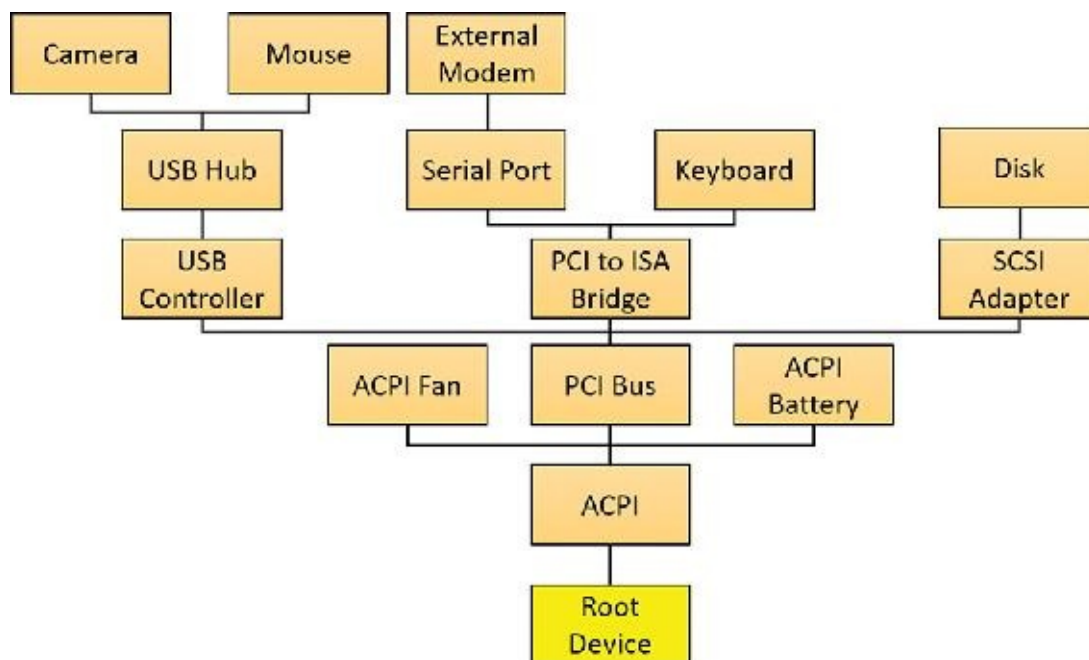


FIGURE 6-32 An example of a device tree.

The Device Manager utility, which is accessible from the Computer Management snap-in in the Programs/Administrative Tools folder of the Start menu (and also from the Device Manager link of the System utility in Control Panel), shows a simple list of devices present on a system in its default configuration. You can also select the Devices by Connection option from the Device Manager's View menu to see the devices as they relate to the device tree. [Figure 6-33](#) shows an example of the Device

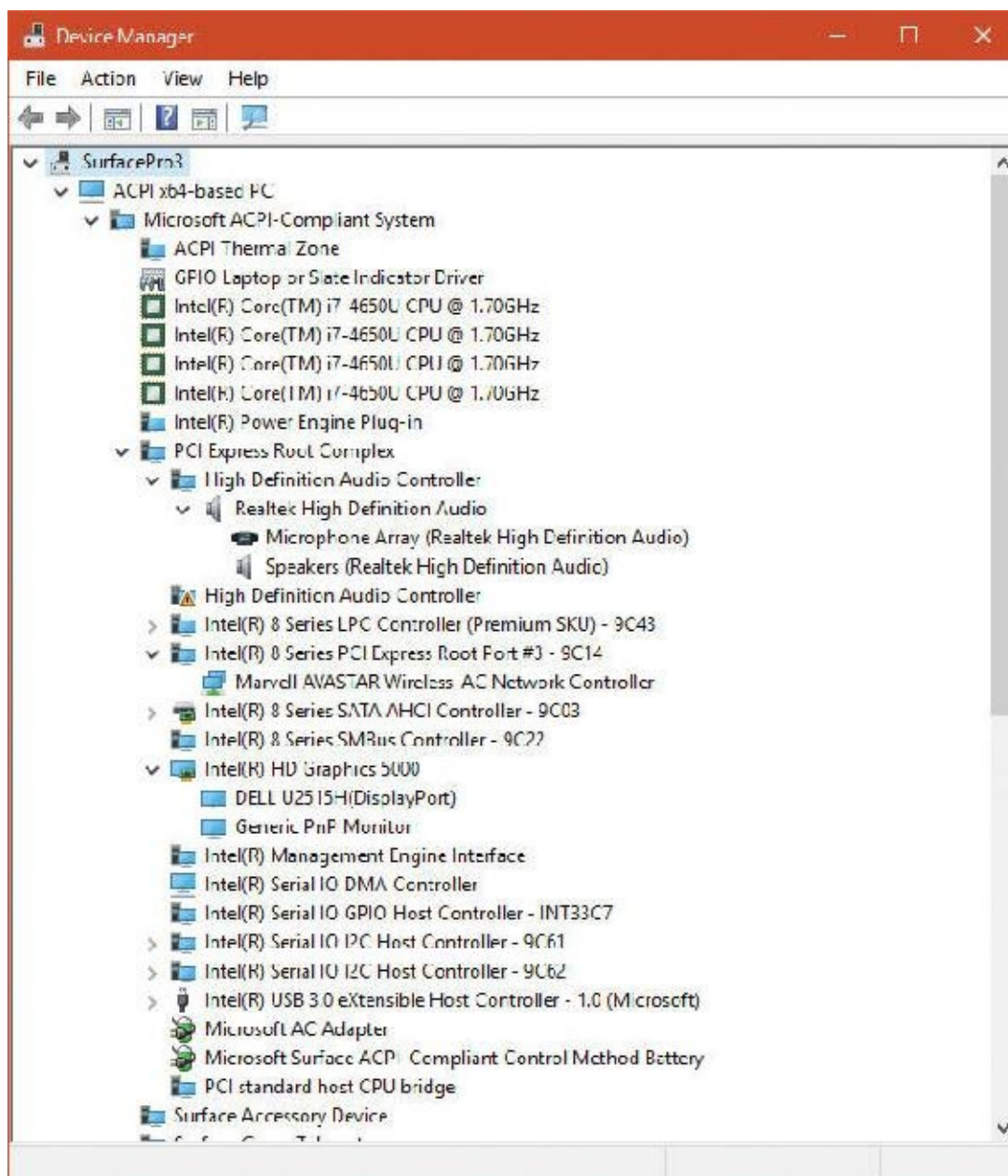


FIGURE 6-33 Device Manager, with the device tree shown.

EXPERIMENT: Dumping the device tree

A more detailed way to view the device tree than using Device Manager is to use the `!devnode` kernel debugger command. Specifying `0 1` as command options dumps the internal device tree devnode structures, indenting entries to show their hierarchical relationships, as shown here:

[Click here to view code image](#)

```
lkd> !devnode 0 1
Dumping IopRootDeviceNode (= 0x85161a98)
DevNode 0x85161a98 for PDO 0x84d10390
    InstancePath is "HTREE\ROOT\0"
    State = DeviceNodeStarted (0x308)
    Previous State = DeviceNodeEnumerateCompletion (0x30d)
    DevNode 0x8515bea8 for PDO 0x8515b030
    DevNode 0x8515c698 for PDO 0x8515c820
```

```

InstancePath is "Root\ACPI_HAL\0000"
State = DeviceNodeStarted (0x308)
Previous State = DeviceNodeEnumerateCompletion (0x30d)
DevNode 0x84d1c5b0 for PDO 0x84d1c738
    InstancePath is "ACPI_HAL\PNP0C08\0"
    ServiceName is "ACPI"
    State = DeviceNodeStarted (0x308)
    Previous State = DeviceNodeEnumerateCompletion (0x30d)
    DevNode 0x85ebf1b0 for PDO 0x85ec0210
        InstancePath is "ACPI\GenuineIntel_-
_x86_Family_6_Model_15\_0"
        ServiceName is "intelppm"
        State = DeviceNodeStarted (0x308)
        Previous State = DeviceNodeEnumerateCompletion (0x30d)
        DevNode 0x85ed6970 for PDO 0x8515e618
            InstancePath is "ACPI\GenuineIntel_-
_x86_Family_6_Model_15\_1"
            ServiceName is "intelppm"
            State = DeviceNodeStarted (0x308)
            Previous State = DeviceNodeEnumerateCompletion (0x30d)
            DevNode 0x85ed75c8 for PDO 0x85ed79e8
                InstancePath is "ACPI\ThermalZone\THM_"
                State = DeviceNodeStarted (0x308)
                Previous State = DeviceNodeEnumerateCompletion (0x30d)
                DevNode 0x85ed6cd8 for PDO 0x85ed6858
                    InstancePath is "ACPI\pnp0c14\0"
                    ServiceName is "WmiAcpi"
                    State = DeviceNodeStarted (0x308)
                    Previous State = DeviceNodeEnumerateCompletion (0x30d)
                    DevNode 0x85ed7008 for PDO 0x85ed6730
                        InstancePath is "ACPI\ACPI0003\2&daba3ff&2"
                        ServiceName is "CmBatt"
                        State = DeviceNodeStarted (0x308)
                        Previous State = DeviceNodeEnumerateCompletion (0x30d)
                        DevNode 0x85ed7e60 for PDO 0x84d2e030
                            InstancePath is "ACPI\PNP0C0A\1"
                            ServiceName is "CmBatt"

```

...

Information shown for each devnode includes the InstancePath, which is the name of the device's enumeration registry key stored under HKLM\SYSTEM\CurrentControlSet\Enum, and the ServiceName, which corresponds to the device's driver registry key under HKLM\SYSTEM\CurrentControlSet\Services. To see the resources assigned to each devnode, such as interrupts, ports, and memory, specify 0 3 as the command options for the !devnode command.

Device stacks

As devnodes are created by the PnP manager, driver objects and device objects are created to manage and logically represent the linkage between the devices that make up the devnode. This linkage is called a *device stack* (briefly discussed in the “[IRP flow](#)” section earlier in this chapter). You can think of the device stack as an ordered list of device object/driver pairs. Each device stack is built from the bottom to the top. [Figure 6-34](#) shows an example of a devnode (a reprint of [Figure 6-6](#)), with seven device objects (all managing the same physical device). Each devnode contains at least two devices (PDO and FDO), but can contain more device objects. A device stack consists of the following:

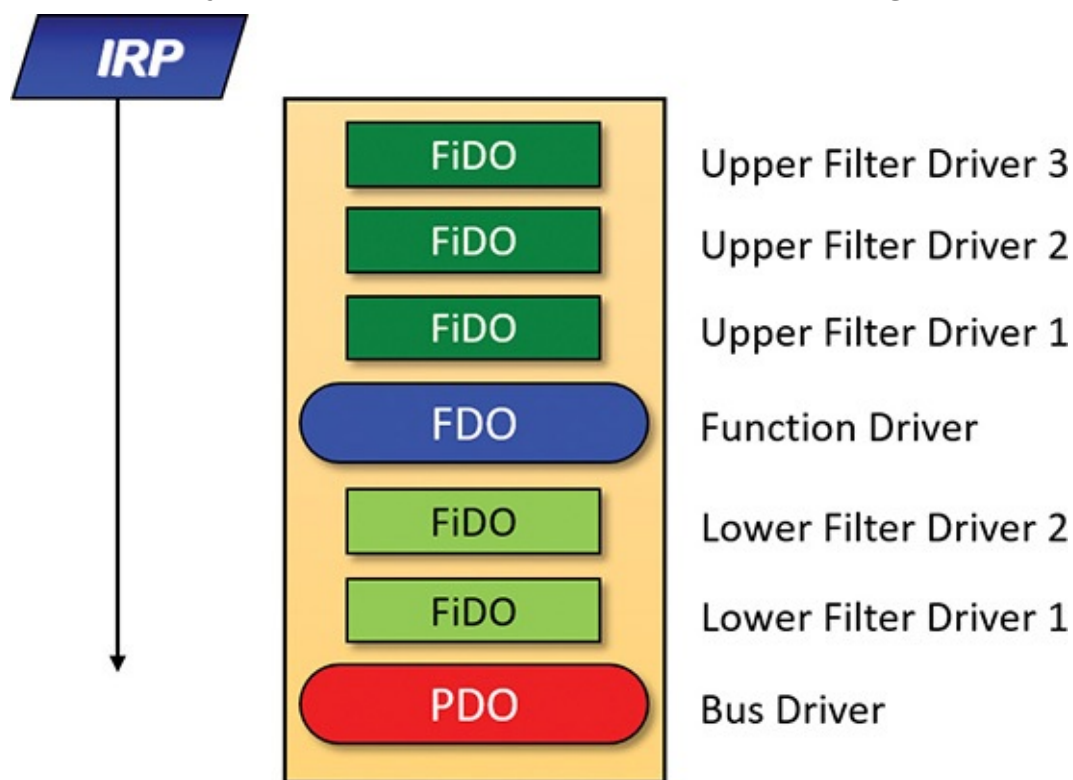


FIGURE 6-34 Devnode (device stack).

- A physical device object (PDO) that the PnP manager instructs a bus driver to create when the bus driver reports the presence of a device on its bus during enumeration. The PDO represents the physical interface to the device and is always at the bottom of the device stack.
- One or more optional filter device objects (FiDOs) that layer between the PDO and the functional device object (FDO; described in the next bullet), called *lower filters* (the term “lower” is always considered in relation to the FDO). These may be used for intercepting IRPs coming out of the FDO and towards the bus driver (which may be of interest to bus filters).
- One (and only one) functional device object (FDO) that is created by the driver, which is called a *function driver*, that the PnP manager loads to manage a detected device. An FDO represents the logical interface to a device, having the most “intimate” knowledge of the functionality provided by the device. A function driver can also act as a bus driver if devices are attached to the device represented by the FDO. The function driver often creates an interface (essentially a name) to the FDO’s corresponding PDO so that applications and other drivers can open the device and interact with it. Sometimes function drivers are divided into a separate class/port driver and miniport driver that work together to manage I/O for the FDO.
- One or more optional FiDOs that layer above the FDO, called *upper filters*. These get first crack at an IRP header for the FDO.



Note

The various device objects have different names in [Figure 6-34](#) to make them easier to describe. However, they are all instances of `DEVICE_OBJECT` structures.

Device stacks are built from the bottom up and rely on the I/O manager's layering functionality, so IRPs flow from the top of a device stack toward the bottom. However, any level in the device stack can choose to complete an IRP, as described in the "[IRP flow](#)" section earlier in this chapter.

Device-stack driver loading

How does the PnP manager find the correct drivers as part of building the device stack? The registry has this information scattered in three important keys (and their subkeys), shown in [Table 6-7](#). (Note that CCS is short for *CurrentControlSet*.)

Registry Key	Short Name	Description
HKLM\System\CCS\Enum	Hardware key	Settings for known hardware devices
HKLM\System\CCS\Control\Class	Class key	Settings for device types
HKLM\System\CCS\Services	Software key	Settings for drivers

TABLE 6-7 Important registry keys for plug-and-play driver loading

When a bus driver performs device enumeration and discovers a new device, it first creates a PDO to represent the existence of the physical device that has been detected. Next, it informs the PnP manager by calling `IoInvalidateDeviceRelations` (documented in the WDK) with the `BusRelations` enumeration value and the PDO, indicating to the PnP manager that a change on its bus has been detected. In response, the PnP manager asks the bus driver (through an IRP) for the device identifier.

The identifiers are bus-specific; for example, a USB device identifier consists of a vendor ID (VID) for the hardware vendor that made the device and a product ID (PID) that the vendor assigned to the device. For a PCI device, a similar vendor ID is required, along with a device ID, to uniquely identify the device within a vendor (plus some optional components; see the WDK for more information on device ID formats). Together, these IDs form what Plug and Play calls a *device ID*. The PnP manager also queries the bus driver for an instance ID to help it distinguish different instances of the same hardware. The instance ID can describe either a bus-relative location (for example, the USB port) or a globally unique descriptor (for example, a serial number).

The device ID and instance ID are combined to form a device instance ID (DIID), which the PnP manager uses to locate the device's key under the Hardware key shown in [Table 6-7](#). The subkeys under that key have the form `<Enumerator>\<Device ID>\<Instance ID>`, where the enumerator is a bus driver, the device ID is a unique identifier for a type of device, and the instance ID uniquely identifies different instances of the same hardware.

[Figure 6-35](#) presents an example of an enumeration subkey of an Intel display card. The device's key contains descriptive data and includes values named `Service` and `ClassGUID` (which are obtained from a driver's INF file upon installation) that help the PnP manager locate the device's drivers as follows:

- The `Service` value is looked up in the Software key, and there the path to the driver (SYS file) is stored in the `ImagePath` value. [Figure 6-36](#) shows the Software subkey named `igfx` (from [Figure 6-35](#)) where the Intel display driver can be located. The PnP manager will load that driver (if it's

not already loaded), call its add-device routine, and there the driver will create the FDO.

- If a value named **LowerFilters** is present, it contains a multiple string list of drivers to load as lower filters, which can be located in the Software subkey. The PnP manager loads these drivers before loading the driver associated with the **Service** value above.

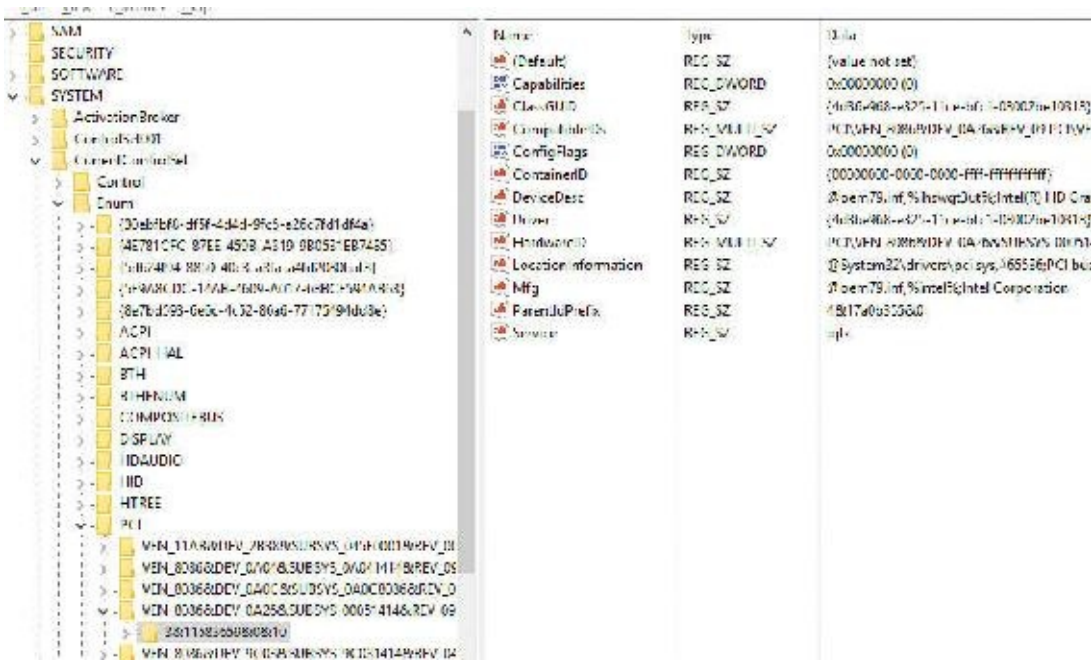


FIGURE 6-35 Example of a Hardware subkey.



FIGURE 6-36 Example of a Software subkey.

- If a value named **UpperFilters** is present, it indicates a list of driver names (under the Software key, similar to **LowerFilters**) which the PnP manager will load in much the same way after it loads the driver pointed to by the **Service** value.
- The **ClassGUID** value represents the general type of device (display, keyboard, disk, etc.), and points to a subkey under the Class key (from [Table 6-7](#)). The key represents settings applicable to all drivers for that type of device. In particular, if the values **LowerFilters** and/or **UpperFilters** are present, they are treated just like the same values in the Hardware key of the particular device. This allows, for example, the loading of an upper filter for keyboard devices, regardless of the particular keyboard or the vendor. [Figure 6-37](#) shows the class key for keyboard devices. Notice the friendly name (Keyboard), although the GUID is what matters (the decision on the particular class is provided as part of the installation INF file). An **UpperFilters** value exists, listing the system provided keyboard class driver that always loads as part of any keyboard devnode. (You can also see the **ImagePath** value that is used as the icon for the keyboard type in the Device Manager’s UI.)

	Name	Type	Data
{4d35e96f-e825-11ce-bfc1-00002be...}	(Default)	REG_SZ	(value not set)
{4d35e968-e825-11ce-bfc1-00002be...}	Class	REG_SZ	Keyboard
{4d35e96a-e825-11ce-bfc1-00002be...}	Class Desc	REG_SZ	@%SystemRoot%\System32\sysClass\UI-3002
{4d35e96b-e825-11ce-bfc1-00002be...}	IconPath	REG_MULTI_SZ	%SystemRoot%\System32\setupapi.dll,-7
{4d35e96d-e825-11ce-bfc1-00002be...}	UninstallClass	REG_SZ	1
{4d35e96e-e825-11ce-bfc1-00002be...}	UpperFilters	REG_MULTI_SZ	kbdclass

FIGURE 6-37 The keyboard class key.

To summarize, the order of driver loading for a devnode is as follows:

1. The bus driver is loaded, creating the PDO.
2. Any lower filters listed in the Hardware instance key are loaded, in the order listed (multi string), creating their filter device objects (FiDOs in [Figure 6-34](#)).
3. Any lower filters listed in the corresponding Class key are loaded in the order listed, creating their FiDOs.
4. The driver listed in the Service value is loaded, creating the FDO.
5. Any upper filters listed in the Hardware instance key are loaded, in the order listed, creating their FiDOs.
6. Any upper filters listed in the corresponding Class key are loaded in the order listed creating their FiDOs.

To deal with multifunction devices (such as all-in-one printers or cell phones with integrated camera and music player functionalities), Windows also supports a container ID property that can be associated with a devnode. The container ID is a GUID that is unique to a single instance of a physical device and shared between all the function devnodes that belong to it, as shown in [Figure 6-38](#).



FIGURE 6-38 All-in-one printer with a unique ID as seen by the PnP manager.

The container ID is a property that, similar to the instance ID, is reported back by the bus driver of the corresponding hardware. Then, when the device is being enumerated, all devnodes associated with the same PDO share the container ID. Because Windows already supports many buses out of the box—such as PnP-X, Bluetooth, and USB—most device drivers can simply return the bus-specific ID, from which Windows will generate the corresponding container ID. For other kinds of devices or buses, the driver can generate its own unique ID through software.

Finally, when device drivers do not supply a container ID, Windows can make educated guesses by querying the topology for the bus, when that's available, through mechanisms such as ACPI. By understanding whether a certain device is a child of another, and whether it is removable, hot-pluggable, or user-reachable (as opposed to an internal motherboard component), Windows is able to assign container IDs to device nodes that reflect multifunction devices correctly.

The final end-user benefit of grouping devices by container IDs is visible in the Devices and Printers UI. This feature is able to display the scanner, printer, and faxing components of an all-in-one printer as a single graphical element instead of three distinct devices. For example, in [Figure 6-39](#), the HP 6830 printer/fax/scanner is identified as a single device.

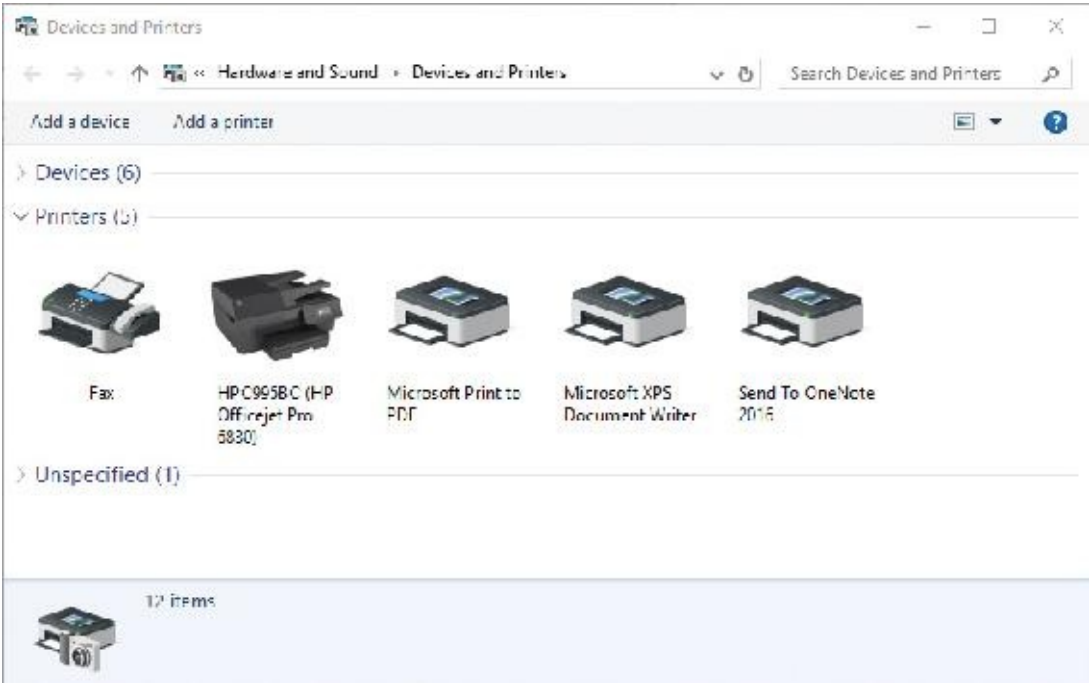
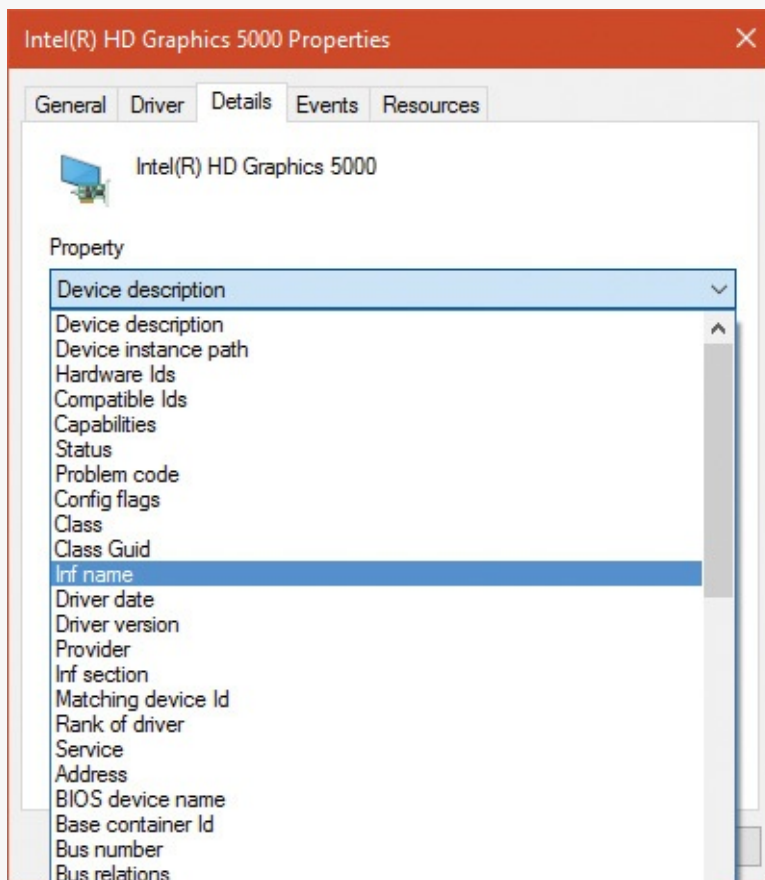


FIGURE 6-39 The Devices and Printers Control Panel applet.

EXPERIMENT: Viewing detailed devnode information in Device Manager

The Device Manager applet shows detailed information about a device node on its Details tab. The tab allows you to view an assortment of fields, including the devnode’s device instance ID, hardware ID, service name, filters, and power capabilities.

The following screen shows the selection combo box of the Details tab expanded to reveal some of the types of information you can access:



Driver support for Plug and Play

To support Plug and Play, a driver must implement a Plug and Play dispatch routine (IRP_MJ_PNP), a power-management dispatch routine (IRP_MJ_POWER, described in the section “[The power manager](#)” later in this chapter), and an add-device routine. Bus drivers must support Plug and Play requests that are different than the ones that function or filter drivers support, however. For example, when the PnP manager guides device enumeration during the system boot, it asks bus drivers for a description of the devices that they find on their respective buses through PnP IRPs.

Function and filter drivers prepare to manage their devices in their add-device routines, but they don’t actually communicate with the device hardware. Instead, they wait for the PnP manager to send a start-device command (IRP_MN_START_DEVICE minor PnP IRP code) for the device to their Plug and Play dispatch routine. Before sending the start-device command, the PnP manager performs resource arbitration to decide what resources to assign the device. The start-device command includes the resource assignment that the PnP manager determines during resource arbitration. When a driver receives a start-device command, it can configure its device to use the specified resources. If an application tries to open a device that hasn’t finished starting, it receives an error indicating that the device does not exist.

After a device has started, the PnP manager can send the driver additional Plug and Play commands, including ones related to the device’s removal from the system or to resource reassignment. For example,

when the user invokes the remove/eject device utility, shown in [Figure 6-40](#) (accessible by clicking the USB connector icon in the taskbar notification area), to tell Windows to eject a USB flash drive, the PnP manager sends a query-remove notification to any applications that have registered for Plug and Play notifications for the device. Applications typically register for notifications on their handles, which they close during a query-remove notification. If no applications veto the query-remove request, the PnP manager sends a query-remove command to the driver that owns the device being ejected (IRP_MN_QUERY_REMOVE_DEVICE). At that point, the driver has a chance to deny the removal or to ensure that any pending I/O operations involving the device have completed, and to begin rejecting further I/O requests aimed at the device. If the driver agrees to the remove request and no open handles to the device remain, the PnP manager next sends a remove command to the driver (IRP_MN_REMOVE_DEVICE) to request that the driver stop accessing the device and release any resources the driver has allocated on behalf of the device.



FIGURE 6-40 The remove/eject device utility.

When the PnP manager needs to reassign a device's resources, it first asks the driver whether it can temporarily suspend further activity on the device by sending the driver a query-stop command (IRP_MN_QUERY_STOP_DEVICE). The driver either agrees to the request (if doing so won't cause data loss or corruption) or denies the request. As with a query-remove command, if the driver agrees to the request, the driver completes pending I/O operations and won't initiate further I/O requests for the device that can't be aborted and subsequently restarted. The driver typically queues new I/O requests so that the resource reshuffling is transparent to applications currently accessing the device. The PnP manager then sends the driver a stop command (IRP_MN_STOP_DEVICE). At that point, the PnP manager can direct the driver to assign different resources to the device and once again send the driver a start-device command for the device.

The various Plug and Play commands essentially guide a device through an operational state machine, forming a well-defined state-transition table, which is shown in [Figure 6-41](#). (The state diagram reflects the state machine implemented by function drivers. Bus drivers implement a more complex state machine.) Each transition in [Figure 6-41](#) is marked by its minor IRP constant name without the IRP_MN_ prefix. One state that we haven't discussed is the one that results from the PnP manager's command (IRP_MN_SURPRISE_REMOVAL). This command results when either a user removes a device without warning, as when the user ejects a PCMCIA card without using the remove/eject utility, or the device fails. The command tells the driver to immediately cease all interaction with the device because the device is no longer attached to the system and to cancel any pending I/O requests.

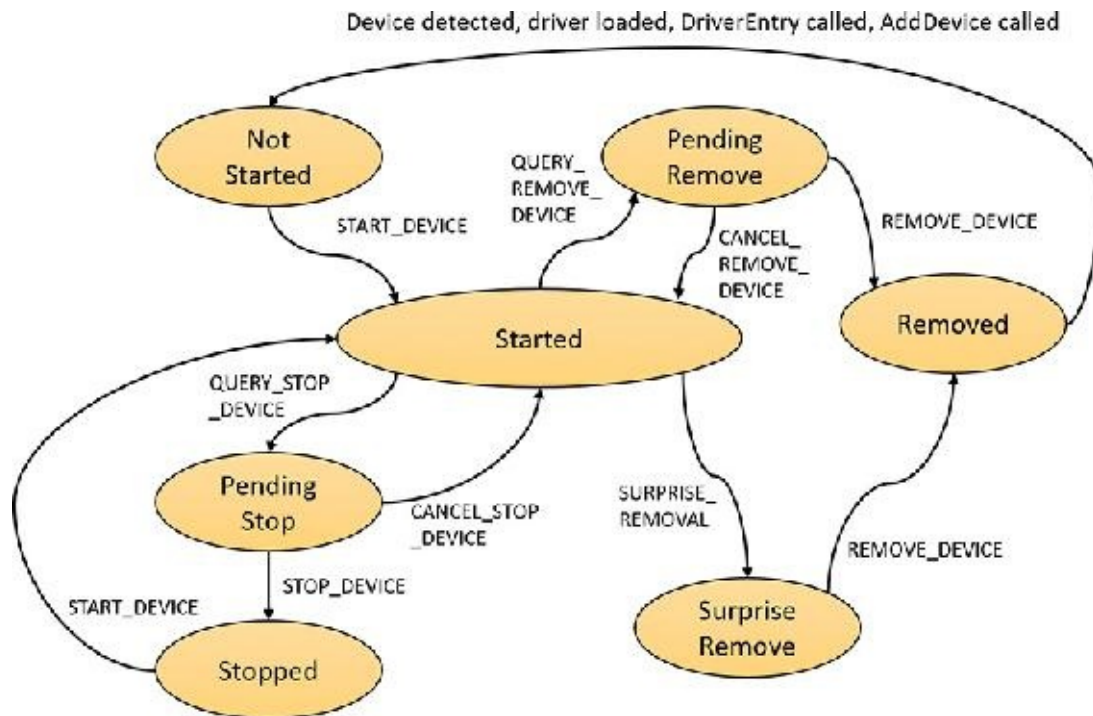


FIGURE 6-41 Device plug-and-play state transitions.

Plug-and-play driver installation

If the PnP manager encounters a device for which no driver is installed, it relies on the user-mode PnP manager to guide the installation process. If the device is detected during the system boot, a devnode is defined for the device, but the loading process is postponed until the user-mode PnP manager starts. (The user-mode PnP manager service is implemented in Umpnpgmr.dll hosted in a standard Svchost.exe instance.)

The components involved in a driver's installation are shown in [Figure 6-42](#). Dark-shaded objects in the figure correspond to components generally supplied by the system, whereas lighter-shaded objects are those included in a driver's installation files. First, a bus driver informs the PnP manager of a device it enumerates using a Device ID (1). The PnP manager checks the registry for the presence of a corresponding function driver, and when it doesn't find one, it informs the user-mode PnP manager (2) of the new device by its Device ID. The user-mode PnP manager first tries to perform an automatic install without user intervention. If the installation process involves the posting of dialog boxes that require user interaction and the currently logged-on user has administrator privileges, the user-mode PnP manager launches the Rundll32.exe application (the same application that hosts classic .cpl Control Panel utilities) to execute the Hardware Installation Wizard (3) (%SystemRoot%\System32\Newdev.dll). If the currently logged-on user doesn't have administrator privileges (or if no user is logged on) and the installation of the device requires user interaction, the user-mode PnP manager defers the installation until a privileged user logs on. The Hardware Installation Wizard uses Setupapi.dll and CfgMgr32.dll (configuration manager) API functions to locate INF files that correspond to drivers that are compatible with the detected device. This process might involve having the user insert installation media containing a vendor's INF files, or the wizard might locate a suitable INF file in the driver store (%SystemRoot%\System32\DriverStore) that contains drivers that ship with Windows or others that are downloaded through Windows Update. Installation is performed in two steps. In the first, the third-party driver developer imports the driver package into the driver store, and in the second, the system performs the actual installation, which is always done through the %SystemRoot%\System32\Drvinst.exe process.

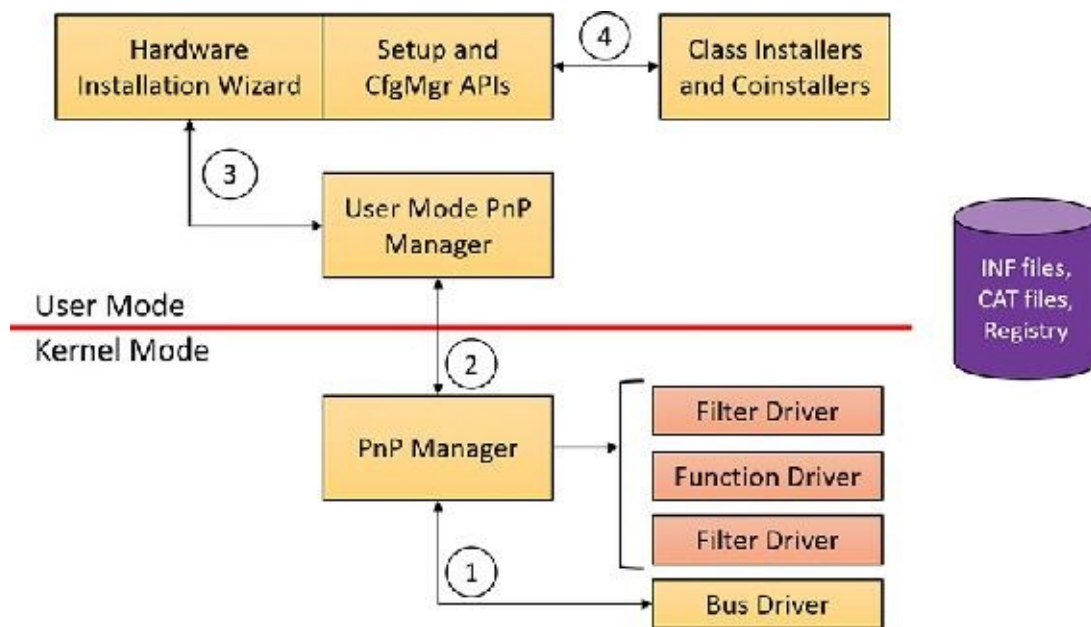


FIGURE 6-42 Driver installation components.

To find drivers for the new device, the installation process gets a list of hardware IDs (discussed earlier) and compatible IDs from the bus driver. Compatible IDs are more generic—for example a USB mouse from a specific vendor might have a special button that does something unique, but a compatible ID for a generic mouse can utilize a more generic driver that ships with Windows if the specific driver is not available and at least provide the basic, common functionality of a mouse.

These IDs describe all the various ways the hardware might be identified in a driver installation file (INF). The lists are ordered so that the most specific description of the hardware is listed first. If matches are found in multiple INFs, the following points apply:

- More-precise matches are preferred over less-precise matches.
- Digitally signed INFs are preferred over unsigned ones.
- Newer signed INFs are preferred over older signed ones.



Note

If a match is found based on a compatible ID, the Hardware Installation wizard can prompt for media in case a more up-to-date driver came with the hardware.

The INF file locates the function driver's files and contains instructions that fill in the driver's enumeration and class keys in the registry, copy required files, and the INF file might direct the Hardware Installation Wizard to (4) launch class or device co-installer DLLs that perform class-specific or device-specific installation steps, such as displaying configuration dialog boxes that let the user specify settings for a device. Finally, when the drivers that make up a devnode load, the device/driver stack is built (5).

EXPERIMENT: Looking at a driver's INF file

When a driver or other software that has an INF file is installed, the system copies its INF file to the %SystemRoot%\Inf directory. One file that will always be there is Keyboard.inf because it's the INF file for the keyboard class driver. View its contents by opening it in Notepad and you should see something like this (anything after a semicolon is a comment):

[Click here to view code image](#)

```
;
; KEYBOARD.INF  -- This file contains descriptions of Keyboard class
devices
;
;
; Copyright (c) Microsoft Corporation.  All rights reserved.
```

```
[Version]
Signature    ="$Windows NT$"
Class        =Keyboard
ClassGUID    ={4D36E96B-E325-11CE-BFC1-08002BE10318}
Provider     =%MSFT%
DriverVer=06/21/2006,10.0.10586.0
```

```
[SourceDisksNames]
3426=windows cd
```

```
[SourceDisksFiles]
i8042prt.sys      = 3426
kbdclass.sys      = 3426
kbdhid.sys        = 3426
...
```

An INF has the classic INI format, with sections in square brackets and underneath are key/value pairs separated by an equal sign. An INF is not “executed” from start to end sequentially; instead, it's built more like a tree, where certain values point to sections with the value name where execution continues. (Consult the WDK for the details.)

If you search the file for .sys, you'll come across sections that direct the user-mode PnP manager to install the i8042prt.sys and kbdclass.sys drivers:

```
...
[i8042prt_CopyFiles]
i8042prt.sys,,,0x100

[KbdClass.CopyFiles]
kbdclass.sys,,,0x100
...
```

Before installing a driver, the user-mode PnP manager checks the system's driver-signing policy. If the

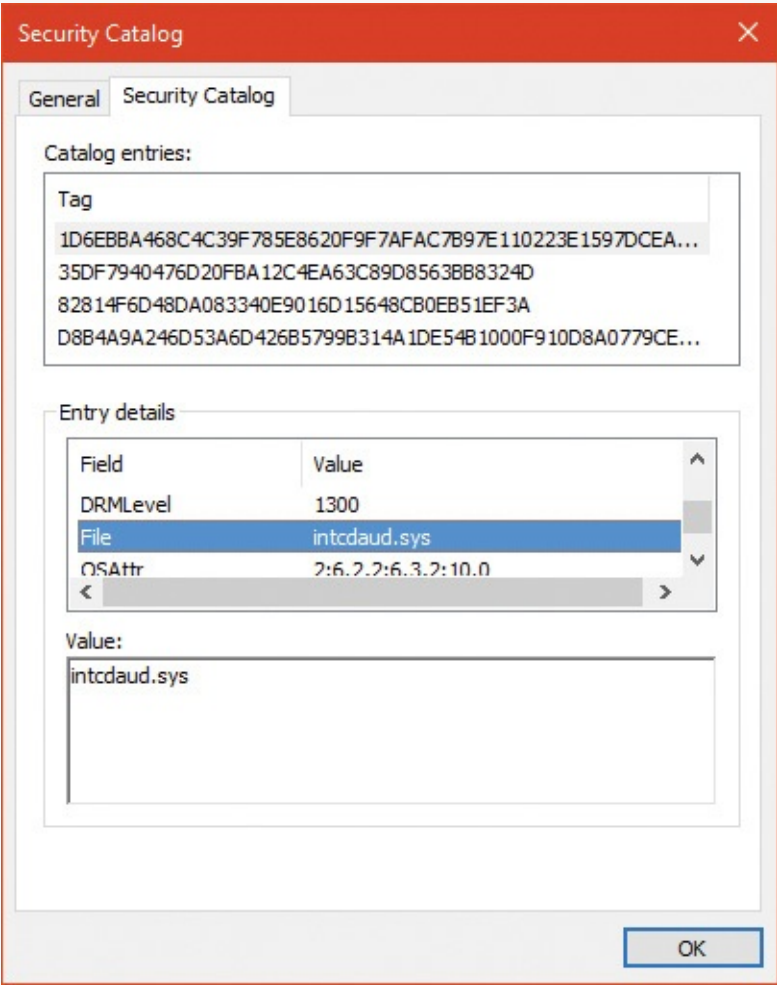
settings specify that the system should block or warn of the installation of unsigned drivers, the user-mode PnP manager checks the driver’s INF file for an entry that locates a catalog (a file that ends with the .cat extension) containing the driver’s digital signature.

Microsoft’s WHQL tests the drivers included with Windows and those submitted by hardware vendors. When a driver passes the WHQL tests, it is “signed” by Microsoft. This means that WHQL obtains a *hash*, or unique value representing the driver’s files, including its image file, and then cryptographically signs it with Microsoft’s private driver-signing key. The signed hash is stored in a catalog file and included on the Windows installation media or returned to the vendor that submitted the driver for inclusion with its driver.

EXPERIMENT: Viewing catalog files

When you install a component such as a driver that includes a catalog file, Windows copies the catalog file to a directory under %SystemRoot%\System32\Catroot. Navigate to that directory in Explorer, and you’ll find a subdirectory that contains .cat files. For example, Nt5.cat and Nt5ph.cat store the signatures and page hashes for Windows system files.

If you open one of the catalog files, a dialog box appears with two pages. The page labeled “General” shows information about the signature on the catalog file, and the Security Catalog page has the hashes of the components that are signed with the catalog file. This screenshot of a catalog file for an Intel audio driver shows the hash for the audio driver SYS file. Other hashes in the catalog are associated with the various support DLLs that ship with the driver.



As it installs a driver, the user-mode PnP manager extracts the driver’s signature from its catalog file, decrypts the signature using the public half of Microsoft’s driver-signing private/public key pair, and

compares the resulting hash with a hash of the driver file it's about to install. If the hashes match, the driver is verified as having passed WHQL testing. If a driver fails the signature verification, the user-mode PnP manager acts according to the settings of the system driver-signing policy, either failing the installation attempt, warning the user that the driver is unsigned, or silently installing the driver.



Note

Drivers installed using setup programs that manually configure the registry and copy driver files to a system and driver files that are dynamically loaded by applications aren't checked for signatures by the PnP manager's signing policy. Instead, they are checked by the kernel-mode code-signing policy described in Chapter 8 in Part 2. Only drivers installed using INF files are validated against the PnP manager's driver-signing policy.



Note

The user-mode PnP manager also checks whether the driver it's about to install is on the *protected driver list* maintained by Windows Update and, if so, blocks the installation with a warning to the user. Drivers that are known to have incompatibilities or bugs are added to the list and blocked from installation.

General driver loading and installation

The preceding section showed how drivers for hardware devices are discovered and loaded by the PnP manager. These drivers mostly load “on demand,” meaning such a driver is not loaded unless needed—a device that the driver is responsible for enters the system; conversely, if all devices managed by a driver are removed, the driver will be unloaded.

More generally, the Software key in the registry holds settings for drivers (as well as Windows Services). Although services are managed within the same registry key, they are user-mode programs and have no connection to kernel drivers (although the Service Control Manager can be used to load both services and device drivers). This section focuses on drivers; for a complete treatment of services, see Chapter 9 in Part 2.

Driver loading

Each subkey under the Software key (HKLM\System\CurrentControlSet\Services) holds a set of values that control some static aspects of a driver (or service). One such value, `ImagePath`, was encountered already when we discussed the loading process of PnP drivers. [Figure 6-36](#) shows an example of a driver key and [Table 6-8](#) summarizes the most important values in a driver's Software key (see Chapter 9 in Part 2 for a complete list).

The `Start` value indicates the phase in which a driver (or service) is loaded. There are two main differences between device drivers and services in this regard:

- Only device drivers can specify `Start` values of boot-start (0) or system-start (1). This is because at these phases, no user mode exists yet, so services cannot be loaded.
- Device drivers can use the `Group` and `Tag` values (not shown in [Table 6-8](#)) to control the order of loading within a phase of the boot, but unlike services, they can't specify `DependOnGroup` or

DependOnService values (see Chapter 9 in Part 2 for more details).

Value Name	Description
ImagePath	This is the path to the driver's image file (SYS).
Type	This indicates whether this key represents a service or a driver. A value of 1 means a driver and a value of 2 means a file system (or filter) driver. Values of 16 (0x10) and 32 (0x20) mean a service. See Chapter 9 in Part 2 for more information.
Start	This indicates when the driver should load. The options are as follows: 0 (SERVTCF_ROOT_START) The driver is loaded by the boot loader. 1 (SERVTCF_SYSTEM_START) The driver is loaded after the executive is initialized. 2 (SERVTCF_AUTO_START) The driver is loaded by the service control manager. 3 (SERVTCF_DEMAND_START) The driver is loaded on demand. 4 (SERVTCF_DISABLED) The driver is not loaded.

TABLE 6-8 Important values in a driver's registry key

Chapter 11, “Startup and shutdown, in Part 2 describes the phases of the boot process and explains that a driver **Start** value of 0 means that the operating system loader loads the driver. A **Start** value of 1 means that the I/O manager loads the driver after the executive subsystems have finished initializing. The I/O manager calls driver initialization routines in the order that the drivers load within a boot phase. Like Windows services, drivers use the **Group** value in their registry key to specify which group they belong to; the registry value `HKLM\SYSTEM\CurrentControlSet\Control\ServiceGroupOrder\List` determines the order that groups are loaded within a boot phase.

A driver can further refine its load order by including a **Tag** value to control its order within a group. The I/O manager sorts the drivers within each group according to the **Tag** values defined in the drivers' registry keys. Drivers without a tag go to the end of the list in their group. You might assume that the I/O manager initializes drivers with lower-number tags before it initializes drivers with higher-number tags, but such isn't necessarily the case. The registry key `HKLM\SYSTEM\CurrentControlSet\Control\GroupOrderList` defines tag precedence within a group; with this key, Microsoft and device-driver developers can take liberties with redefining the integer number system.



Note

The use of **Group** and **Tag** is reminiscent from the early Windows NT days. These tags are rarely used in practice. Most drivers should not have dependencies on other drivers (only on kernel libraries linked to the driver, such as `NDIS.sys`).

Here are the guidelines by which drivers set their **Start** value:

- Non-Plug and Play drivers set their **Start** value to reflect the boot phase they want to load in.
- Drivers, including both Plug and Play and non-Plug and Play drivers, that must be loaded by the boot loader during the system boot specify a **Start** value of boot-start (0). Examples include system bus drivers and the boot file-system driver.
- A driver that isn't required for booting the system and that detects a device that a system bus driver can't enumerate specifies a **Start** value of system-start (1). An example is the serial port driver, which informs the PnP manager of the presence of standard PC serial ports that were detected by **Setup** and recorded in the registry.
- A non-Plug and Play driver or file-system driver that doesn't have to be present when the system boots specifies a **Start** value of auto-start (2). An example is the Multiple Universal Naming

Convention (UNC) Provider (MUP) driver, which provides support for UNC-based path names to remote resources (for example, \\RemoteComputerName\SomeShare).

- Plug and Play drivers that aren't required to boot the system specify a `Start` value of demand-start (3). Examples include network adapter drivers.

The only purpose that the `Start` values for Plug and Play drivers and drivers for enumerable devices have is to ensure that the operating system loader loads the driver—if the driver is required for the system to boot successfully. Beyond that, the PnP manager's device enumeration process determines the load order for Plug and Play drivers.

Driver installation

As we've seen, Plug and Play drivers require an INF file for installation. The INF includes the hardware device IDs this driver can handle and the instructions for copying files and setting registry values. Other type of drivers (such as file system drivers, file system filters and network filters) require an INF as well, which includes a unique set of values for the particular type of driver.

Software-only drivers (such as the one Process Explorer uses) can use an INF for installation, but don't have to. These can be installed by a call to the `CreateService` API (or use a tool such as `sc.exe` that wraps it), as Process Explorer does after extracting its driver from a resource within the executable (if running with elevated permissions). As the API name suggests, it's used to install services as well as drivers. The arguments to `CreateService` indicate whether it's installing a driver or a service, the `Start` value and other parameters (see the Windows SDK documentation for the details). Once installed, a call to `StartService` loads the driver (or service), calling `DriverEntry` (for a driver) as usual.

A software-only driver typically creates a device object with a name its clients know. For example, Process Explorer creates a device named `PROCEXP152` that is then used by Process Explorer in a `CreateFile` call, followed by calls such as `DeviceIoControl` to send requests to the driver (turned into IRPs by the I/O manager). [Figure 6-43](#) shows the Process Explorer object symbolic link (using the WinObj Sysinternals tool) in the `\GLOBAL??` directory (recall that the names in this directory are accessible to user mode clients) that's created by Process Explorer the first time it's running with elevated privileges. Notice that it points to the real device object under the `\Device` directory and it has the same name (which is not a requirement).

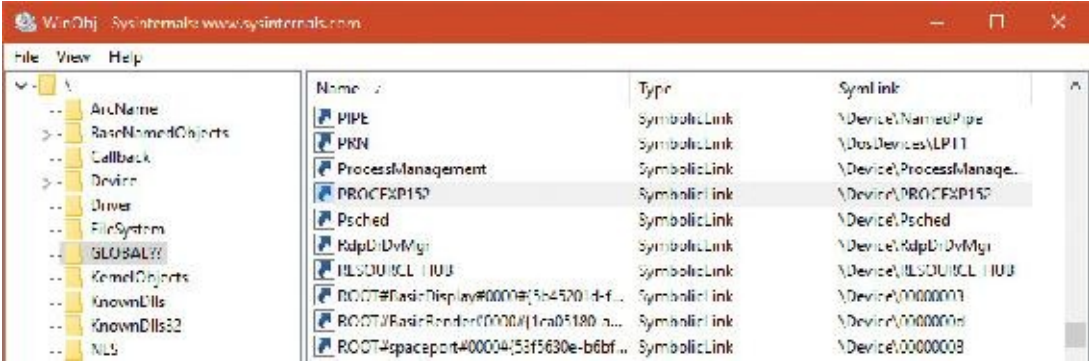


FIGURE 6-43 Process Explorer's symbolic link and device name.

The Windows Driver Foundation

The Windows Driver Foundation (WDF) is a framework for developing drivers that simplifies common tasks such as handing Plug and Play and Power IRPs correctly. WDF includes the Kernel-Mode Driver Framework (KMDF) and the User-Mode Driver Framework (UMDF). WDF is now open source and can be found at <https://github.com/Microsoft/Windows-Driver-Frameworks>. Table 6-9 shows the Windows version support (for Windows 7 and later) for KMDF. Table 6-10 shows the same for UMDF.

KMDF Version	Release Method	Included in Windows	Drivers Using It Run On
1.9	Windows 7 WDK	Windows 7	Windows XP and later
1.11	Windows 8 WDK	Windows 8	Windows Vista and later
1.13	Windows 8.1 WDK	Windows 8.1	Windows 8.1 and later
1.15	Windows 10 WDK	Windows 10	Windows 10, Windows Server 2016
1.17	Windows 10 version 1511 WDK	Windows 10 version 1511	Windows 10 version 1511 and later, Windows Server 2016
1.19	Windows 10 version 1607 WDK	Windows 10 version 1607	Windows 10 version 1607 and later, Windows Server 2016

TABLE 6-9 KMDF versions

UMDF Version	Release Method	Included in Windows	Drivers Using It Run On
1.9	Windows 7 WDK	Windows 7	Windows XP and later
1.11	Windows 8 WDK	Windows 8	Windows Vista and later
2.0	Windows 8.1 WDK	Windows 8.1	Windows 8.1 and later
2.15	Windows 10 WDK	Windows 10	Windows 10 and later, Windows Server 2016
2.17	Windows 10 version 1511 WDK	Windows 10 version 1511	Windows 10 version 1511 and later, Windows Server 2016
2.19	Windows 10 version 1607 WDK	Windows 10 version 1607	Windows 10 version 1607, Windows Server 2016

TABLE 6-10 UMDF versions

Windows 10 introduced the concept of Universal Drivers, briefly described in Chapter 2, “[System architecture](#).” These drivers use a common set of DDIs implemented in multiple editions of Windows 10—from IoT Core, to Mobile, to desktops. Universal drivers can be built with KMDF, UMDF 2.x, or WDM. Building such drivers is relatively easy with the aid of Visual Studio, where the Target Platform setting is set to Universal. Any DDI that is outside the boundaries of Universal will be flagged by the compiler.

UMDF versions 1.x used a COM based model for programming drivers, which is a very different programming model than KMDF, which is using object-based C. UMDF 2 has been aligned with KMDF and provides an almost identical API, reducing overall cost associated with WDF driver development; in fact, UMDF 2.x drivers can be converted to KMDF if the need arises with little work. UMDF 1.x will not be discussed in this book; consult the WDK for more information.

The following sections discuss KMDF and UMDF, which essentially behave in a consistent manner, no matter the exact OS they’re running on.

Kernel-Mode Driver Framework

We've already discussed some details about the Windows Driver Foundation (WDF) in [Chapter 2](#). In this section, we'll take a deeper look at the components and functionality provided by the kernel-mode part of the framework, KMDF. Note that this section will only briefly touch on some of the core architecture of KMDF. For a much more complete overview on the subject, please refer to the Windows Driver Kit documentation.



Note

Most of the details presented in this section are the same for UMDF 2.x, with the exceptions discussed in the next section.

Structure and operation of a KMDF driver

First, let's look at which kinds of drivers or devices are supported by KMDF. In general, any WDM-conformant driver should be supported by KMDF, as long as it performs standard I/O processing and IRP manipulation. KMDF is not suitable for drivers that don't use the Windows kernel API directly but instead perform library calls into existing port and class drivers. These types of drivers cannot use KMDF because they only provide callbacks for the actual WDM drivers that do the I/O processing. Additionally, if a driver provides its own dispatch functions instead of relying on a port or class driver, IEEE 1394, ISA, PCI, PCMCIA, and SD Client (for Secure Digital storage devices) drivers can also use KMDF.

Although KMDF provides an abstraction on top of WDM, the basic driver structure shown earlier also generally applies to KMDF drivers. At their core, KMDF drivers must have the following functions:

- **An initialization routine** Like any other driver, a KMDF driver has a `DriverEntry` function that initializes the driver. KMDF drivers initiate the framework at this point and perform any configuration and initialization steps that are part of the driver or part of describing the driver to the framework. For non-Plug and Play drivers, this is where the first device object should be created.
- **An add-device routine** KMDF driver operation is based on events and callbacks (described shortly), and the `EvtDriverDeviceAdd` callback is the single most important one for PnP devices because it receives notifications when the PnP manager in the kernel enumerates one of the driver's devices.
- **One or more `EvtIo*` routines** Similar to a WDM driver's dispatch routines, these callback routines handle specific types of I/O requests from a particular device queue. A driver typically creates one or more queues in which KMDF places I/O requests for the driver's devices. These queues can be configured by request type and dispatching type.

The simplest KMDF driver might need to have only an initialization and add-device routine because the framework will provide the default, generic functionality that's required for most types of I/O processing, including power and Plug and Play events. In the KMDF model, *events* refer to run-time states to which a driver can respond or during which a driver can participate. These events are not related to the synchronization primitives (synchronization is discussed in Chapter 8 in Part 2), but are internal to the framework.

For events that are critical to a driver's operation, or that need specialized processing, the driver registers a given callback routine to handle this event. In other cases, a driver can allow KMDF to perform a default, generic action instead. For example, during an eject event (`EvtDeviceEject`), a

driver can choose to support ejection and supply a callback or to fall back to the default KMDF code that will tell the user that the device does not support ejection. Not all events have a default behavior, however, and callbacks must be provided by the driver. One notable example is the EvtDriverDeviceAdd event just described that is at the core of any Plug and Play driver.

EXPERIMENT: Displaying KMDF and UMDF 2 drivers

The Wdfkd.dll extension that ships with the Debugging Tools for Windows package provides many commands that can be used to debug and analyze KMDF drivers and devices (instead of using the built-in WDM-style debugging extension, which may not offer the same kind of WDF-specific information). You can display installed KMDF drivers with the !wdfkd.wdfldr debugger command. In the following example, the output from a Windows 10 32-bit Hyper-V virtual machine is shown, displaying the built-in drivers that are installed.

[Click here to view code image](#)

```
lkd> !wdfkd.wdfldr
-----
KMDF Drivers
-----
LoadedModuleList      0x870991ec
-----
LIBRARY_MODULE  0x8626aad8
  Version        v1.19
  Service        \Registry\Machine\System\CurrentControlSet\Services\W
  ImageName      Wdf01000.sys
  ImageAddress   0x87000000
  ImageSize      0x8f000
  Associated Clients: 25

  ImageName      Ver    WdfGlobals
FxGlobals  ImageAddress  ImageSize
  umpass.sys    v1.15  0xa1ae53f8  0xa1ae52f8
0x9e5f0000  0x00008000
  peauth.sys    v1.7   0x95e798d8  0x95e797d8
0x9e400000  0x000ba000
  mslldp.sys    v1.15  0x9aed1b50  0x9aed1a50
0x8e300000  0x00014000
  vmgid.sys     v1.15  0x97d0fd08  0x97d0fc08
0x8e260000  0x00008000
  monitor.sys   v1.15  0x97cf7e18  0x97cf7d18
0x8e250000  0x0000c000
  tsusbhub.sys  v1.15  0x97cb3108  0x97cb3008
0x8e4b0000  0x0001b000
  NdisVirtualBus.sys  v1.15  0x8d0fc2b0  0x8d0fc1b0
0x87a90000  0x00009000
  vmgencounter.sys  v1.15  0x8d0fefd0  0x8d0feed0
0x87a80000  0x00008000
  intelppm.sys  v1.15  0x8d0f4cf0  0x8d0f4bf0
```

0x87a50000	0x00021000			
vms3cap.sys		v1.15	0x8d0f5218	0x8d0f5118
0x87a40000	0x00008000			
netvsc.sys		v1.15	0x8d11ded0	0x8d11ddd0
0x87a20000	0x00019000			
hyperkbd.sys		v1.15	0x8d114488	0x8d114388
0x87a00000	0x00008000			
dmvsc.sys		v1.15	0x8d0ddb28	0x8d0dda28
0x879a0000	0x0000c000			
umbus.sys		v1.15	0x8b86ffd0	0x8b86fed0
0x874f0000	0x00011000			
CompositeBus.sys		v1.15	0x8b869910	0x8b869810
0x87df0000	0x0000d000			
cdrom.sys		v1.15	0x8b863320	0x8b863220
0x87f40000	0x00024000			
vmstorfl.sys		v1.15	0x8b2b9108	0x8b2b9008
0x87c70000	0x0000c000			
EhStorClass.sys		v1.15	0x8a9dacf8	0x8a9dabf8
0x878d0000	0x00015000			
vmbus.sys		v1.15	0x8a9887c0	0x8a9886c0
0x82870000	0x00018000			
vdrvroot.sys		v1.15	0x8a970728	0x8a970628
0x82800000	0x0000f000			
msisadrv.sys		v1.15	0x8a964998	0x8a964898
0x873c0000	0x00008000			
WindowsTrustedRTPProxy.sys		v1.15	0x8a1f4c10	0x8a1f4b10
0x87240000	0x00008000			
WindowsTrustedRT.sys		v1.15	0x8a1f1fd0	0x8a1f1ed0
0x87220000	0x00017000			
intelpep.sys		v1.15	0x8a1ef690	0x8a1ef590
0x87210000	0x0000d000			
acpiex.sys		v1.15	0x86287fd0	0x86287ed0
0x870a0000	0x00019000			

Total: 1 library loaded

If UMDF 2.x drivers were loaded, they would have been shown as well. This is one of the benefits of the UMDF 2.x library (see the UMDF section later in this chapter for more on this subject).

Notice that the KMDF library is implemented in Wdf01000.sys, which is the current version 1.x of KMDF. Future versions of KMDF may have a major version of 2 and will be implemented in another kernel module, Wdf02000.sys. This future module can live side by side with the version 1.x module, each loaded with the drivers that compiled against it. This ensures isolation and independence between drivers built against different KMDF major version libraries.

KMDF object model

The KMDF object model is object-based, with properties, methods and events, implemented in C, much like the model for the kernel, but it does not make use of the object manager. Instead, KMDF manages its own objects internally, exposing them as handles to drivers and keeping the actual data structures opaque. For each object type, the framework provides routines to perform operations on the object (called *methods*), such as `WdfDeviceCreate`, which creates a device. Additionally, objects can have specific data fields or members that can be accessed by `Get/Set` (used for modifications that should never fail) or `Assign/Retrieve` APIs (used for modifications that can fail), which are called *properties*. For example, the `WdfInterruptGetInfo` function returns information on a given interrupt object (`WDFINTERRUPT`).

Also unlike the implementation of kernel objects, which all refer to distinct and isolated object types, KMDF objects are all part of a hierarchy—most object types are bound to a parent. The root object is the `WDFDRIVER` structure, which describes the actual driver. The structure and meaning is analogous to the `DRIVER_OBJECT` structure provided by the I/O manager, and all other KMDF structures are children of it. The next most important object is `WDFDEVICE`, which refers to a given instance of a detected device on the system, which must have been created with `WdfDeviceCreate`. Again, this is analogous to the `DEVICE_OBJECT` structure that's used in the WDM model and by the I/O manager. [Table 6-11](#) lists the object types supported by KMDF.

Object	Type	Description
Child list	WDFCHILDLIST	This is a list of child <code>WDFDEVICE</code> objects associated with the device. It is used only by bus drivers.
Collection	WDFCOLLECTION	This is a list of objects of a similar type, such as a group of <code>WDFDEVICE</code> objects being filtered.
Deferred Procedure Call	WDFDPC	This is an instance of a DPC object.
Device	WDFDEVICE	This is an instance of a device.
DMA common buffer	WDFCOMMONBUFFER	This is a region of memory that a device and driver can access for DMA.
DMA enabler	WDFDMACNABLER	This enables DMA on a given channel for a driver.
DMA transaction	WDFDMATRANSACTION	This is an instance of a DMA transaction.
Driver	WDFDRIVER	This is an object for the driver. It represents the driver, its parameters, and its callbacks, among other items.
File	WDFFILEOBJECT	This is an instance of a file object that can be used as a channel for communication between an application and the driver.
Generic object	WDFOBJECT	This allows driver-defined custom data to be wrapped inside the framework's object data model as an object.
Interrupt	WDFINTERRUPT	This is an instance of an interrupt that the driver must handle.
I/O queue	WDFQUEUE	This represents a given I/O queue.
I/O request	WDFREQUEST	This represents a given request on a <code>WDFQUEUE</code> .
I/O target	WDFIOTARGET	This represents the device stack being targeted by a given <code>WDFREQUEST</code> .
Look-aside list	WDFLOOKASIDE	This describes an executive look-aside list. (See Chapter 5.)
Memory	WDFMEMORY	This describes a region of paged or nonpaged pool.
Registry key	WDFKEY	This describes a registry key.
Resource list	WDFCMRESLIST	This identifies the hardware resources assigned to a <code>WDFDEVICE</code> .
Resource range list	WDFCMRESLIST	This identifies a given possible hardware resource range for a <code>WDFDEVICE</code> .
Resource requirements list	WDFIORESREQLIST	This contains an array of <code>WDFIORESLIST</code> objects describing all possible resource ranges for a <code>WDFDEVICE</code> .
Spinlock	WDFSPINLOCK	This describes a spinlock.
String	WDFSTRING	This describes a Unicode string structure.
Timer	WDFTIMER	This describes an executive timer. (See Chapter 8 in Part 2 for more information.)
USB device	WDFUSBDEVICE	This identifies the one instance of a USB device.
USB interface	WDFUSBINTERFACE	This identifies one interface on the given <code>WDFUSBDEVICE</code> .
USB pipe	WDFUSBPIPE	This identifies a pipe to an endpoint on a given <code>WDFUSBINTERFACE</code> .
Wait lock	WDFWAITLOCK	This represents a kernel dispatcher event object.
WMI instance	WDFWMIINSTANCE	This represents a WMI data block for a given <code>WDFWMI_PROVIDER</code> .
WMI provider	WDFWMI_PROVIDER	This describes the WMI schema for all the <code>WDFWMIINSTANCE</code> objects supported by the driver.
Work item	WDFWORKITEM	This describes an executive work item.

TABLE 6-11 KMDF object types

For each of these objects, other KMDF objects can be attached as children. Some objects have only one or two valid parents, while others can be attached to any parent. For example, a `WDFINTERRUPT` object must be associated with a given `WDFDEVICE`, but a `WDFSPINLOCK` or `WDFSTRING` object can have any object as a parent. This allows for fine-grained control over their validity and usage and the reduction of global state variables. [Figure 6-44](#) shows the entire KMDF object hierarchy.

Attribute	Description
ContextSizeOverride	This is the size of the object context area.
ContextTypeInfo	This is the type of the object context area.
EvtCleanupCallback	This is the callback to notify the driver of the object's cleanup before deletion. (References may still exist.)
EvtDestroyCallback	This is the callback to notify the driver of the object's imminent deletion. (The reference count will be 0.)
ExecutionLevel	This describes the maximum IRQL at which the callbacks may be invoked by KMDF.
ParentObject	This identifies the parent of the object.
SynchronizationScope	Specifies whether callbacks should be synchronized with the parent, a queue, a device, or nothing.

TABLE 6-12 KMDF object attributes

KMDF I/O model

The KMDF I/O model follows the WDM mechanisms discussed earlier in this chapter. In fact, you can even think of the framework itself as a WDM driver, since it uses kernel APIs and WDM behavior to abstract KMDF and make it functional. Under KMDF, the framework driver sets its own WDM-style IRP dispatch routines and takes control of all IRPs sent to the driver. After being handled by one of three KMDF I/O handlers (described shortly), it then packages these requests in the appropriate KMDF objects, inserts them in the appropriate queues (if required), and performs driver callback if the driver is interested in those events. [Figure 6-45](#) describes the flow of I/O in the framework.

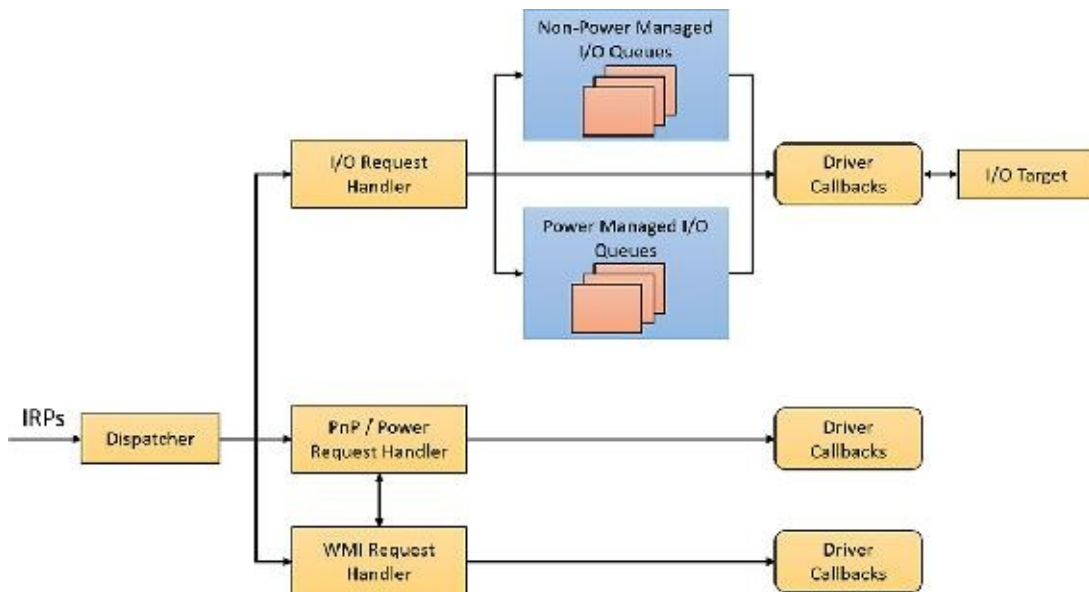


FIGURE 6-45 KMDF I/O flow and IRP processing.

Based on the IRP processing discussed previously for WDM drivers, KMDF performs one of the following three actions:

- It sends the IRP to the I/O handler, which processes standard device operations.
- It sends the IRP to the PnP and power handler that processes these kinds of events and notifies other drivers if the state has changed.
- It sends the IRP to the WMI handler, which handles tracing and logging.

These components then notify the driver of any events it registered for, potentially forward the request to another handler for further processing, and then complete the request based on an internal handler action or as the result of a driver call. If KMDF has finished processing the IRP but the request itself has still not been fully processed, KMDF will take one of the following actions:

- For bus drivers and function drivers, it completes the IRP with `STATUS_INVALID_DEVICE_REQUEST`.
- For filter drivers, it forwards the request to the next lower driver.

I/O processing by KMDF is based on the mechanism of queues (`WDFQUEUE`, not the `KQUEUE` object discussed earlier in this chapter). KMDF queues are highly scalable containers of I/O requests (packaged as `WDFREQUEST` objects) and provide a rich feature set beyond merely sorting the pending I/Os for a given device. For example, queues track currently active requests and support I/O cancellation, I/O concurrency (the ability to perform and complete more than one I/O request at a time), and I/O synchronization (as noted in the list of object attributes in [Table 6-12](#)). A typical KMDF driver creates at least one queue (if not more) and associates one or more events with each queue, as well as some of the following options:

- The callbacks registered with the events associated with this queue.
- The power management state for the queue. KMDF supports both power-managed and non-power managed queues. For the former, the I/O handler wakes up the device when required (and when possible), arms the idle timer when the device has no I/Os queued up, and calls the driver's I/O cancellation routines when the system is switching away from a working state.
- The dispatch method for the queue. KMDF can deliver I/Os from a queue in sequential, parallel, or manual mode. Sequential I/Os are delivered one at a time (KMDF waits for the driver to complete the previous request), while parallel I/Os are delivered to the driver as soon as possible. In manual mode, the driver must manually retrieve I/Os from the queue.
- Whether the queue can accept zero-length buffers, such as incoming requests that don't actually contain any data.



Note

The dispatch method only affects the number of requests that can be active inside a driver's queue at one time. It does not determine whether the event callbacks themselves will be called concurrently or serially. That behavior is determined through the synchronization scope object attribute described earlier. Therefore, it is possible for a parallel queue to have concurrency disabled but still have multiple incoming requests.

Based on the mechanism of queues, the KMDF I/O handler can perform various tasks upon receiving a create, close, cleanup, write, read, or device control (IOCTL) request:

- For create requests, the driver can request to be immediately notified through the `EvtDeviceFileCreate` callback event, or it can create a non-manual queue to receive create requests. It must then register an `EvtIoDefault` callback to receive the notifications. Finally, if none of these methods are used, KMDF will simply complete the request with a success code, meaning that by default, applications will be able to open handles to KMDF drivers that don't supply their own code.
- For cleanup and close requests, the driver will be immediately notified through `EvtFileCleanup` and `EvtFileClose` callbacks, if registered. Otherwise, the framework will simply complete with a success code.
- For write, read, and IOCTL requests, the flow shown in [Figure 6-46](#) applies.

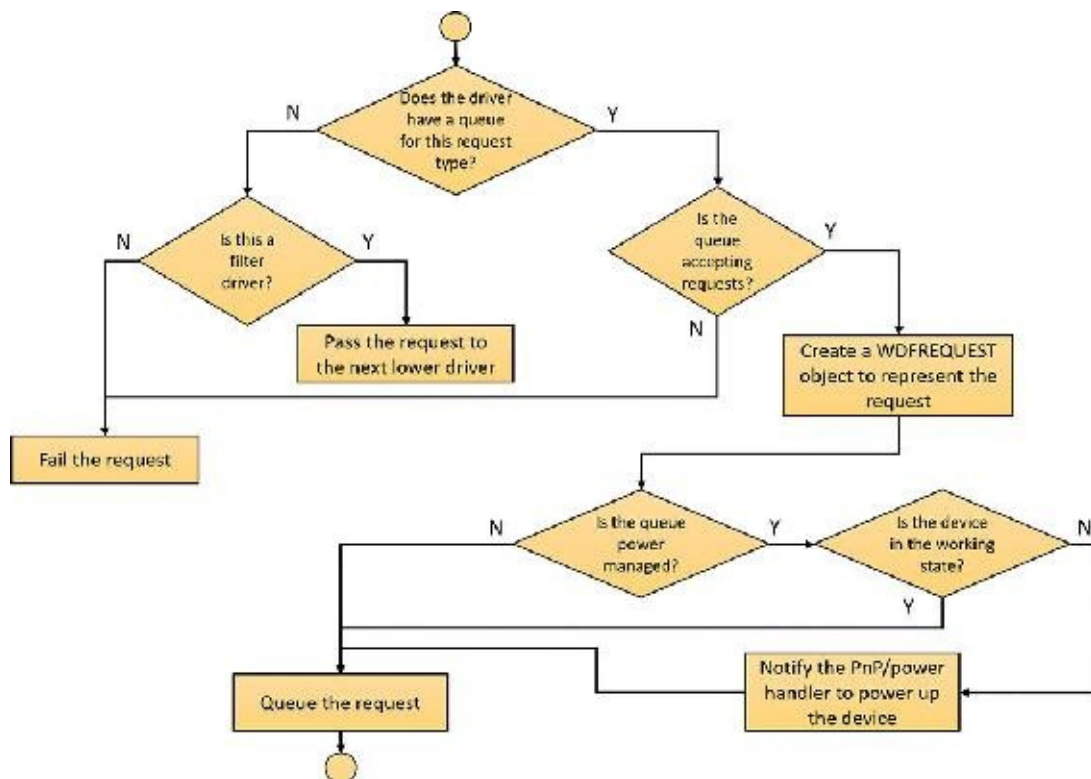


FIGURE 6-46 Handling read, write, and IOCTL I/O requests by KMDF.

User-Mode Driver Framework

Windows includes a growing number of drivers that run in user mode, using the User-Mode Driver Framework (UMDF), which is part of the WDF. UMDF version 2 is aligned with KMDF in terms of object model, programming model and I/O model. The frameworks are not identical, however, because of some of the inherent differences between user mode and kernel mode. For example, some KMDF objects listed in [Table 6-12](#) don't exist in UMDF, including `WDFCHILDLIST`, DMA-related objects, `WDFLOOKASIDELIST` (look-aside lists can be allocated only in kernel mode), `WDFIORESLIST`, `WDFIORESREQLIST`, `WDFDPC`, and WMI objects. Still, most KMDF objects and concepts apply equally to UMDF 2.x.

UMDF provides several advantages over KMDF:

- UMDF drivers execute in user mode, so any unhandled exception crashes the UMDF host process, but not the entire system.
- The UMDF host process runs with the Local Service account, which has very limited privileges on the local machine and only anonymous access in network connections. This reduces the security attack surface.
- Running in user mode means the IRQL is always 0 (`PASSIVE_LEVEL`). Thus, the driver can always take page faults and use kernel dispatcher objects for synchronization (events, mutexes, and so on).
- Debugging UMDF drivers is easier than debugging KMDF drivers because the debugging setup does not require two separate machines (virtual or physical).

The main drawback to UMDF is increased latency because of the kernel/user transitions and communication required (as described shortly). Also, some types of drivers, such as drivers for high-speed PCI devices, are simply not meant to execute in user mode and thus cannot be written with UMDF.

UMDF is designed specifically to support protocol device classes, which refers to devices that all use

the same standardized, generic protocol and offer specialized functionality on top of it. These protocols currently include IEEE 1394 (FireWire), USB, Bluetooth, human interface devices (HIDs) and TCP/IP. Any device running on top of these buses (or connected to a network) is a potential candidate for UMDF. Examples include portable music players, input devices, cell phones, cameras and webcams, and so on. Two other users of UMDF are SideShow-compatible devices (auxiliary displays) and the Windows Portable Device (WPD) Framework, which supports USB-removable storage (USB bulk transfer devices). Finally, as with KMDF, it's possible to implement software-only drivers, such as for a virtual device, in UMDF.

Unlike KMDF drivers, which run as driver objects representing a SYS image file, UMDF drivers run in a driver host process (running the image %SystemRoot%\System32\WUDFHost.exe), similar to a service-hosting process. The host process contains the driver itself, the User-Mode Driver Framework (implemented as a DLL), and a run-time environment (responsible for I/O dispatching, driver loading, device-stack management, communication with the kernel, and a thread pool).

As in the kernel, each UMDF driver runs as part of a stack. This can contain multiple drivers that are responsible for managing a device. Naturally, because user-mode code can't access the kernel address space, UMDF also includes components that allow this access to occur through a specialized interface to the kernel. This is implemented by a kernel-mode side of UMDF that uses ALPC—essentially an efficient inter-process communication mechanism to talk to the run-time environment in the user-mode driver host processes. (See Chapter 8 in Part 2 for more information on ALPC.) [Figure 6-47](#) shows the architecture of the UMDF driver model.

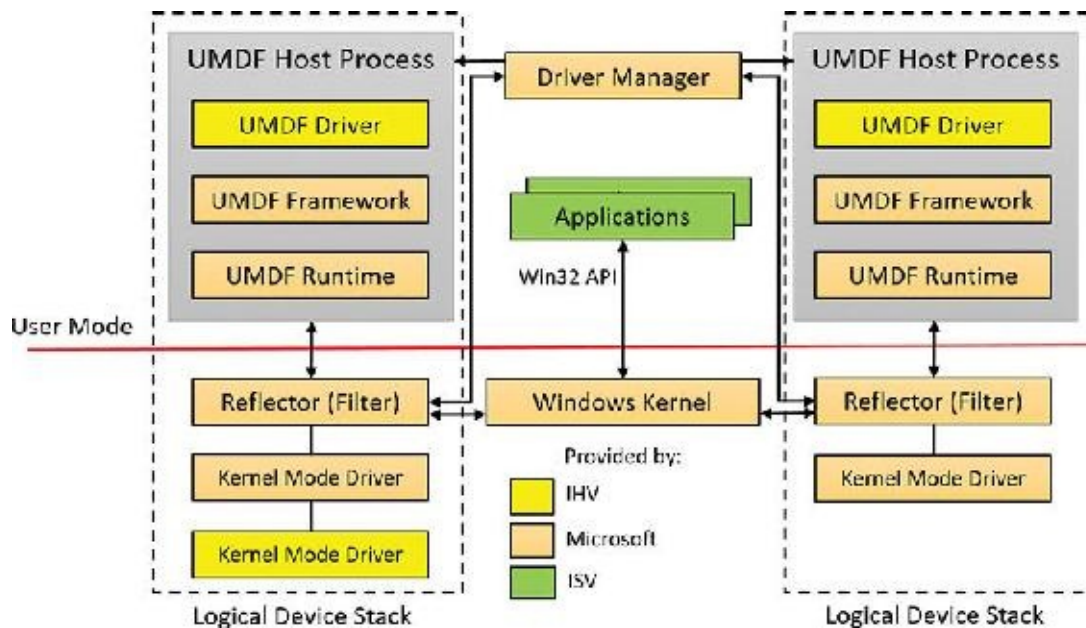


FIGURE 6-47 UMDF architecture.

[Figure 6-47](#) shows two different device stacks that manage two different hardware devices, each with a UMDF driver running inside its own driver host process. From the diagram, you can see that the following components comprise the architecture:

- **Applications** These are the clients of the drivers. They are standard Windows applications that use the same APIs to perform I/Os as they would with a KMDF-managed or WDM-managed device. Applications don't know (nor care) that they're talking to a UMDF-based device, and the calls are still sent to the kernel's I/O manager.
- **Windows kernel (I/O manager)** Based on the application I/O APIs, the I/O manager builds the IRPs for the operations, just like for any other standard device.

- **Reflector** The reflector is what makes UMDF “tick.” It is a standard WDM filter driver (%SystemRoot%\System32\Drivers\WUDFRd.Sys) that sits at the top of the device stack of each device that is being managed by a UMDF driver. The reflector is responsible for managing the communication between the kernel and the user-mode driver host process. IRPs related to power management, Plug and Play, and standard I/O are redirected to the host process through ALPC. This enables the UMDF driver to respond to the I/Os and perform work, as well as be involved in the Plug and Play model, by providing enumeration, installation, and management of its devices. Finally, the reflector is responsible for keeping an eye on the driver host processes by making sure they remain responsive to requests within an adequate time to prevent drivers and applications from hanging.
- **Driver manager** The driver manager is responsible for starting and quitting the driver host processes, based on which UMDF-managed devices are present, and also for managing information on them. It is also responsible for responding to messages coming from the reflector and applying them to the appropriate host process (such as reacting to device installation). The driver manager runs as a standard Windows service implemented in %SystemRoot%\System32\WUDFsvc.dll (hosted in a standard Svchost.exe), and is configured for automatic startup as soon as the first UMDF driver for a device is installed. Only one instance of the driver manager runs for all driver host processes (as is always the case with services), and it must always be running to allow UMDF drivers to work.
- **Host process** The host process provides the address space and run-time environment for the actual driver (WUDFHost.exe). Although it runs in the local service account, it is not actually a Windows service and is not managed by the SCM—only by the driver manager. The host process is also responsible for providing the user-mode device stack for the actual hardware, which is visible to all applications on the system. Currently, each device instance has its own device stack, which runs in a separate host process. In the future, multiple instances may share the same host process. Host processes are child processes of the driver manager.
- **Kernel-mode drivers** If specific kernel support for a device that is managed by a UMDF driver is needed, it is also possible to write a companion kernel-mode driver that fills that role. In this way, it is possible for a device to be managed both by a UMDF and a KMDF (or WDM) driver.

You can easily see UMDF in action on your system by inserting a USB flash drive with some content on it. Run Process Explorer, and you should see a WUDFHost.exe process that corresponds to a driver host process. Switch to DLL view and scroll down until you see DLLs like the ones shown in [Figure 6-48](#).

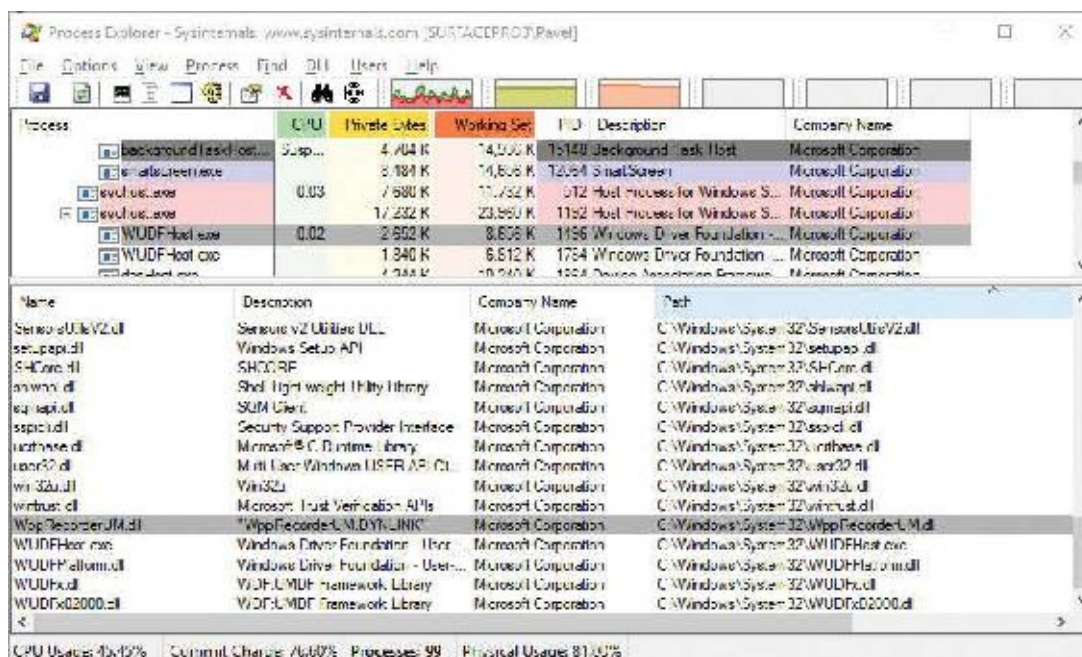


FIGURE 6-48 DLL in UMDf host process.

You can identify three main components, which match the architectural overview described earlier:

- **WUDFHost.exe** This is the UMDf host executable.
- **WUDFx02000.dll** This is the UMDf 2.x framework DLL.
- **WUDFPlatform.dll** This is the run-time environment.

The power manager

Just as Windows Plug and Play features require support from a system's hardware, its power-management capabilities require hardware that complies with the Advanced Configuration and Power Interface (ACPI) specification, which is now part of the Unified Extensible Firmware Interface (UEFI). (The ACPI spec is available at <http://www.uefi.org/specifications>.)

The ACPI standard defines various power levels for a system and for devices. The six system power states are described in [Table 6-13](#). They are referred to as S0 (fully on or working) through S5 (fully off). Each state has the following characteristics:

- **Power consumption** This is the amount of power the system consumes.
- **Software resumption** This is the software state from which the system resumes when moving to a "more on" state.
- **Hardware latency** This is the length of time it takes to return the system to the fully on state.

State	Power Consumption	Software Resumption	Hardware Latency
S0 (fully on)	Maximum	Not applicable	None
S1 (sleeping)	Less than S0, more than S2	System resumes where it left off (returns to S0)	Less than 2 seconds
S2 (sleeping)	Less than S1, more than S3	System resumes where it left off (returns to S0)	2 or more seconds
S3 (sleeping)	Less than S2; processor is off	System resumes where it left off (returns to S0)	Same as S2
S4 (hibernating)	Trickle current to power button and wake circuitry	System restarts from saved hibernation file and resumes where it left off before hibernation (returns to S0)	Long and undefined
S5 (fully off)	Trickle current to power button	System boot	Long and undefined

TABLE 6-13 System power-state definitions

As noted in [Table 6-13](#), states S1 through S4 are sleeping states, in which the system appears to be off because of reduced power consumption. However, in these sleeping states, the system retains enough information—either in memory or on disk—to move to S0. For states S1 through S3, enough power is required to preserve the contents of the computer’s memory so that when the transition is made to S0 (when the user or a device wakes up the computer), the power manager continues executing where it left off before the suspend.

When the system moves to S4, the power manager saves the compressed contents of memory to a hibernation file named Hiberfil.sys, which is large enough to hold the uncompressed contents of memory, in the root directory of the system volume (hidden file). (Compression is used to minimize disk I/O and to improve hibernation and resume-from-hibernation performance.) After it finishes saving memory, the power manager shuts off the computer. When a user subsequently turns on the computer, a normal boot process occurs, except that the boot manager checks for and detects a valid memory image stored in the hibernation file. If the hibernation file contains the saved system state, the boot manager launches %SystemRoot%\System32\Winresume.exe, which reads the contents of the file into memory, and then resumes execution at the point in memory that is recorded in the hibernation file.

On systems with hybrid sleep enabled, a user request to put the computer to sleep will actually be a combination of both the S3 state and the S4 state. While the computer is put to sleep, an emergency hibernation file will also be written to disk. Unlike typical hibernation files, which contain almost all active memory, the emergency hibernation file includes only data that could not be paged in at a later time, making the suspend operation faster than a typical hibernation (because less data is written to disk). Drivers will then be notified that an S4 transition is occurring, allowing them to configure themselves and save state just as if an actual hibernation request had been initiated. After this point, the system is put in the normal sleep state just like during a standard sleep transition. However, if the power goes out, the system is now essentially in an S4 state—the user can power on the machine, and Windows will resume from the emergency hibernation file.



Note

You can disable hibernation completely and gain some disk space by running `powercfg /h off` from an elevated command prompt.

The computer never directly transitions between states S1 and S4 (because that requires code execution, but the CPU is off in these states); instead, it must move to state S0 first. As illustrated in [Figure 6-49](#), when the system is moving from any of states S1 through S5 to state S0, it’s said to be

waking, and when it's transitioning from state S0 to any of states S1 through S5, it's said to be *sleeping*.

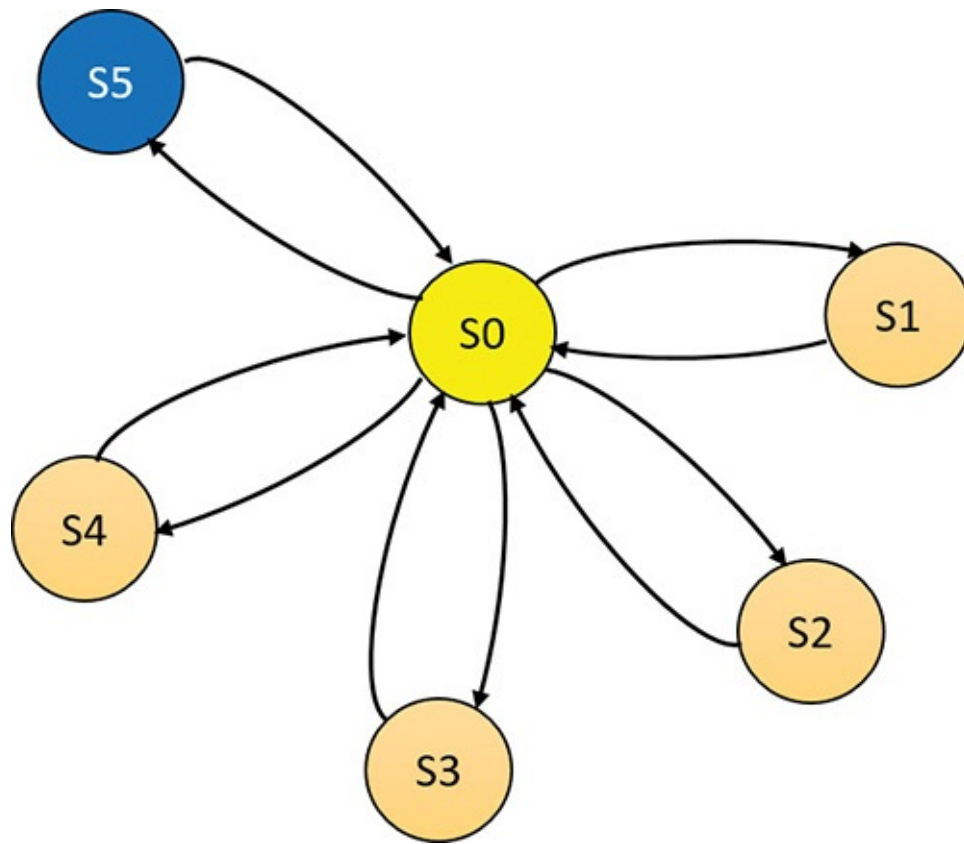


FIGURE 6-49 System power-state transitions.

EXPERIMENT: System power states

To view the supported power states, open an elevated command window and type in the command **powercfg /a**. You'll see output similar to the following:

[Click here to view code image](#)

```
C:\WINDOWS\system32>powercfg /a
```

The following sleep states are available on this system:

Standby (S3)

Hibernate

Fast Startup

The following sleep states are not available on this system:

Standby (S1)

The system firmware does not support this standby state.

Standby (S2)

The system firmware does not support this standby state.

Standby (S0 Low Power Idle)

The system firmware does not support this standby state.

Hybrid Sleep

The hypervisor does not support this standby state.

Notice that the standby state is S3 and hibernation is available. Let's turn off hibernation and re-execute the command:

[Click here to view code image](#)

```
C:\WINDOWS\system32>powercfg /h off
```

```
C:\WINDOWS\system32>powercfg /a
```

The following sleep states are available on this system:
Standby (S3)

The following sleep states are not available on this system:
Standby (S1)

The system firmware does not support this standby state.

Standby (S2)

The system firmware does not support this standby state.

Hibernate

Hibernation has not been enabled.

Standby (S0 Low Power Idle)

The system firmware does not support this standby state.

Hybrid Sleep

Hibernation is not available.

The hypervisor does not support this standby state.

Fast Startup

Hibernation is not available.

For devices, ACPI defines four power states, from D0 through D3. State D0 is fully on, while state D3 is fully off. The ACPI standard leaves it to individual drivers and devices to define the meanings of states D1 and D2, except that state D1 must consume an amount of power less than or equal to that consumed in state D0, and when the device is in state D2, it must consume power less than or equal to that consumed in D1.

Windows 8 (and later) splits the D3 state into two sub-states, D3-hot and D3-cold. In D3-hot state, the device is mostly turned off, but is not disconnected from its main power source, and its parent bus controller can detect the presence of the device on the bus. In D3-cold, the main power source is removed from the device, and the bus controller cannot detect the device. This state provides another opportunity for saving power. [Figure 6-50](#) shows the device states and the possible state transitions.

[Figure 6-50](#) shows the device states and the possible state transitions.

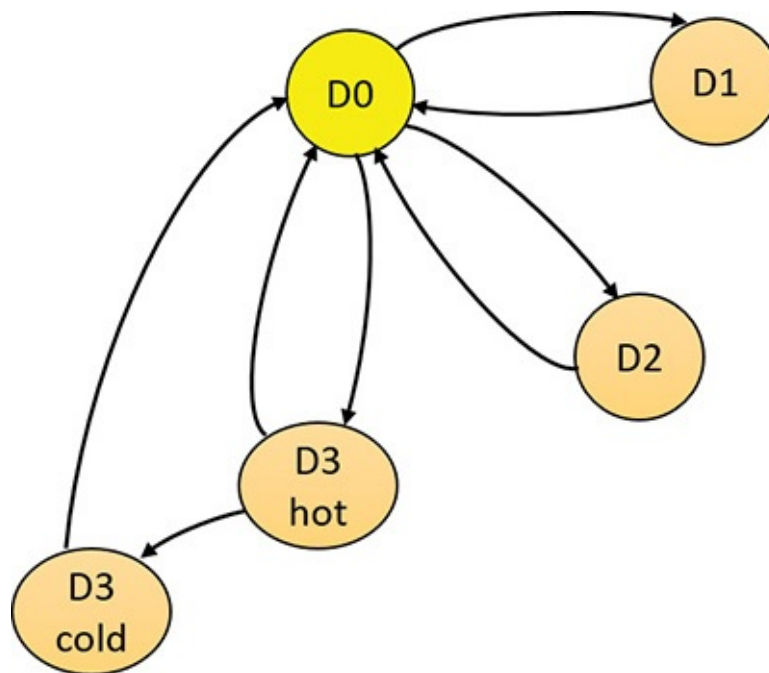


FIGURE 6-50 Device power-state transitions.

Before Windows 8, devices could only reach D3-hot state while the system is fully on (S0). The transition to D3-cold was implicit when the system went into a sleep state. Starting with Windows 8, a device’s power state can be set to D3-cold while the system is fully on. The driver that controls the device cannot put the device into D3-cold state directly; instead, it can put the device into D3-hot state, and then, depending on other devices on the same bus entering D3-hot states, the bus driver and firmware may decide to move all the devices to D3-cold. The decision whether to move the devices to D3-cold states depends on two factors: first, the actual ability of the bus driver and firmware, and second on the driver that must enable the transition to D3-cold either by specifying that in the installation INF file or by calling the `SetD3DColdSupport` function dynamically.

Microsoft, in conjunction with the major hardware OEMs, has defined a series of power management reference specifications that specify the device power states that are required for all devices in a particular class (for the major device classes: display, network, SCSI, and so on). For some devices, there’s no intermediate power state between fully on and fully off, which results in these states being undefined.

Connected Standby and Modern Standby

You may have noticed in the experiment above another system state called **Standby** (S0 Low Power Idle). Although not an official ACPI state, it is a variant of S0 known as *Connected Standby* on Windows 8.x and later enhanced in Windows 10 (desktop and mobile editions) and called *Modern Standby*. The “normal” standby state (S3 above) is sometimes referred to as *Legacy Standby*.

The main problem with Legacy Standby is that the system is not working, and therefore, for example, the user receives an email, the system can’t pick that up without waking to S0, which may or may not happen, depending on configuration and device capabilities. Even if the system wakes up to get that email, it won’t go immediately to sleep again. Modern Standby solves both issues.

Systems that support Modern Standby normally go into this state when the system is instructed to go to Standby. The system is technically still at S0, meaning the CPU is active and code can execute. However, desktop processes (non-UWP apps) are suspended, as well as UWP apps (most are not in the foreground and suspended anyway), but background tasks created by UWP apps are allowed to execute. For example,

an email client would have a background task that periodically polls for new messages.

Being in Modern Standby also means that the system is able to wake to full S0 very quickly, sometimes referred to as *Instant On*. Note that not all systems support Modern Standby, as it depends on the chipset and other platform components (as can be seen in the last experiment, the system on which the experiment ran does not support Modern Standby and thus supports Legacy Standby).

For more information on Modern Standby, consult the Windows Hardware documentation at [https://msdn.microsoft.com/en-us/library/windows/hardware/mt282515\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/mt282515(v=vs.85).aspx).

Power manager operation

Windows power-management policy is split between the power manager and the individual device drivers. The power manager is the owner of the system power policy. This ownership means the power manager decides which system power state is appropriate at any given point, and when a sleep, hibernation, or shutdown is required, the power manager instructs the power-capable devices in the system to perform appropriate system power-state transitions.

The power manager decides when a system power-state transition is necessary by considering several factors:

- System activity level
- System battery level
- Shutdown, hibernate, or sleep requests from applications
- User actions, such as pressing the power button
- Control Panel power settings

When the PnP manager performs device enumeration, part of the information it receives about a device is its power-management capabilities. A driver reports whether its devices support device states D1 and D2 and, optionally, the latencies, or times required, to move from states D1 through D3 to D0. To help the power manager determine when to make system power-state transitions, bus drivers also return a table that implements a mapping between each of the system power states (S0 through S5) and the device power states that a device supports.

The table lists the lowest possible device power state for each system state and directly reflects the state of various power planes when the machine sleeps or hibernates. For example, a bus that supports all four device power states might return the mapping table shown in [Table 6-14](#). Most device drivers turn their devices completely off (D3) when leaving S0 to minimize power consumption when the machine isn't in use. Some devices, however, such as network adapter cards, support the ability to wake up the system from a sleeping state. This ability, along with the lowest device power state in which the capability is present, is also reported during device enumeration.

System Power State	Device Power State
S0 (fully on)	D0 (fully on)
S1 (sleeping)	D1
S2 (sleeping)	D2
S3 (sleeping)	D2
S4 (hibernating)	D3 (fully off)
S5 (fully off)	D3 (fully off)

TABLE 6-14 An example of system-to-device power mappings

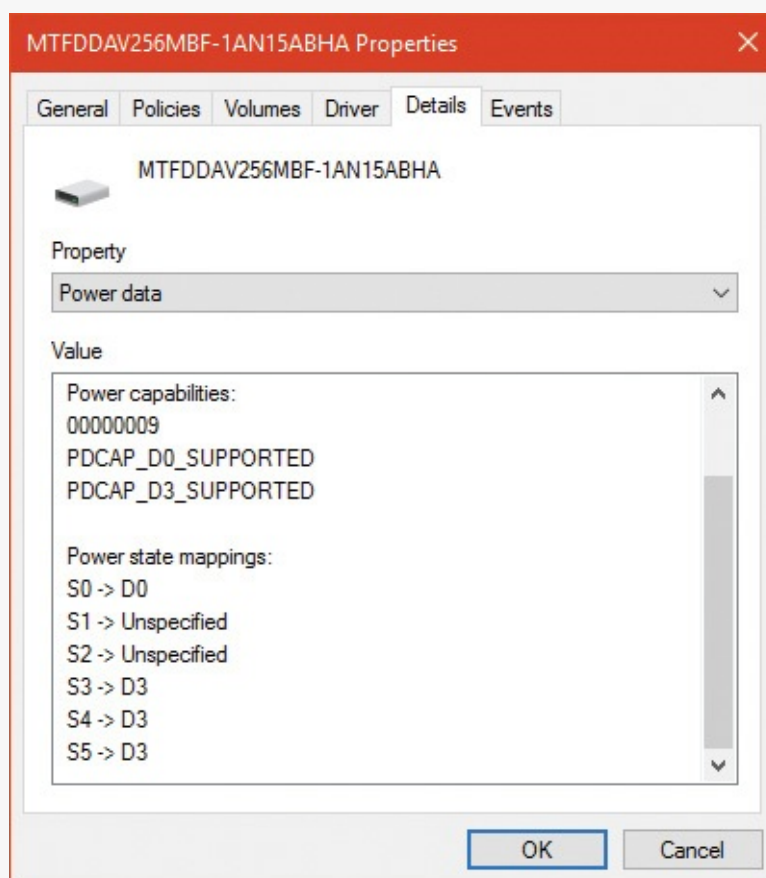
Driver power operation

When the power manager decides to make a transition between system power states, it sends power commands to a driver's power dispatch routine (`IRP_MJ_POWER`). More than one driver can be responsible for managing a device, but only one of the drivers is designated as the device power-policy owner. This is typically the driver that manages the FDO. This driver determines, based on the system state, a device's power state. For example, if the system transitions between state S0 and S3, a driver might decide to move a device's power state from D0 to D1.

Instead of directly informing the other drivers that share the management of the device of its decision, the device power-policy owner asks the power manager, via the `PowerRequestPowerIrp` function, to tell the other drivers by issuing a device power command to their power dispatch routines. This behavior enables the power manager to control the number of power commands that are active on a system at any given time. For example, some devices in the system might require a significant amount of current to power up. The power manager ensures that such devices aren't powered up simultaneously.

EXPERIMENT: Viewing a driver's power mappings

You can use Device Manager to see a driver's system power state-to-driver power state mappings. To do so, open the Properties dialog box for a device, click the **Details** tab, click the **Property** drop-down list, and choose **Power Data**. The Properties dialog box also displays the current power state of the device, the device-specific power capabilities that it provides, and the power states from which it can wake the system:



Many power commands have corresponding query commands. For example, when the system is moving to a sleep state, the power manager will first ask the devices on the system whether the transition is acceptable. A device that is busy performing time-critical operations or interacting with device hardware might reject the command, which results in the system maintaining its current system power-state setting.

EXPERIMENT: Viewing the system power capabilities and policy

You can view a computer's system power capabilities by using the `!pocaps` kernel debugger command. Here's the output of the command when run on an x64 Windows 10 laptop:

[Click here to view code image](#)

```
lkd> !pocaps
PopCapabilities @ 0xffffffff8035a98ce60
  Misc Supported Features:  PwrButton SlpButton Lid S3 S4 S5
HiberFile FullWake
VideoDim
Processor Features:        Thermal
Disk Features:
Battery Features:          BatteriesPresent
  Battery 0 - Capacity:    0  Granularity:          0
```

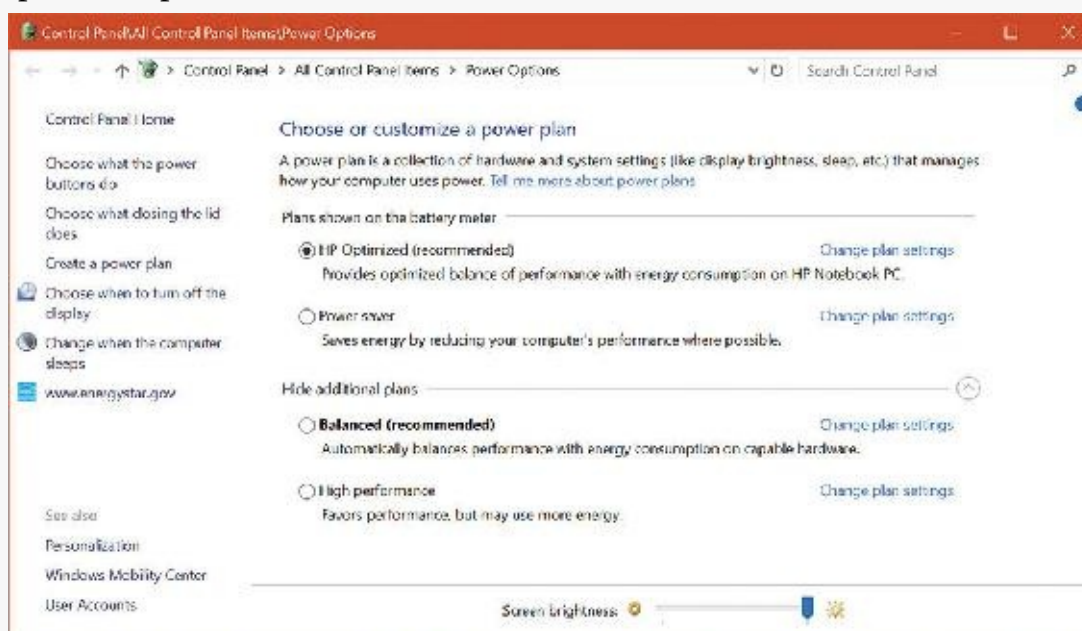
Battery 1 - Capacity:	0	Granularity:	0
Battery 2 - Capacity:	0	Granularity:	0

Wake Caps

Ac OnLine Wake:	Sx
Soft Lid Wake:	Sx
RTC Wake:	S4
Min Device Wake:	Sx
Default Wake:	Sx

The Misc Supported Features line reports that, in addition to S0 (fully on), the system supports system power states of S3, S4 and S5 (it doesn't implement S1 or S2) and has a valid hibernation file to which it can save system memory when it hibernates (state S4).

The Power Options page, which you open by selecting **Power Options** in the Control Panel, lets you configure various aspects of the system's power policy. The exact properties you can configure depend on the system's power capabilities.



Notice that OEMs can add power schemes. These schemes can be listed by typing the **powercfg /list** command as shown here:

[Click here to view code image](#)

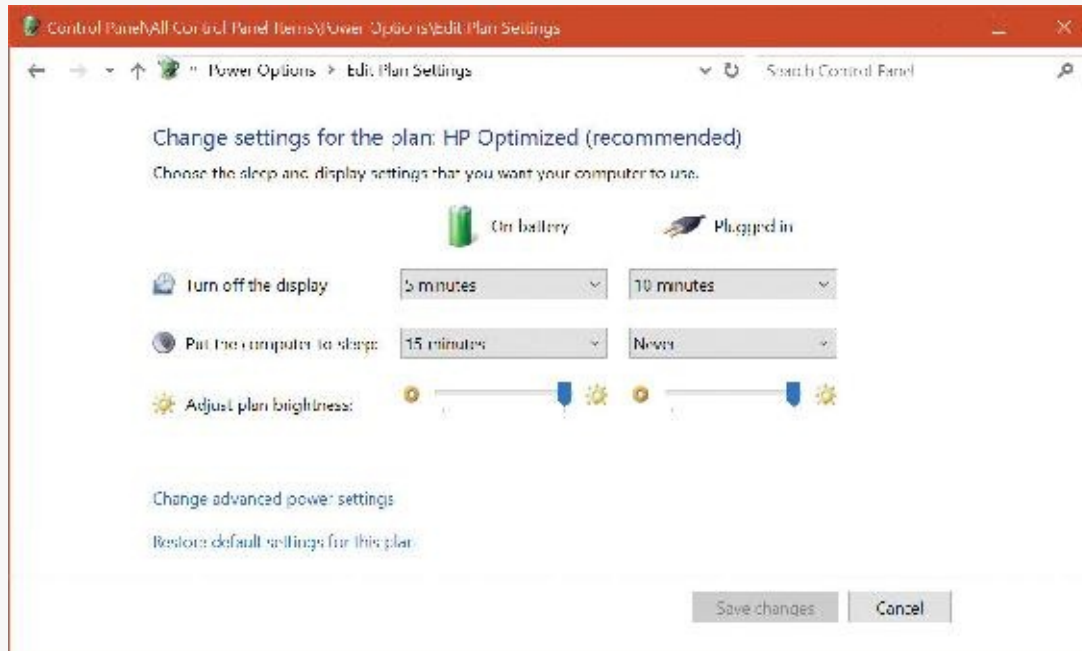
```
C:\WINDOWS\system32>powercfg /list
```

```
Existing Power Schemes (* Active)
```

```
-----
Power Scheme GUID: 381b4222-f694-41f0-9685-ff5bb260df2e (Balanced)
Power Scheme GUID: 8759706d-706b-4c22-b2ec-f91e1ef6ed38 (HP
Optimized
(recommended)) *
Power Scheme GUID: 8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c (High
performance)
Power Scheme GUID: a1841308-3541-4fab-bc81-f71556f20b4a (Power
saver)
```

By changing any of the preconfigured plan settings, you can set the idle detection timeouts that control

when the system turns off the monitor, spins down hard disks, goes to standby mode (moves to system power state S3 in the previous experiment), and hibernates (moves the system to power state S4). In addition, selecting the **Change Plan Settings** link lets you specify the power-related behavior of the system when you press the power or sleep buttons or close a laptop's lid.



The Change Advanced Power Settings link directly affects values in the system's power policy, which you can display with the `!popolicy` debugger command. Here's the output of the command on the same system:

[Click here to view code image](#)

```
lkd> !popolicy
SYSTEM_POWER_POLICY (R.1) @ 0xffffffff8035a98cc64
PowerButton:      Sleep  Flags: 00000000  Event: 00000000
SleepButton:      Sleep  Flags: 00000000  Event: 00000000
LidClose:         None   Flags: 00000000  Event: 00000000
Idle:             Sleep  Flags: 00000000  Event: 00000000
OverThrottled:    None   Flags: 00000000  Event: 00000000
IdleTimeout:      0      IdleSensitivity: 90%
MinSleep:         S3     MaxSleep:        S3
LidOpenWake:      S0     FastSleep:       S3
WinLogonFlags:    1      S4Timeout:       0
VideoTimeout:     600    VideoDim:        0
SpinTimeout:      4b0    OptForPower:     0
FanTolerance:     0%     ForcedThrottle:  0%
MinThrottle:      0%     DyanmicThrottle: None (0)
```

The first lines of the display correspond to the button behaviors specified in the Power Options Advanced Settings window. On this system, both the power and the sleep buttons put the computer in a sleep state. Closing the lid, however, does nothing. The timeout values shown near the end of the output are expressed in seconds and displayed in hexadecimal notation. The values reported here directly correspond to the settings configured in the Power Options window. For example, the video timeout is 600, meaning the monitor turns off after 600 seconds (because of a bug in the debugging tools used here, it's displayed in decimal), or 10 minutes. Similarly, the hard disk spin-down timeout is 0x4b0, which

corresponds to 1200 seconds, or 20 minutes.

Driver and application control of device power

In addition to responding to power manager commands related to system power-state transitions, a driver can unilaterally control the device power state of its devices. In some cases, a driver might want to reduce the power consumption of a device it controls if the device is left inactive for a period of time. Examples include monitors that support a dimmed mode and disks that support spin-down. A driver can either detect an idle device itself or use facilities provided by the power manager. If the device uses the power manager, it registers the device with the power manager by calling the `PoRegisterDeviceForIdleDetection` function.

This function informs the power manager of the timeout values to use to detect whether a device is idle and, if so, the device power state that the power manager should apply. The driver specifies two timeouts: one to use when the user has configured the computer to conserve energy and the other to use when the user has configured the computer for optimum performance. After calling `PoRegisterDeviceForIdleDetection`, the driver must inform the power manager, by calling the `PoSetDeviceBusy` or `PoSetDeviceBusyEx` functions, whenever the device is active, and then register for idle detection again to disable and re-enable it as needed. The `PoStartDeviceBusy` and `PoEndDeviceBusy` APIs are available as well, which simplify the programming logic required to achieve the behavior just described.

Although a device has control over its own power state, it does not have the ability to manipulate the system power state or to prevent system power transitions from occurring. For example, if a badly designed driver doesn't support any low-power states, it can choose to remain on or turn itself completely off without hindering the system's overall ability to enter a low-power state—this is because the power manager only *notifies* the driver of a transition and doesn't ask for *consent*. Drivers do receive a power query IRP (`IRP_MN_QUERY_POWER`) when the system is about to transition to a lower power state. The driver may veto the request, but the power manager does not have to comply; it may delay transition if possible (e.g., the device is running on a battery that is not critically low); transition to hibernation, however, can never fail.

Although drivers and the kernel are chiefly responsible for power management, applications are also allowed to provide their input. User-mode processes can register for a variety of power notifications, such as when the battery is low or critically low, when the machine has switched from DC (battery) to AC (adapter/charger) power, or when the system is initiating a power transition. Applications can never veto these operations, and they can have up to two seconds to clean up any state necessary before a sleep transition.

Power management framework

Starting with Windows 8, the kernel provides a framework for managing power states of individual components (sometimes called *functions*) within a device. For example, suppose an audio device has playback and recording components, but if the playback component is active and the recording component is not, it would be beneficial to put the recording component into a lower power state. The power management framework (PoFx) provides an API that drivers can use to indicate their components' power states and requirements. All components must support the fully-on state, identified as F0. Higher-number F-states indicate lower power states that a component may be in, where each higher F-state represents a lower power consumption and higher transition time to F0. Note that F-state management has meaning only when the device is in power state D0, because it's not working at all in higher D-states.

The power policy owner of the device (typically the FDO) must register with PoFx by calling the `PoFxRegisterDevice` function. The driver passes along the following information in the call:

- The number of components within the device.
- A set of callbacks the driver can implement to be notified by PoFx when various events occur, such as switching to active or idle state, switching the device to D0 state and sending power control codes (see the WDK for more information).
- For each component, the number of F-states it supports.
- For each component, the deepest F-state from which the component can wake.
- For each component, for each F-state, the time required to return from this state to F0, the minimum amount of time the component can be in this F-state to make the transition worthwhile, and the nominal power the component consumes in this F-state. Or, it can be set to indicate that the power consumption is negligible and is not worth considering when PoFx decides to wake several components simultaneously.

PoFx uses this information—combined with information from other devices and system-wide power state information, such as the current power profile—to make intelligent decisions for which power F-state a particular component should be in. The challenge is to reconcile two conflicting objectives: first, ensuring that an idle component consumes as little power as possible, and second, making sure a component can transition to the F0 state quickly enough so that the component is perceived as always on and always connected.

The driver must notify PoFx when a component needs to be active (F0 state) by calling `PoFxActivate-Component`. Sometime after this call, the corresponding callback is invoked by PoFx, indicating to the driver that the component is now at F0. Conversely, when the driver determines the component is not currently needed, it calls `PoFxIdleComponent` to tell PoFx, which responds by transitioning the component to a lower-power F-state and notifies the driver once it does.

Performance state management

The mechanisms just described allow a component in an idle condition (non-F0 states) to consume less power than in F0. But some components can consume less power even in state F0, related to the actual work a device is doing. For example, a graphic card may be able to use less power when showing a mostly static display, whereas it would need higher power when rendering 3D content in 60 frames per second.

In Windows 8.x, such drivers would have to implement a propriety performance state selection algorithm and notify an OS service called platform extension plug-in (PEP). PEP is specific to a particular line of processors or system on a chip (SoC). This makes the driver code tightly coupled to the

PEP.

Windows 10 extends the PoFx API for performance state management, prompting the driver code to use standard APIs and not worry about the particular PEP on the platform. For each component, PoFx provides the following types of performance states:

- A discrete number of states in the frequency (Hz), bandwidth (bits per second), or an opaque number meaningful to the driver.
- A continuous distribution of states between a minimum and maximum (frequency, bandwidth, or custom).

An example of this is for a graphic card to define a discrete set of frequencies in which it can operate, thus indirectly affecting its power consumption. Similar performance sets could be defined for its bandwidth usage, if appropriate.

To register with PoFx for performance state management, a driver must first register the device with PoFx (`PoFxRegisterDevice`) as described in the previous section. Then, the driver calls `PoFxRegisterComponentPerfStates`, passing performance details (discrete or range-based, frequency, bandwidth, or custom) and a callback when state changes actually occur.

When a driver decides that a component should change performance state, it calls `PoFxIssuePerfStateChange` or `PoFxIssuePerfStateChangeMultiple`. These calls request the PEP to place the component in the specified state (based on the provided index or value, depending on whether the set is for a discrete state or range-based). The driver may also specify that the call should be synchronous, asynchronous or “don’t care,” in which case the PEP decides. Either way, PoFx will eventually call into the driver-registered callback with the performance state, which may be the requested one, but it can also be denied by the PEP. If accepted, the driver should make the appropriate calls to its hardware to make the actual change. If the PEP denies the request, the driver may try again with a new call to one of the aforementioned functions. Only a single call can be made before the driver’s callback is invoked.

Power availability requests

Applications and drivers cannot veto sleep transitions that are already initiated. However, certain scenarios demand a mechanism for disabling the ability to initiate sleep transitions when a user is interacting with the system in certain ways. For example, if the user is currently watching a movie and the machine would normally go idle (based on a lack of mouse or keyboard input after 15 minutes), the media player application should have the capability to temporarily disable idle transitions as long as the movie is playing. You can probably imagine other power-saving measures that the system would normally undertake, such as turning off or even just dimming the screen, that would also limit your enjoyment of visual media. In legacy versions of Windows, `SetThreadExecutionState` was a user-mode API capable of controlling system and display idle transitions by informing the power manager that a user was still present on the machine. However, this API did not provide any sort of diagnostic capabilities, nor did it allow sufficient granularity for defining the availability request. Also, drivers could not issue their own requests, and even user applications had to correctly manage their threading model, because these requests were at the thread level, not at the process or system level.

Windows now supports power request objects, which are implemented by the kernel and are bona-fide object manager–defined objects. You can use the `WinObj` utility from Sysinternals (more details on this tool are in Chapter 8 in Part 2) and see the `PowerRequest` object type in the `\ObjectTypes` directory, or use the `!object` kernel debugger command on the `\ObjectTypes\PowerRequest` object type, to validate this.

Power availability requests are generated by user-mode applications through the `PowerCreateRequest` API and then enabled or disabled with the `PowerSetRequest` and `PowerClearRequest` APIs, respectively. In the kernel, drivers use `PoCreatePowerRequest`, `PoSetPowerRequest`, and `PoClearPowerRequest`. Because no handles are used, `PoDeletePowerRequest` is needed to remove the reference on the object (while user mode can simply use `CloseHandle`).

There are four kinds of requests that can be used through the Power Request API:

- **System request** This type request asks that the system not automatically go to sleep due to the idle timer (although the user can still close the lid to enter sleep, for example).
- **Display request** This type of request does the same as a system request, but for the display.
- **Away-mode request** This is a modification to the normal sleep (S3 state) behavior of Windows, which is used to keep the computer in full powered-on mode but with the display and sound card turned off, making it appear to the user as though the machine is really sleeping. This behavior is normally used only by specialized set-top boxes or media center devices when media delivery must continue even though the user has pressed a physical sleep button, for example.
- **Execution required request** This type of request (available starting with Windows 8 and Server 2012) requests a UWP app process continue execution even if normally the Process Lifecycle Manager (PLM) would have terminated it (for whatever reason); the extended length of time depends on factors such as the power policy settings. This request type is only supported for systems that support Modern Standby, otherwise this request is interpreted as a system request.

EXPERIMENT: Viewing power availability requests

Unfortunately, the power request kernel object that's created with a call such as `PowerCreateRequest` is unavailable in the public symbols. However, the `Powercfg` utility provides a way to list power requests without any need for a kernel debugger. Here's the output of the utility while playing a video and a stream audio from the web on a Windows 10 laptop:

[Click here to view code image](#)

```
C:\WINDOWS\system32>powercfg /requests
DISPLAY:
[PROCESS] \Device\HarddiskVolume4\Program
Files\WindowsApps\Microsoft.
ZuneVideo_10.16092.10311.0_x64__8wekyb3d8bbwe\Video.UI.exe
Windows Runtime Package: Microsoft.ZuneVideo_8wekyb3d8bbwe

SYSTEM:
[DRIVER] Conexant ISST Audio
(INTELAUDIO\FUNC_01&VEN_14F1&DEV_50F4&SUBSYS_103C80D3&R
EV_1001\4&1a010da&0&0001)
An audio stream is currently in use.
[PROCESS] \Device\HarddiskVolume4\Program
Files\WindowsApps\Microsoft.
ZuneVideo_10.16092.10311.0_x64__8wekyb3d8bbwe\Video.UI.exe
Windows Runtime Package: Microsoft.ZuneVideo_8wekyb3d8bbwe

AWAYMODE:
None.

EXECUTION:
None.

PERFBOOST:
None.

ACTIVELOCKSCREEN:
None.
```

The output shows six request types (as opposed to the four described previously). The last two—`perfboost` and `active lockscreen`—are declared as part of an internal power request type in a kernel header, but are otherwise currently unused.

Conclusion

The I/O system defines the model of I/O processing on Windows and performs functions that are common to or required by more than one driver. Its chief responsibilities are to create IRPs representing I/O requests and to shepherd the packets through various drivers, returning results to the caller when an I/O is complete. The I/O manager locates various drivers and devices by using I/O system objects, including driver and device objects. Internally, the Windows I/O system operates asynchronously to achieve high performance and provides both synchronous and asynchronous I/O capabilities to user-mode applications.

Device drivers include not only traditional hardware device drivers but also file-system, network, and layered filter drivers. All drivers have a common structure and communicate with each other and the I/O manager by using common mechanisms. The I/O system interfaces allow drivers to be written in a high-level language to lessen development time and to enhance their portability. Because drivers present a common structure to the operating system, they can be layered one on top of another to achieve modularity and reduce duplication between drivers. By using the Universal DDI baseline, drivers can target multiple devices and form factors with no code changes.

Finally, the role of the PnP manager is to work with device drivers to dynamically detect hardware devices and to build an internal device tree that guides hardware device enumeration and driver installation. The power manager works with device drivers to move devices into low-power states when applicable to conserve energy and prolong battery life.

The next chapter touches on one of the most important aspects of today's computer systems: security.